SOCIAL MEDIA SITE USING BLOCKCHAIN
Chain.Social

NAME: SHARON A [PES2UG23CS544]
NAME: SHREELA SUKUMAR SHETTY [PES2UG23CS557]
CLASS: 6I
BATCH: 5

Description:

Chain.Social is a decentralized social media platform built on the Ethereum Sepolia testnet that gives users ownership and control over their digital identity and content.

Unlike traditional platforms such as Twitter or Instagram, where centralized entities manage user data and content, Chain.Social uses decentralized technologies to ensure transparency and user autonomy.

Content is stored on IPFS via Pinata, while cryptographic content hashes and ownership records are permanently stored on the blockchain through smart contracts. This approach ensures data integrity while significantly reducing storage costs.

Users authenticate using their MetaMask wallet, eliminating the need for usernames and passwords. The wallet address acts as a unique identifier, and authentication is performed using cryptographic signatures.

The platform supports key social interactions such as posting, liking, commenting, following, and tipping. These interactions are recorded via smart contracts, making them transparent, verifiable, and tamper-resistant.

Chain.Social also introduces a native ERC-20 token, **CLINK**, which rewards user engagement. Users earn tokens through interactions such as receiving likes, and token balances determine influence in community-driven moderation. A public leaderboard showcases the top contributors on the platform.

Overall, Chain.Social demonstrates a secure, transparent, and decentralized alternative to traditional social media by combining blockchain, decentralized storage, and Web3 authentication.

System Architecture

The platform is built using three core layers:

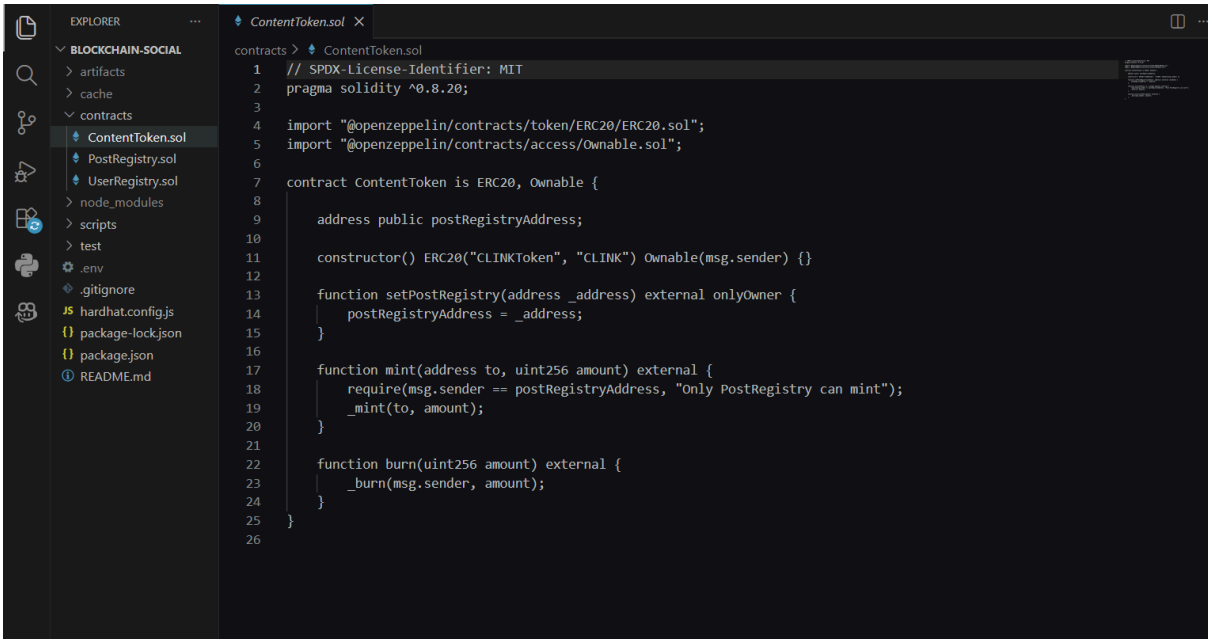
- **Blockchain Layer** → Smart contracts on Ethereum Sepolia
- **Storage Layer** → Decentralized file storage using IPFS via Pinata
- **Application Layer** → Frontend + Backend using Web technologies and APIs

Tech Stack

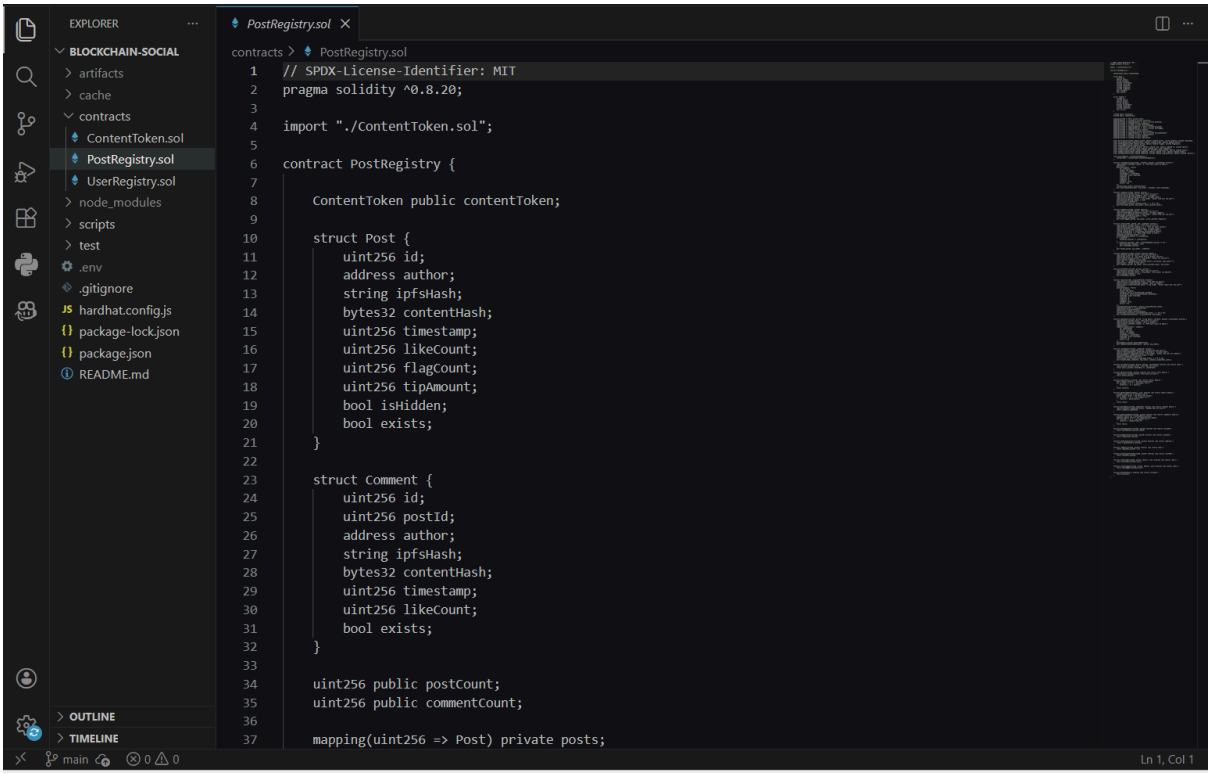
- React.js (Frontend)
- Node.js + Express (Backend)
- ethers.js (Blockchain interaction)
- Solidity (Smart Contracts)
- Ethereum Sepolia
- IPFS + Pinata (Storage)
- MetaMask (Authentication)
- Infura (RPC)

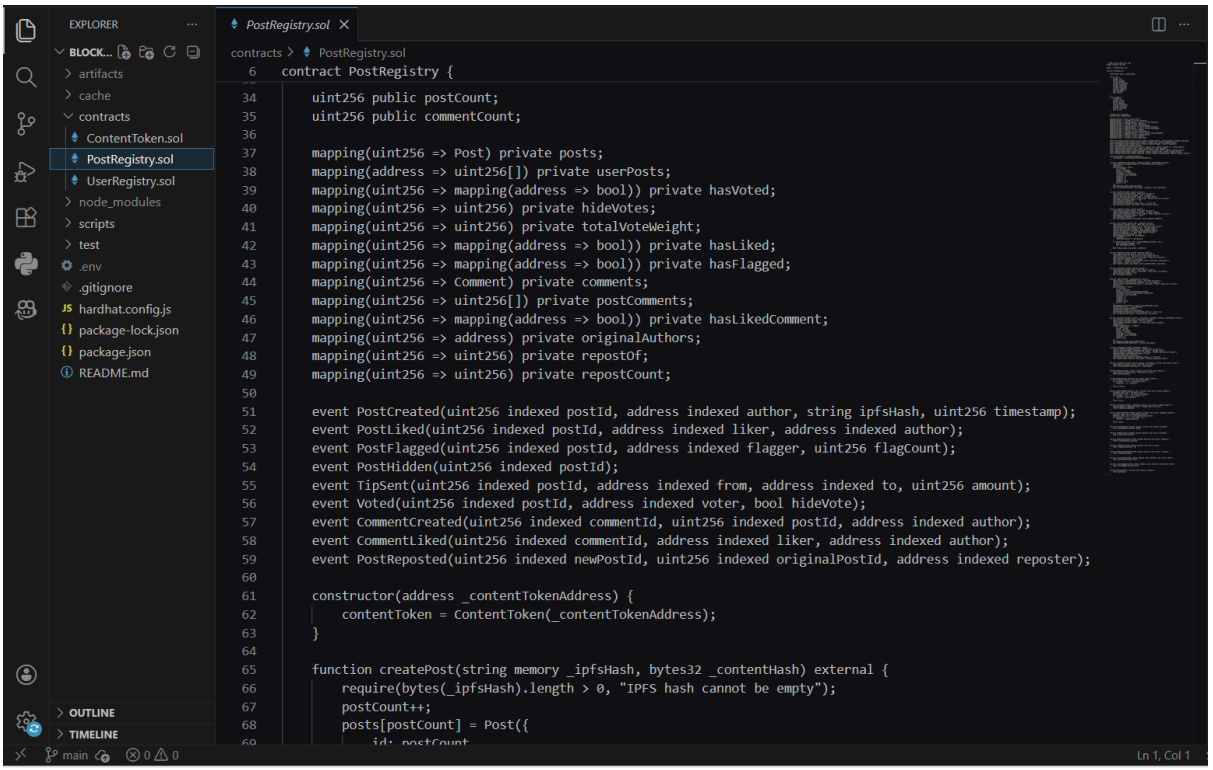
SMART CONTRACTS:

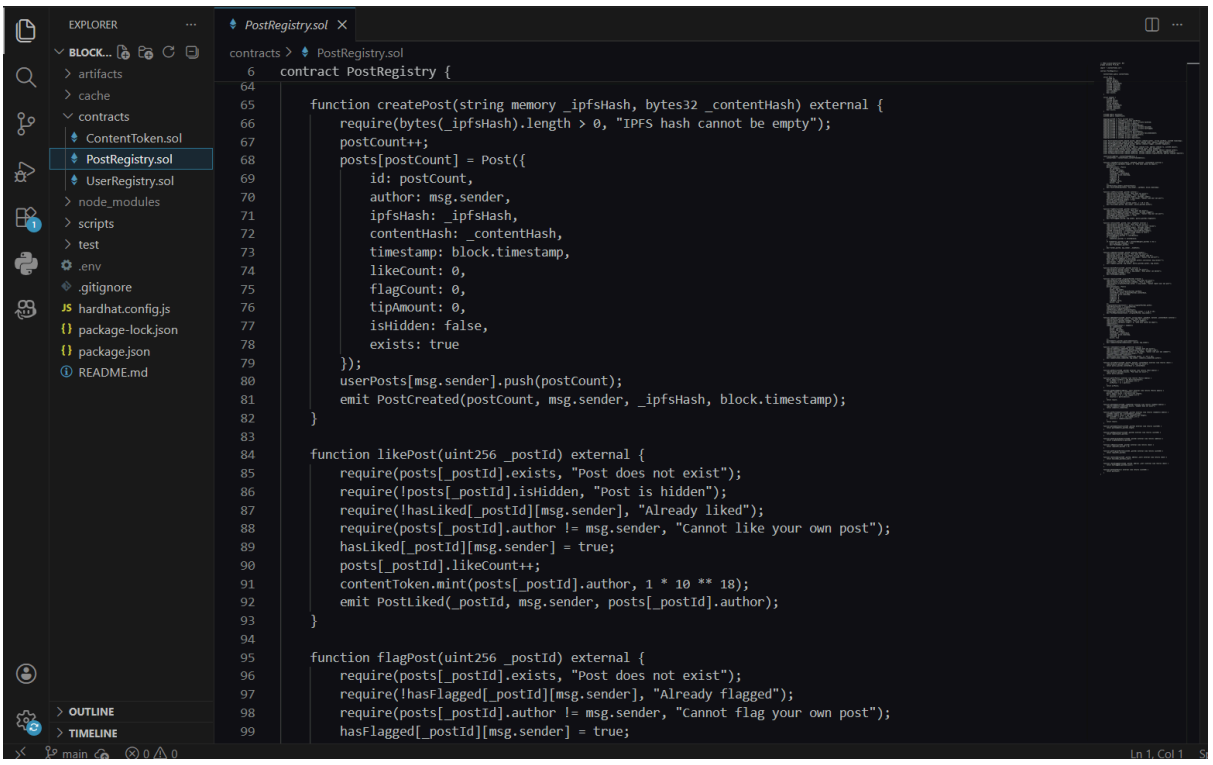
1.ContentToken.sol :

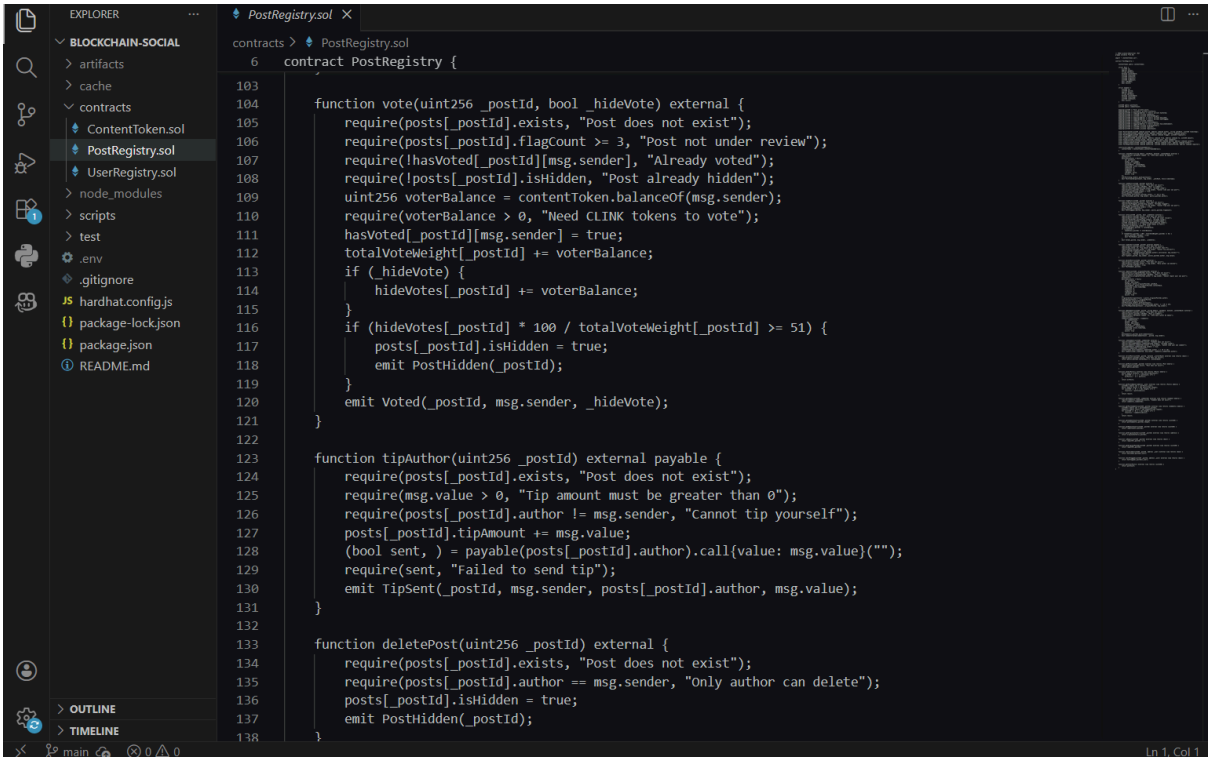


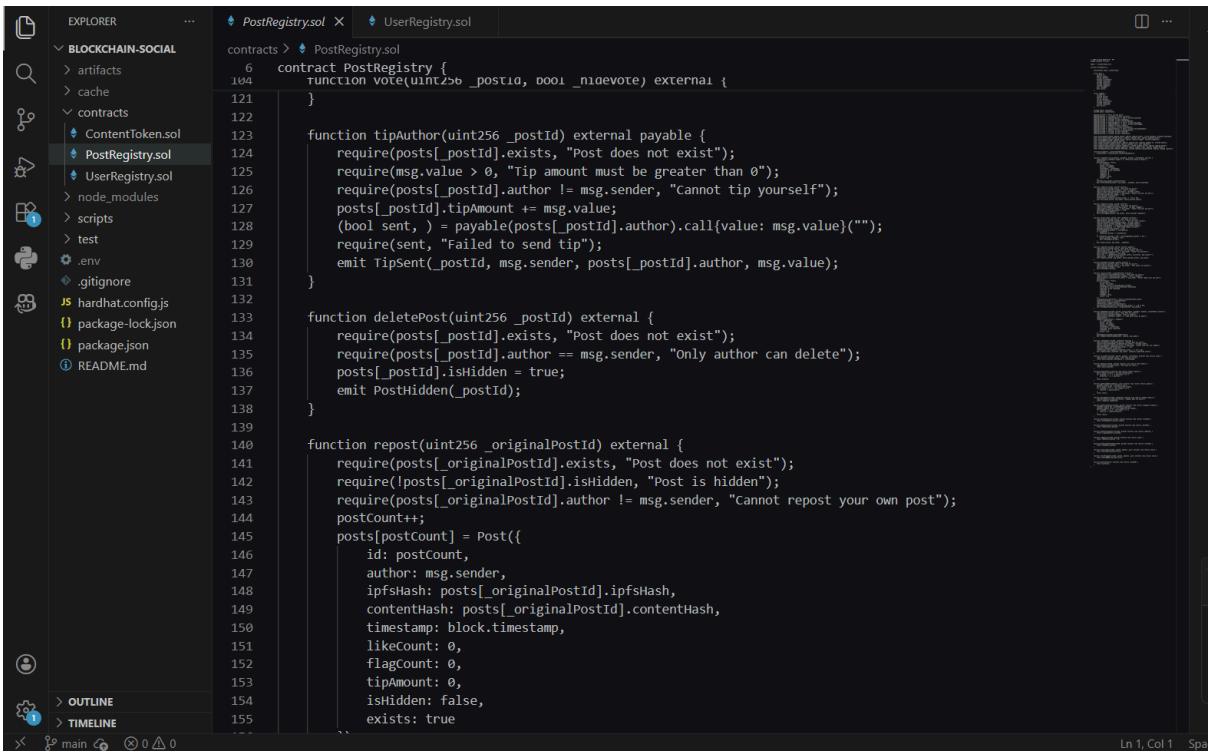
2.PostRegistry.sol :

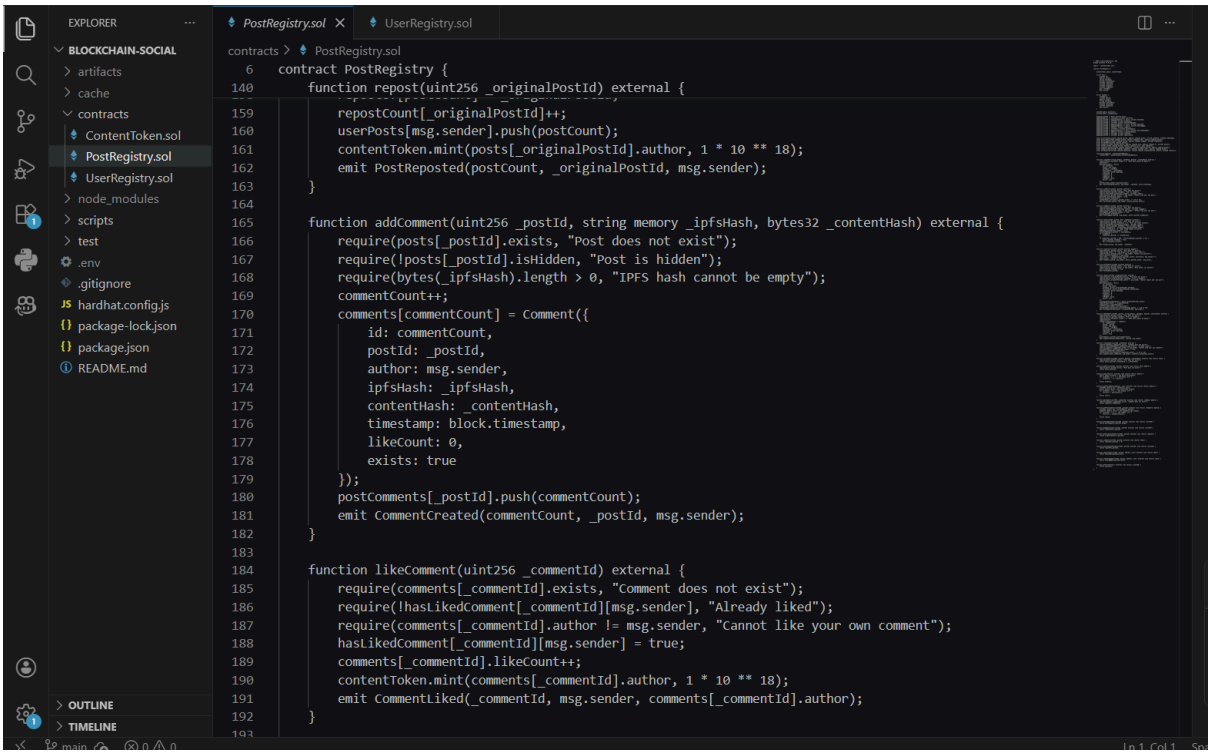

EXPLORER
PostRegistry.sol
BLOCKCHAIN-SOCIAL
artifacts
cache
contracts
ContentToken.sol
PostRegistry.sol
UserRegistry.sol
node_modules
scripts
test
.env
.gitignore
hardhat.config.js
package-lock.json
package.json
README.md
OUTLINE
TIMELINE
main
Ln 1, Col 1
contracts > PostRegistry.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
import "./ContentToken.sol";
contract PostRegistry {
ContentToken public contentToken;
struct Post {
uint256 id;
address author;
string ipfsHash;
bytes32 contentHash;
uint256 timestamp;
uint256 likeCount;
uint256 flagCount;
uint256 tipAmount;
bool isHidden;
bool exists;
}
struct Comment {
uint256 id;
uint256 postId;
address author;
string ipfsHash;
bytes32 contentHash;
uint256 timestamp;
uint256 likeCount;
bool exists;
}
uint256 public postCount;
uint256 public commentCount;
mapping(uint256 => Post) private posts;

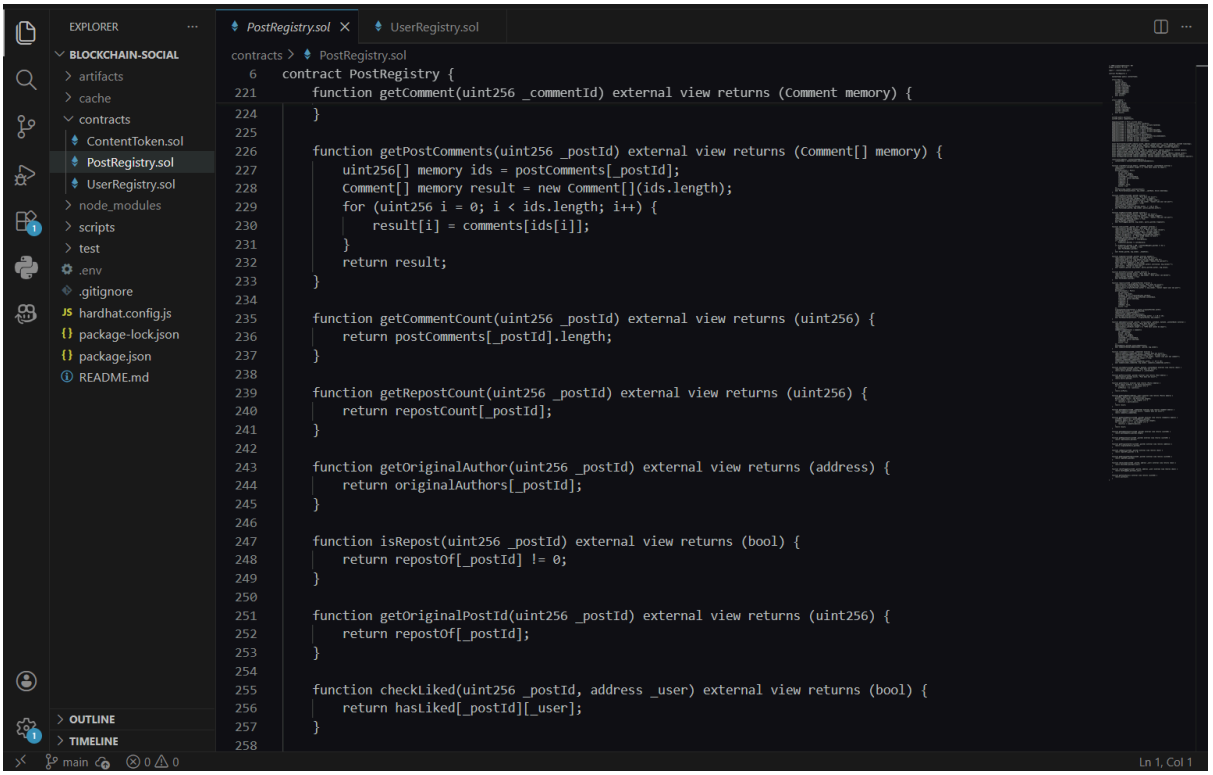

EXPLORER
PostRegistry.sol
BLOCK...
artifacts
cache
contracts
ContentToken.sol
PostRegistry.sol
UserRegistry.sol
node_modules
scripts
test
.env
.gitignore
hardhat.config.js
package-lock.json
package.json
README.md
OUTLINE
TIMELINE
main
Ln 1, Col 1
contracts > PostRegistry.sol
contract PostRegistry {
uint256 public postCount;
uint256 public commentCount;
mapping(uint256 => Post) private posts;
mapping(address => uint256[]) private userPosts;
mapping(uint256 => mapping(address => bool)) private hasVoted;
mapping(uint256 => uint256) private hideVotes;
mapping(uint256 => uint256) private totalVoteWeight;
mapping(uint256 => mapping(address => bool)) private hasLiked;
mapping(uint256 => mapping(address => bool)) private hasFlagged;
mapping(uint256 => Comment) private comments;
mapping(uint256 => uint256[]) private postComments;
mapping(uint256 => mapping(address => bool)) private hasLikedComment;
mapping(uint256 => address) private originalAuthors;
mapping(uint256 => uint256) private repostOf;
mapping(uint256 => uint256) private repostCount;
event PostCreated(uint256 indexed postId, address indexed author, string ipfsHash, uint256 timestamp);
event PostLiked(uint256 indexed postId, address indexed liker, address indexed author);
event PostFlagged(uint256 indexed postId, address indexed flagger, uint256 flagCount);
event PostHidden(uint256 indexed postId);
event TipSent(uint256 indexed postId, address indexed from, address indexed to, uint256 amount);
event Voted(uint256 indexed postId, address indexed voter, bool hideVote);
event CommentCreated(uint256 indexed commentId, uint256 indexed postId, address indexed author);
event CommentLiked(uint256 indexed commentId, address indexed liker, address indexed author);
event PostReposted(uint256 indexed newPostId, uint256 indexed originalPostId, address indexed reposter);
constructor(address _contentTokenAddress) {
contentToken = ContentToken(_contentTokenAddress);
}
function createPost(string memory _ipfsHash, bytes32 _contentHash) external {
require(bytes(_ipfsHash).length > 0, "IPFS hash cannot be empty");
postCount++;
posts[postCount] = Post({

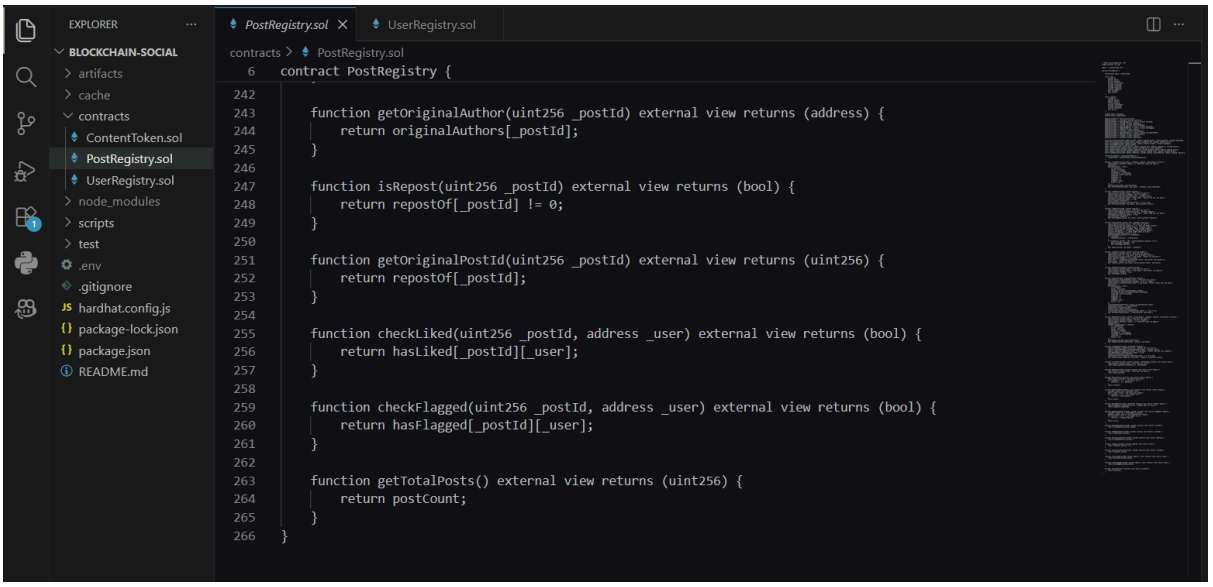

EXPLORER
PostRegistry.sol
contracts > PostRegistry.sol
BLOCK...
artifacts
cache
contracts
ContentToken.sol
PostRegistry.sol
UserRegistry.sol
node_modules
scripts
test
.env
.gitignore
hardhat.config.js
package-lock.json
package.json
README.md
OUTLINE
TIMELINE
main
Ln 1, Col 1
6 contract PostRegistry {
64
65 function createPost(string memory _ipfsHash, bytes32 _contentHash) external {
66 require(bytes(_ipfsHash).length > 0, "IPFS hash cannot be empty");
67 postCount++;
68 posts[postCount] = Post({
69 id: postCount,
70 author: msg.sender,
71 ipfsHash: _ipfsHash,
72 contentHash: _contentHash,
73 timestamp: block.timestamp,
74 likeCount: 0,
75 flagCount: 0,
76 tipAmount: 0,
77 isHidden: false,
78 exists: true
79 });
80 userPosts[msg.sender].push(postCount);
81 emit PostCreated(postCount, msg.sender, _ipfsHash, block.timestamp);
82 }
83
84 function likePost(uint256 _postId) external {
85 require(posts[_postId].exists, "Post does not exist");
86 require(!posts[_postId].isHidden, "Post is hidden");
87 require(!hasLiked[_postId][msg.sender], "Already liked");
88 require(posts[_postId].author != msg.sender, "Cannot like your own post");
89 hasLiked[_postId][msg.sender] = true;
90 posts[_postId].likeCount++;
91 contentToken.mint(posts[_postId].author, 1 * 10 ** 18);
92 emit PostLiked(_postId, msg.sender, posts[_postId].author);
93 }
94
95 function flagPost(uint256 _postId) external {
96 require(posts[_postId].exists, "Post does not exist");
97 require(!hasFlagged[_postId][msg.sender], "Already flagged");
98 require(posts[_postId].author != msg.sender, "Cannot flag your own post");
99 hasFlagged[_postId][msg.sender] = true;


EXPLORER
PostRegistry.sol
contracts > PostRegistry.sol
BLOCKCHAIN-SOCIAL
artifacts
cache
contracts
ContentToken.sol
PostRegistry.sol
UserRegistry.sol
node_modules
scripts
test
.env
.gitignore
hardhat.config.js
package-lock.json
package.json
README.md
OUTLINE
TIMELINE
main
Ln 1, Col 1
6 contract PostRegistry {
103
104 function vote(uint256 _postId, bool _hideVote) external {
105 require(posts[_postId].exists, "Post does not exist");
106 require(posts[_postId].flagCount >= 3, "Post not under review");
107 require(!hasVoted[_postId][msg.sender], "Already voted");
108 require(!posts[_postId].isHidden, "Post already hidden");
109 uint256 voterBalance = contentToken.balanceOf(msg.sender);
110 require(voterBalance > 0, "Need CLINK tokens to vote");
111 hasVoted[_postId][msg.sender] = true;
112 totalVoteWeight[_postId] += voterBalance;
113 if (_hideVote) {
114 hideVotes[_postId] += voterBalance;
115 }
116 if (hideVotes[_postId] * 100 / totalVoteWeight[_postId] >= 51) {
117 posts[_postId].isHidden = true;
118 emit PostHidden(_postId);
119 }
120 emit Voted(_postId, msg.sender, _hideVote);
121 }
122
123 function tipAuthor(uint256 _postId) external payable {
124 require(posts[_postId].exists, "Post does not exist");
125 require(msg.value > 0, "Tip amount must be greater than 0");
126 require(posts[_postId].author != msg.sender, "Cannot tip yourself");
127 posts[_postId].tipAmount += msg.value;
128 (bool sent,) = payable(posts[_postId].author).call{value: msg.value}("");
129 require(sent, "Failed to send tip");
130 emit TipSent(_postId, msg.sender, posts[_postId].author, msg.value);
131 }
132
133 function deletePost(uint256 _postId) external {
134 require(posts[_postId].exists, "Post does not exist");
135 require(posts[_postId].author == msg.sender, "Only author can delete");
136 posts[_postId].isHidden = true;
137 emit PostHidden(_postId);
138 }

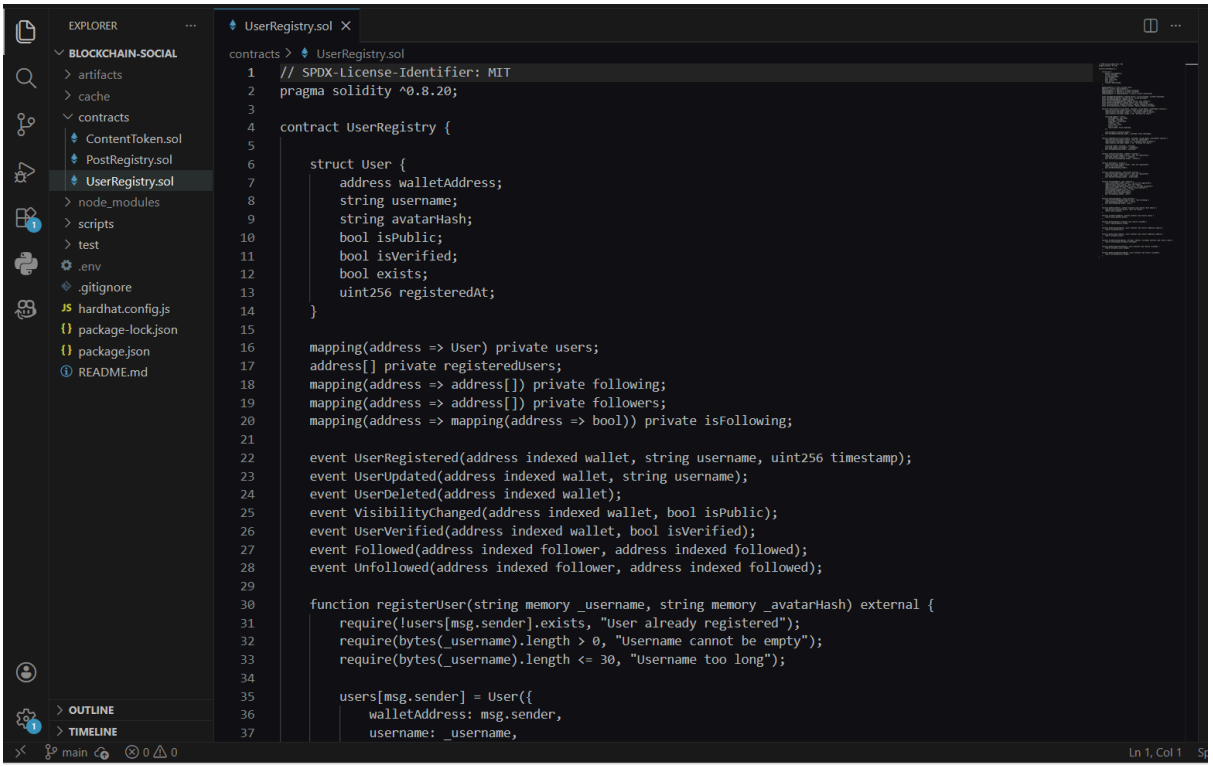

EXPLORER
BLOCKCHAIN-SOCIAL
artifacts
cache
contracts
ContentToken.sol
PostRegistry.sol
UserRegistry.sol
node_modules
scripts
test
.env
.gitignore
hardhat.config.js
package-lock.json
package.json
README.md
OUTLINE
TIMELINE
PostRegistry.sol
UserRegistry.sol
contracts > PostRegistry.sol
contract PostRegistry {
function vote(uint256 _postId, bool _isDevote) external {
}
function tipAuthor(uint256 _postId) external payable {
require(posts[_postId].exists, "Post does not exist");
require(msg.value > 0, "Tip amount must be greater than 0");
require(posts[_postId].author != msg.sender, "Cannot tip yourself");
posts[_postId].tipAmount += msg.value;
(bool sent,) = payable(posts[_postId].author).call{value: msg.value}("");
require(sent, "Failed to send tip");
emit TipSent(_postId, msg.sender, posts[_postId].author, msg.value);
}
function deletePost(uint256 _postId) external {
require(posts[_postId].exists, "Post does not exist");
require(posts[_postId].author == msg.sender, "Only author can delete");
posts[_postId].isHidden = true;
emit PostHidden(_postId);
}
function repost(uint256 _originalPostId) external {
require(posts[_originalPostId].exists, "Post does not exist");
require(!posts[_originalPostId].isHidden, "Post is hidden");
require(posts[_originalPostId].author != msg.sender, "Cannot repost your own post");
postCount++;
posts[postCount] = Post({
id: postCount,
author: msg.sender,
ipfsHash: posts[_originalPostId].ipfsHash,
contentHash: posts[_originalPostId].contentHash,
timestamp: block.timestamp,
likeCount: 0,
flagCount: 0,
tipAmount: 0,
isHidden: false,
exists: true

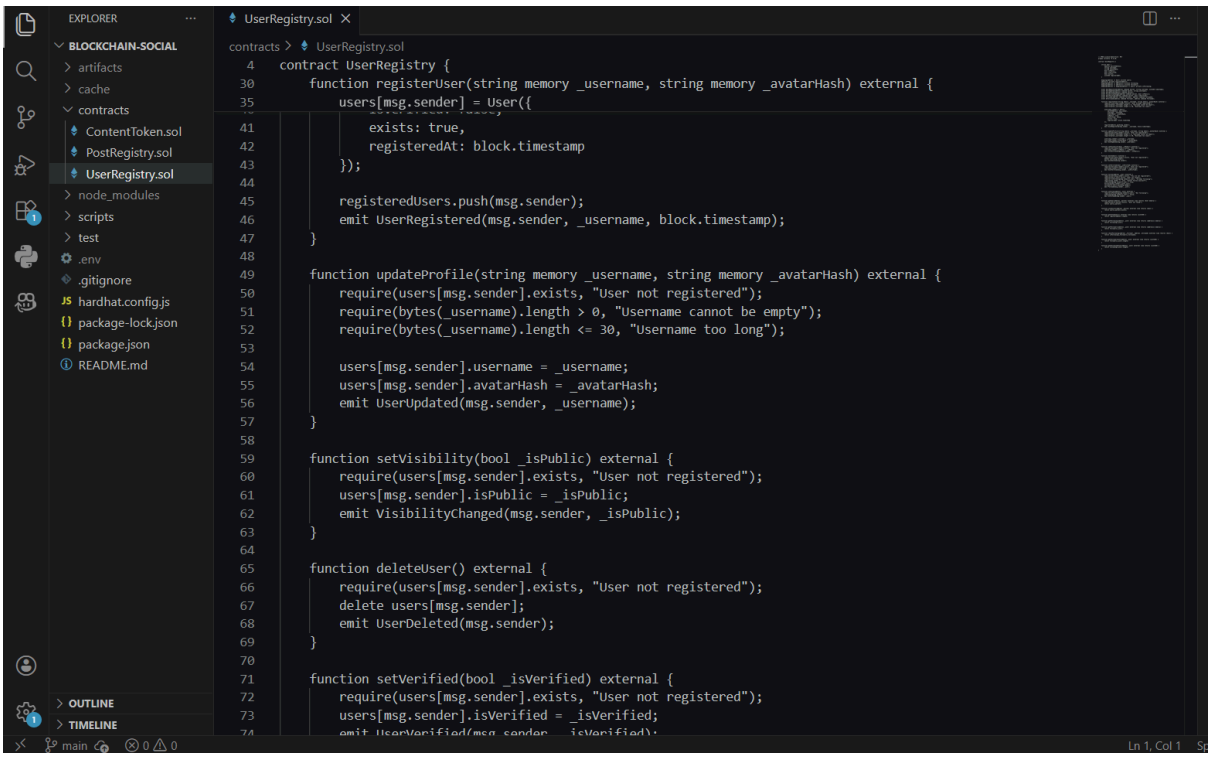

EXPLORER
BLOCKCHAIN-SOCIAL
artifacts
cache
contracts
ContentToken.sol
PostRegistry.sol
UserRegistry.sol
node_modules
scripts
test
.env
.gitignore
hardhat.config.js
package-lock.json
package.json
README.md
OUTLINE
TIMELINE
PostRegistry.sol
UserRegistry.sol
contracts > PostRegistry.sol
contract PostRegistry {
function repost(uint256 _originalPostId) external {
repostCount[_originalPostId]++;
userPosts[msg.sender].push(postCount);
contentToken.mint(posts[_originalPostId].author, 1 * 10 ** 18);
emit PostReposted(postCount, _originalPostId, msg.sender);
}
function addComment(uint256 _postId, string memory _ipfsHash, bytes32 _contentHash) external {
require(posts[_postId].exists, "Post does not exist");
require(!posts[_postId].isHidden, "Post is hidden");
require(bytes(_ipfsHash).length > 0, "IPFS hash cannot be empty");
commentCount++;
comments[commentCount] = Comment({
id: commentCount,
postId: _postId,
author: msg.sender,
ipfsHash: _ipfsHash,
contentHash: _contentHash,
timestamp: block.timestamp,
likeCount: 0,
exists: true
});
postComments[_postId].push(commentCount);
emit CommentCreated(commentCount, _postId, msg.sender);
}
function likeComment(uint256 _commentId) external {
require(comments[_commentId].exists, "Comment does not exist");
require(!hasLikedComment[_commentId][msg.sender], "Already liked");
require(comments[_commentId].author != msg.sender, "Cannot like your own comment");
hasLikedComment[_commentId][msg.sender] = true;
comments[_commentId].likeCount++;
contentToken.mint(comments[_commentId].author, 1 * 10 ** 18);
emit CommentLiked(_commentId, msg.sender, comments[_commentId].author);
}

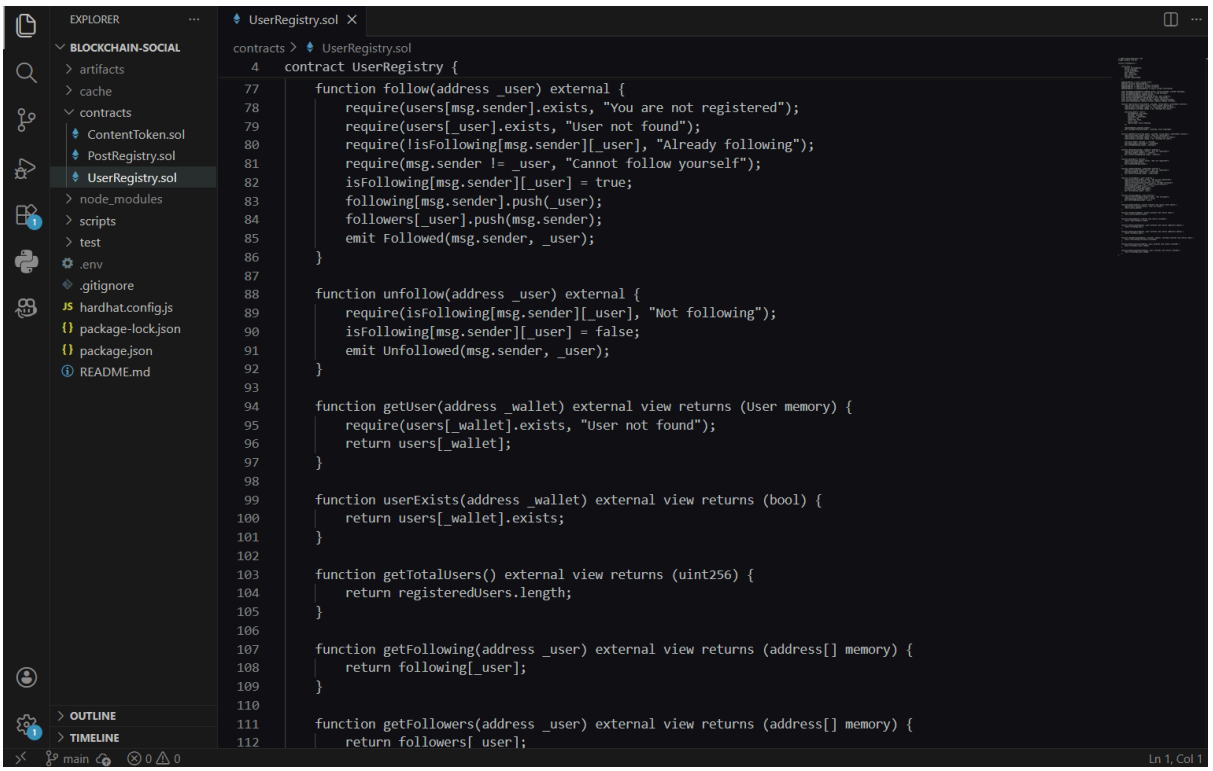


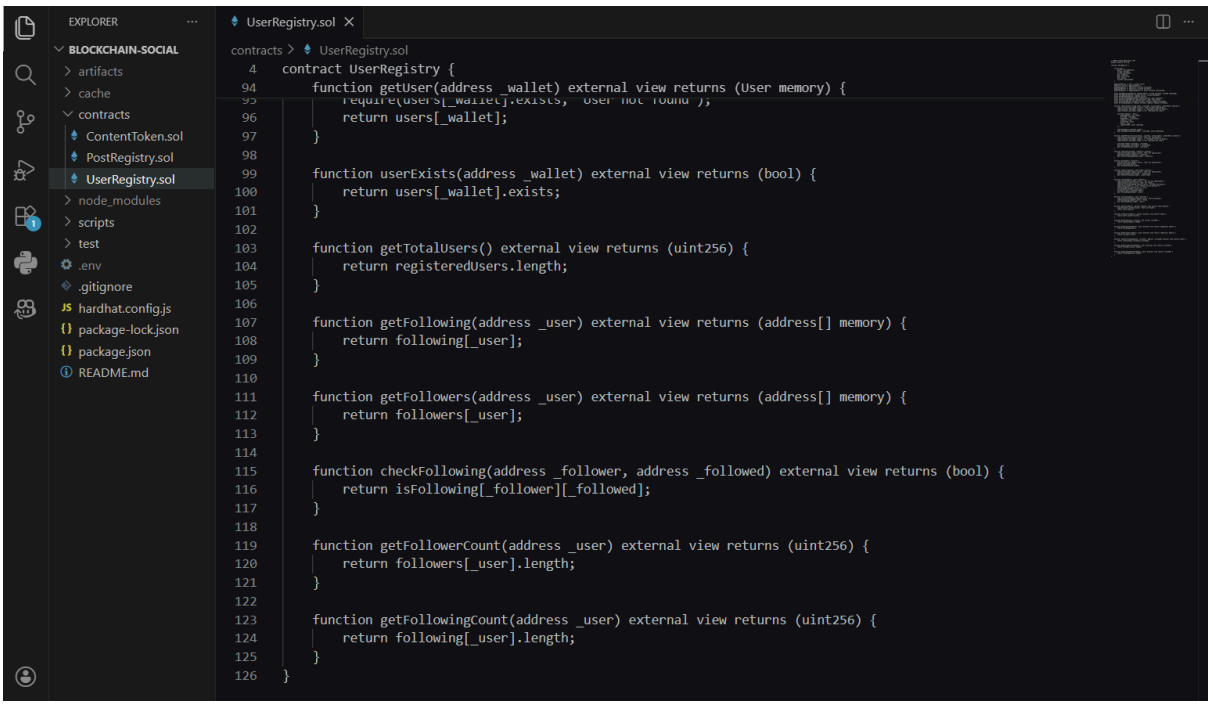


3.UserRegistry.sol:


EXPLORER
BLOCKCHAIN-SOCIAL
artifacts
cache
contracts
ContentToken.sol
PostRegistry.sol
UserRegistry.sol
node_modules
scripts
test
.env
.gitignore
hardhat.config.js
package-lock.json
package.json
README.md
OUTLINE
TIMELINE
UserRegistry.sol
contracts > UserRegistry.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
contract UserRegistry {
struct User {
address walletAddress;
string username;
string avatarHash;
bool isPublic;
bool isVerified;
bool exists;
uint256 registeredAt;
}
mapping(address => User) private users;
address[] private registeredUsers;
mapping(address => address[]) private following;
mapping(address => address[]) private followers;
mapping(address => mapping(address => bool)) private isFollowing;
event UserRegistered(address indexed wallet, string username, uint256 timestamp);
event UserUpdated(address indexed wallet, string username);
event UserDeleted(address indexed wallet);
event VisibilityChanged(address indexed wallet, bool isPublic);
event UserVerified(address indexed wallet, bool isVerified);
event Followed(address indexed follower, address indexed followed);
event Unfollowed(address indexed follower, address indexed followed);
function registerUser(string memory _username, string memory _avatarHash) external {
require(!users[msg.sender].exists, "User already registered");
require(bytes(_username).length > 0, "Username cannot be empty");
require(bytes(_username).length <= 30, "Username too long");
users[msg.sender] = User({
walletAddress: msg.sender,
username: _username,
main
Ln 1, Col 1

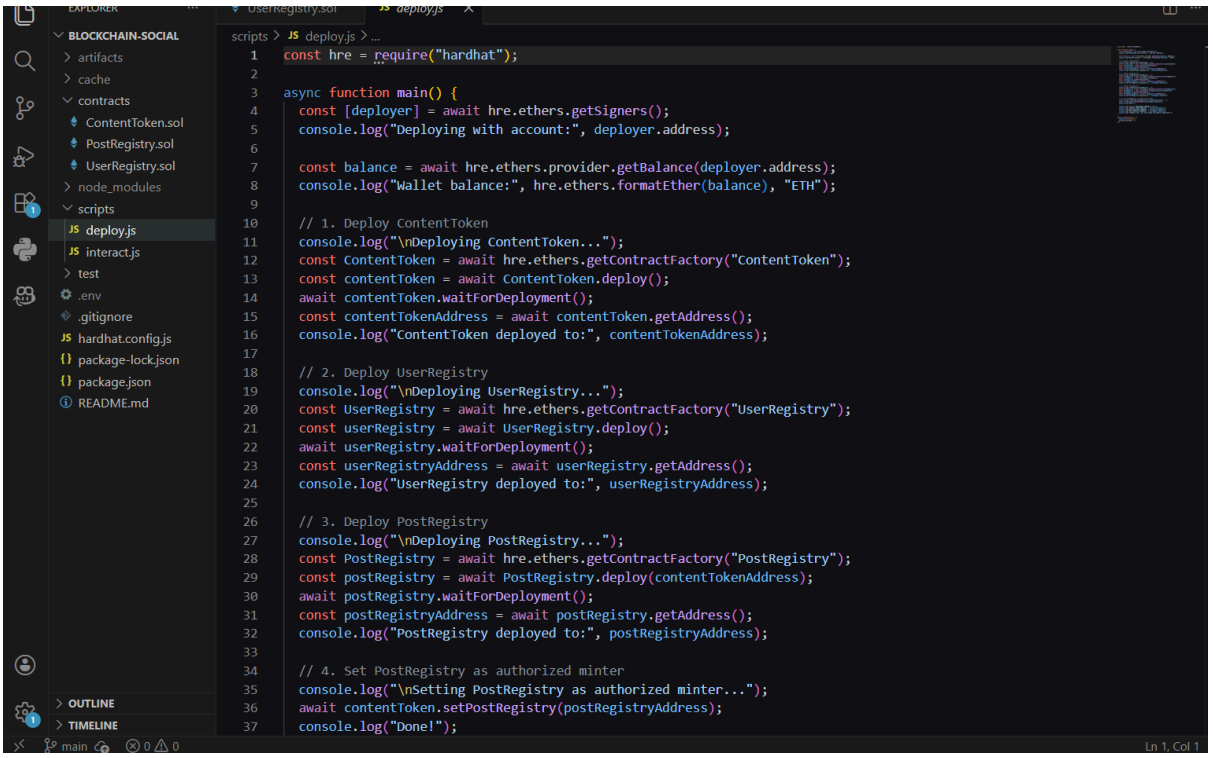

EXPLORER
BLOCKCHAIN-SOCIAL
artifacts
cache
contracts
ContentToken.sol
PostRegistry.sol
UserRegistry.sol
node_modules
scripts
test
.env
.gitignore
hardhat.config.js
package-lock.json
package.json
README.md
OUTLINE
TIMELINE
UserRegistry.sol
contracts > UserRegistry.sol
contract UserRegistry {
function registerUser(string memory _username, string memory _avatarHash) external {
users[msg.sender] = User({
exists: true,
registeredAt: block.timestamp
});
registeredUsers.push(msg.sender);
emit UserRegistered(msg.sender, _username, block.timestamp);
}
function updateProfile(string memory _username, string memory _avatarHash) external {
require(users[msg.sender].exists, "User not registered");
require(bytes(_username).length > 0, "Username cannot be empty");
require(bytes(_username).length <= 30, "Username too long");
users[msg.sender].username = _username;
users[msg.sender].avatarHash = _avatarHash;
emit UserUpdated(msg.sender, _username);
}
function setVisibility(bool _isPublic) external {
require(users[msg.sender].exists, "User not registered");
users[msg.sender].isPublic = _isPublic;
emit VisibilityChanged(msg.sender, _isPublic);
}
function deleteUser() external {
require(users[msg.sender].exists, "User not registered");
delete users[msg.sender];
emit UserDeleted(msg.sender);
}
function setVerified(bool _isVerified) external {
require(users[msg.sender].exists, "User not registered");
users[msg.sender].isVerified = _isVerified;
main
Ln 1, Col 1

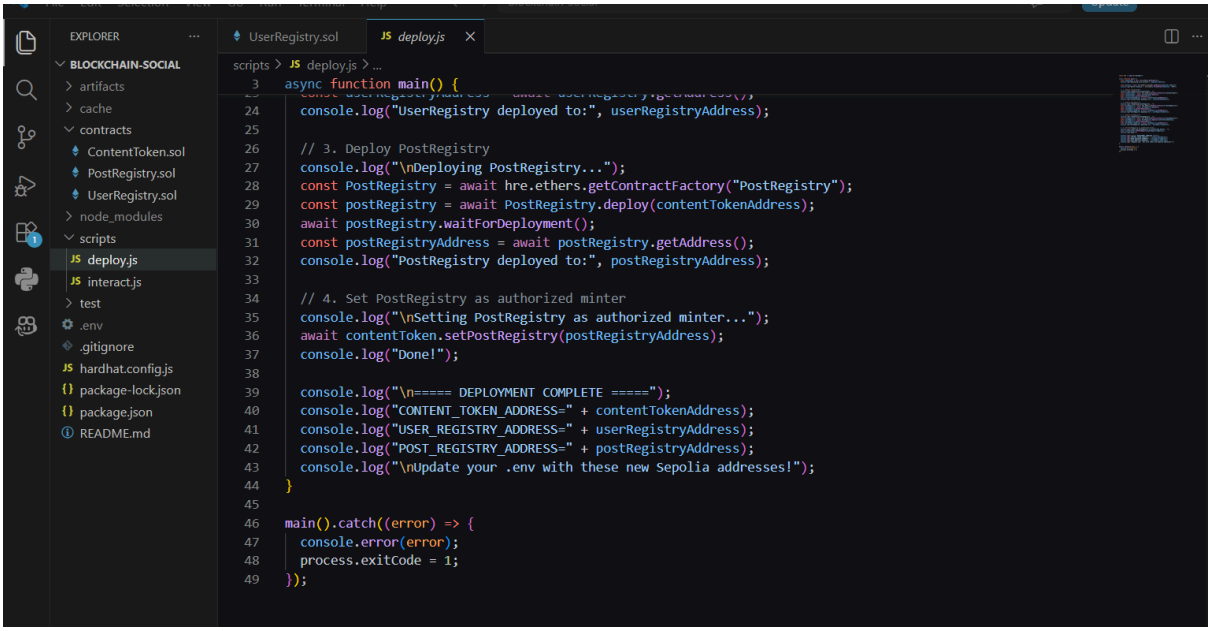




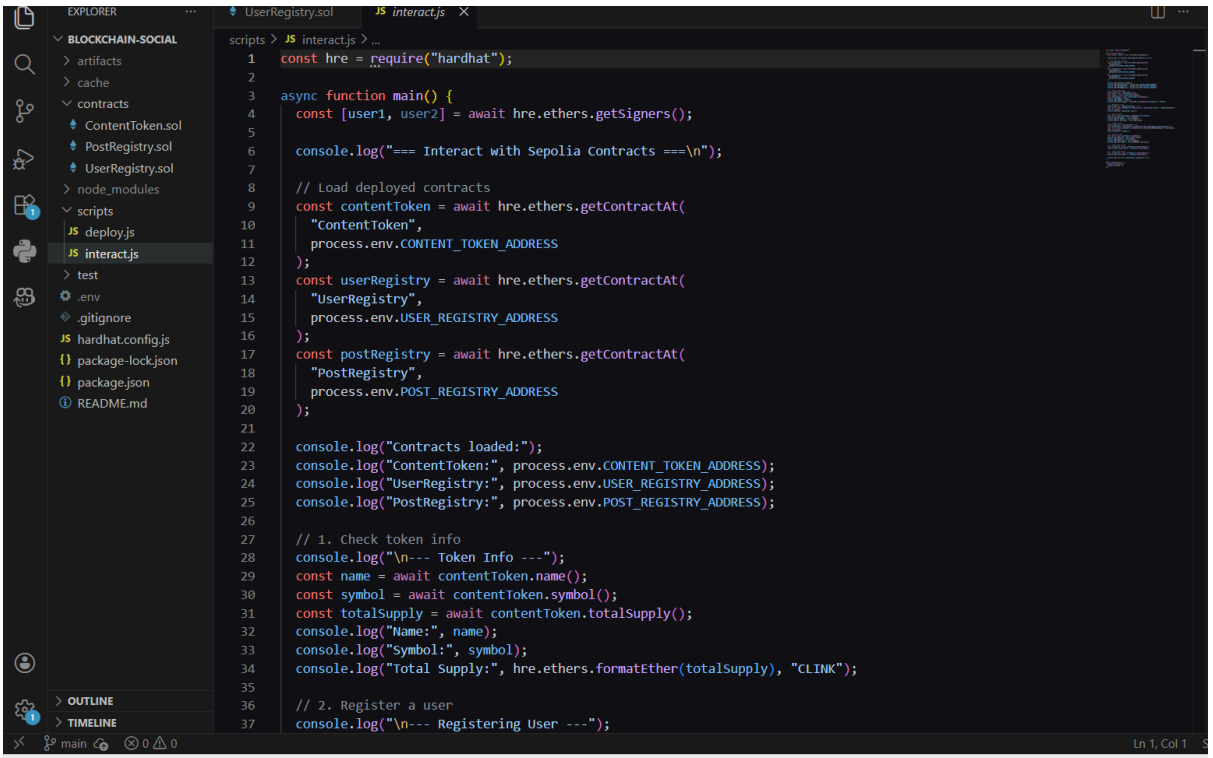
Scripts:

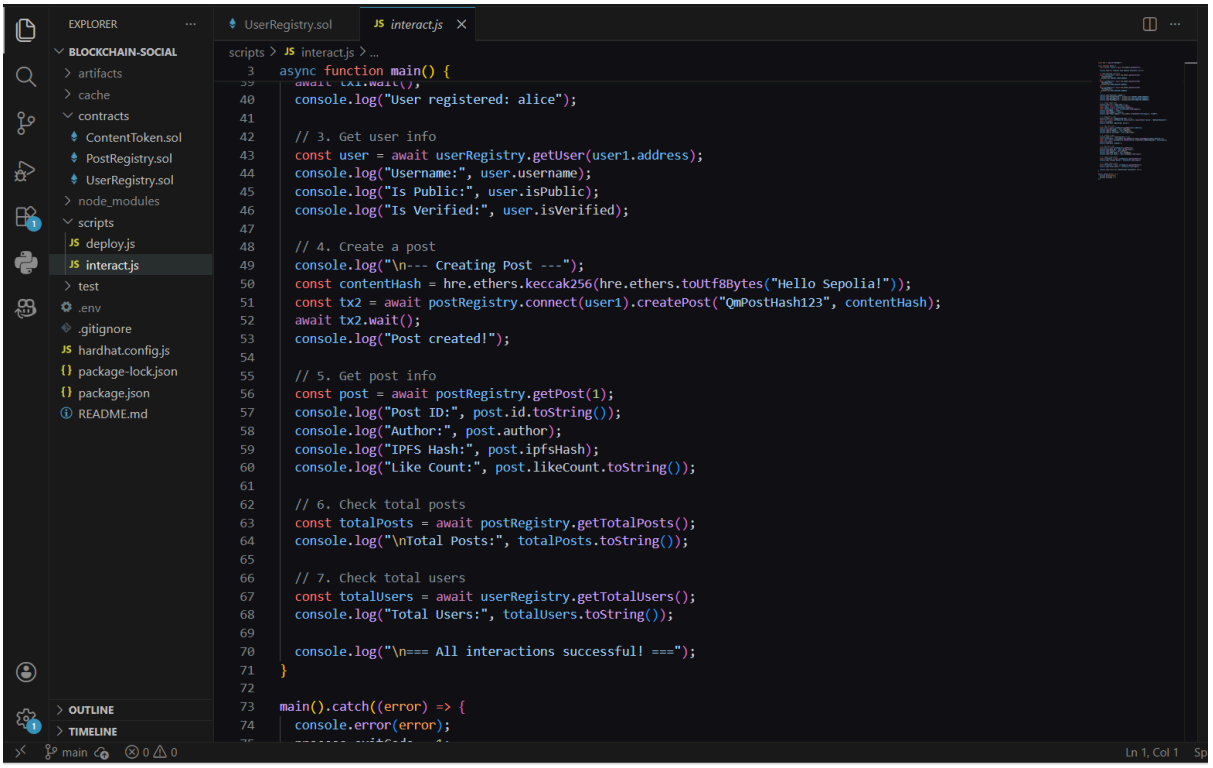
1. **Deploy.js:**

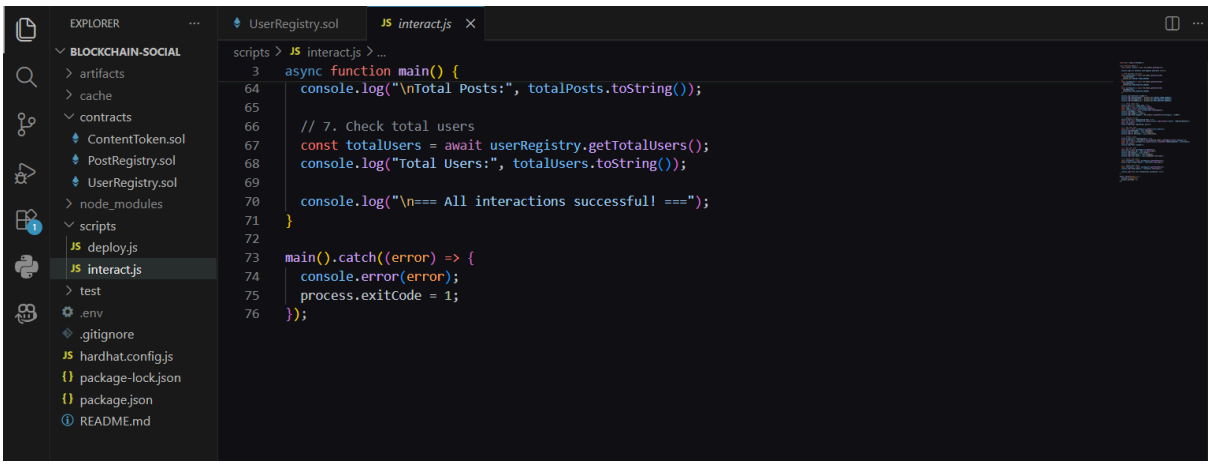




2.Interact.js :

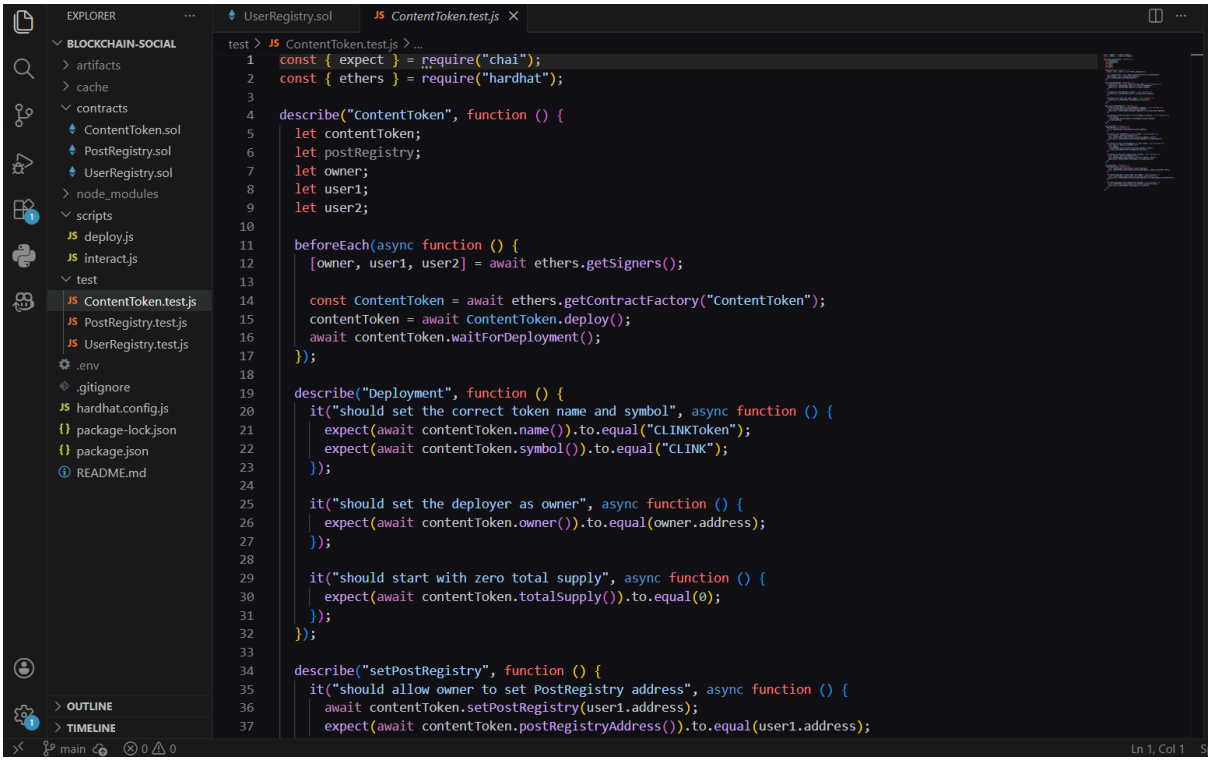

EXPLORER
BLOCKCHAIN-SOCIAL
artifacts
cache
contracts
ContentToken.sol
PostRegistry.sol
UserRegistry.sol
node_modules
scripts
deploy.js
interact.js
test
.env
.gitignore
hardhat.config.js
package-lock.json
package.json
README.md
OUTLINE
TIMELINE
UserRegistry.sol
interact.js
scripts > interact.js > ...
const hre = require("hardhat");
async function main() {
const [user1, user2] = await hre.ethers.getSigners();
console.log("=== Interact with Sepolia Contracts ===\n");
// Load deployed contracts
const contentToken = await hre.ethers.getContractAt(
"ContentToken",
process.env.CONTENT_TOKEN_ADDRESS
);
const userRegistry = await hre.ethers.getContractAt(
"UserRegistry",
process.env.USER_REGISTRY_ADDRESS
);
const postRegistry = await hre.ethers.getContractAt(
"PostRegistry",
process.env.POST_REGISTRY_ADDRESS
);
console.log("Contracts loaded:");
console.log("ContentToken:", process.env.CONTENT_TOKEN_ADDRESS);
console.log("UserRegistry:", process.env.USER_REGISTRY_ADDRESS);
console.log("PostRegistry:", process.env.POST_REGISTRY_ADDRESS);
// 1. Check token info
console.log("\n--- Token Info ---");
const name = await contentToken.name();
const symbol = await contentToken.symbol();
const totalSupply = await contentToken.totalSupply();
console.log("Name:", name);
console.log("Symbol:", symbol);
console.log("Total Supply:", hre.ethers.formatEther(totalSupply), "CLINK");
// 2. Register a user
console.log("\n--- Registering User ---");
main
Ln 1, Col 1

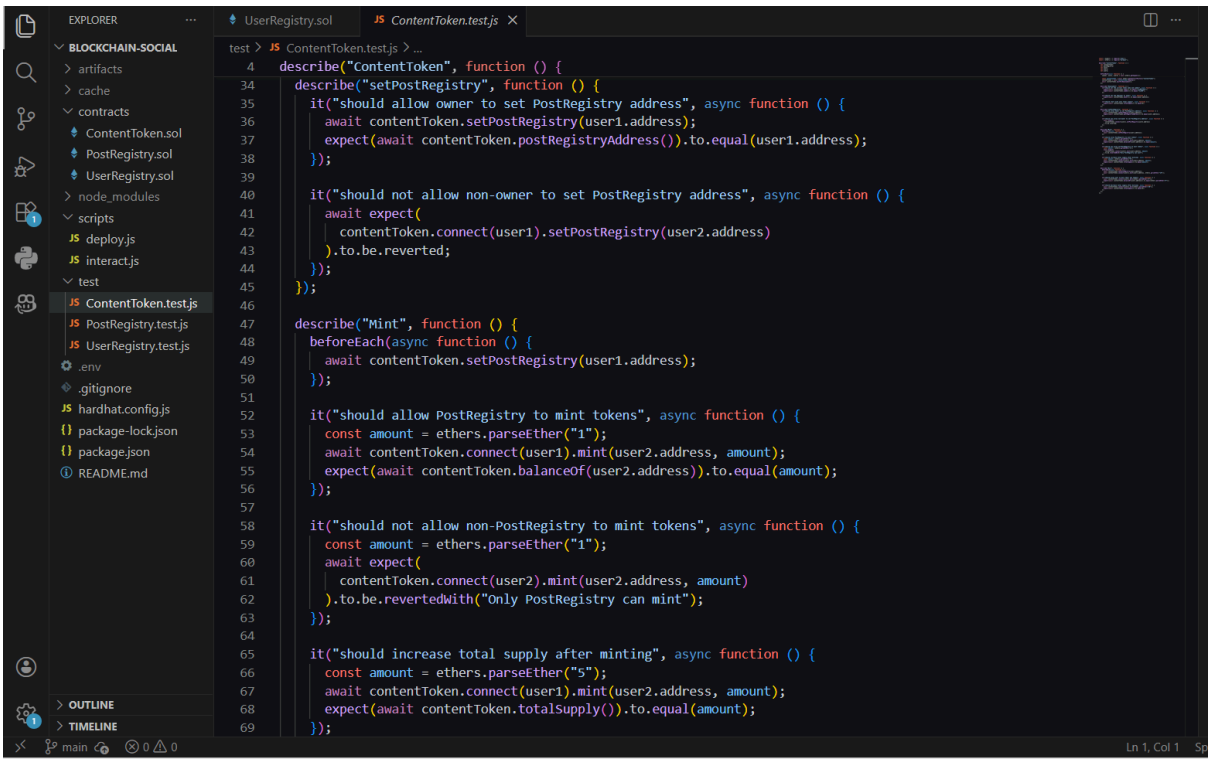

EXPLORER
BLOCKCHAIN-SOCIAL
artifacts
cache
contracts
ContentToken.sol
PostRegistry.sol
UserRegistry.sol
node_modules
scripts
deploy.js
interact.js
test
.env
.gitignore
hardhat.config.js
package-lock.json
package.json
README.md
OUTLINE
TIMELINE
UserRegistry.sol
interact.js
scripts > interact.js > ...
async function main() {
console.log("User registered: alice");
// 3. Get user info
const user = await userRegistry.getUser(user1.address);
console.log("Username:", user.username);
console.log("Is Public:", user.isPublic);
console.log("Is Verified:", user.isVerified);
// 4. Create a post
console.log("\n--- Creating Post ---");
const contentHash = hre.ethers.keccak256(hre.ethers.toUtf8Bytes("Hello Sepolia!"));
const tx2 = await postRegistry.connect(user1).createPost("QmPostHash123", contentHash);
await tx2.wait();
console.log("Post created!");
// 5. Get post info
const post = await postRegistry.getPost(1);
console.log("Post ID:", post.id.toString());
console.log("Author:", post.author);
console.log("IPFS Hash:", post.ipfsHash);
console.log("Like Count:", post.likeCount.toString());
// 6. Check total posts
const totalPosts = await postRegistry.getTotalPosts();
console.log("\nTotal Posts:", totalPosts.toString());
// 7. Check total users
const totalUsers = await userRegistry.getTotalUsers();
console.log("Total Users:", totalUsers.toString());
console.log("\n=== All interactions successful! ===");
}
main().catch((error) => {
console.error(error);
main
Ln 1, Col 1

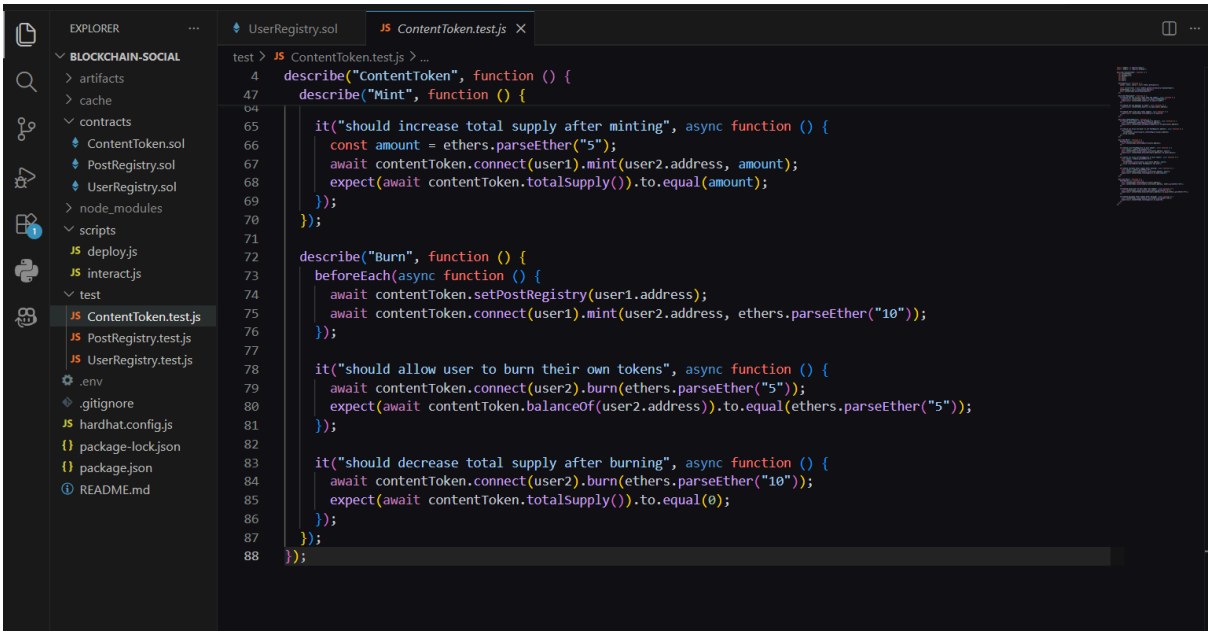


Tests:

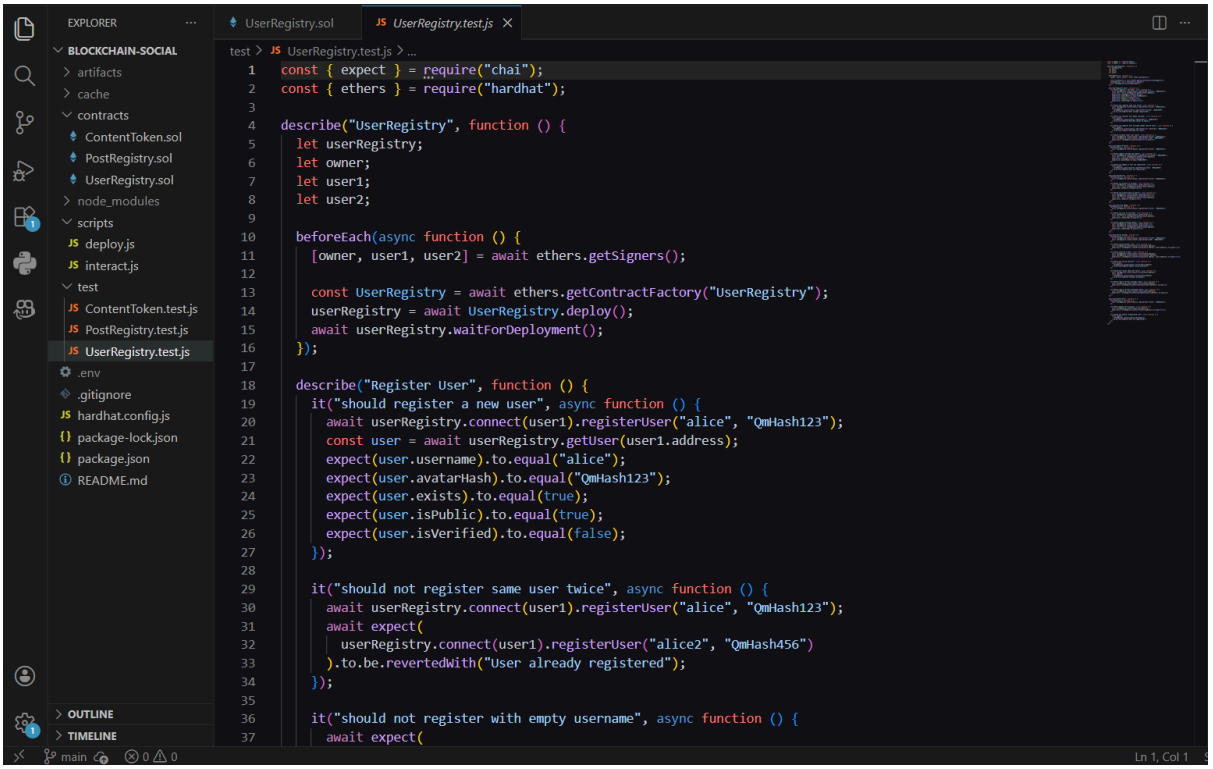
ContentToken.test.js:

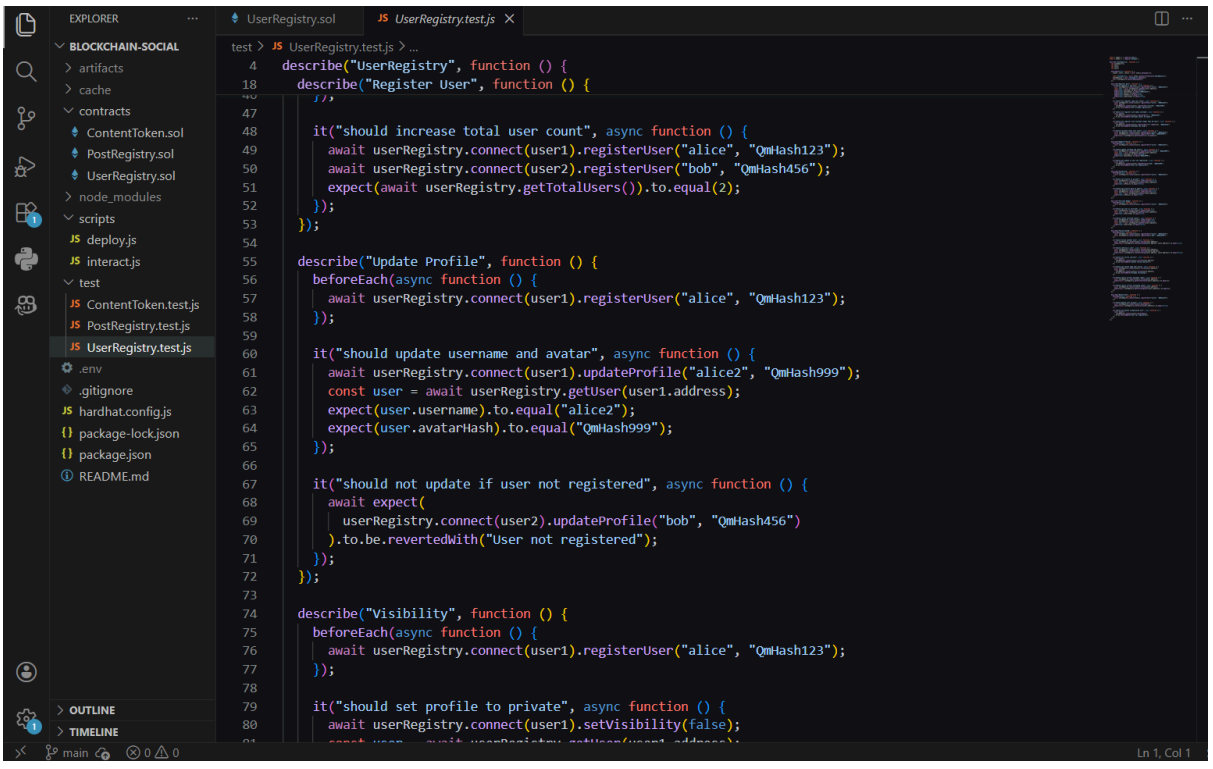


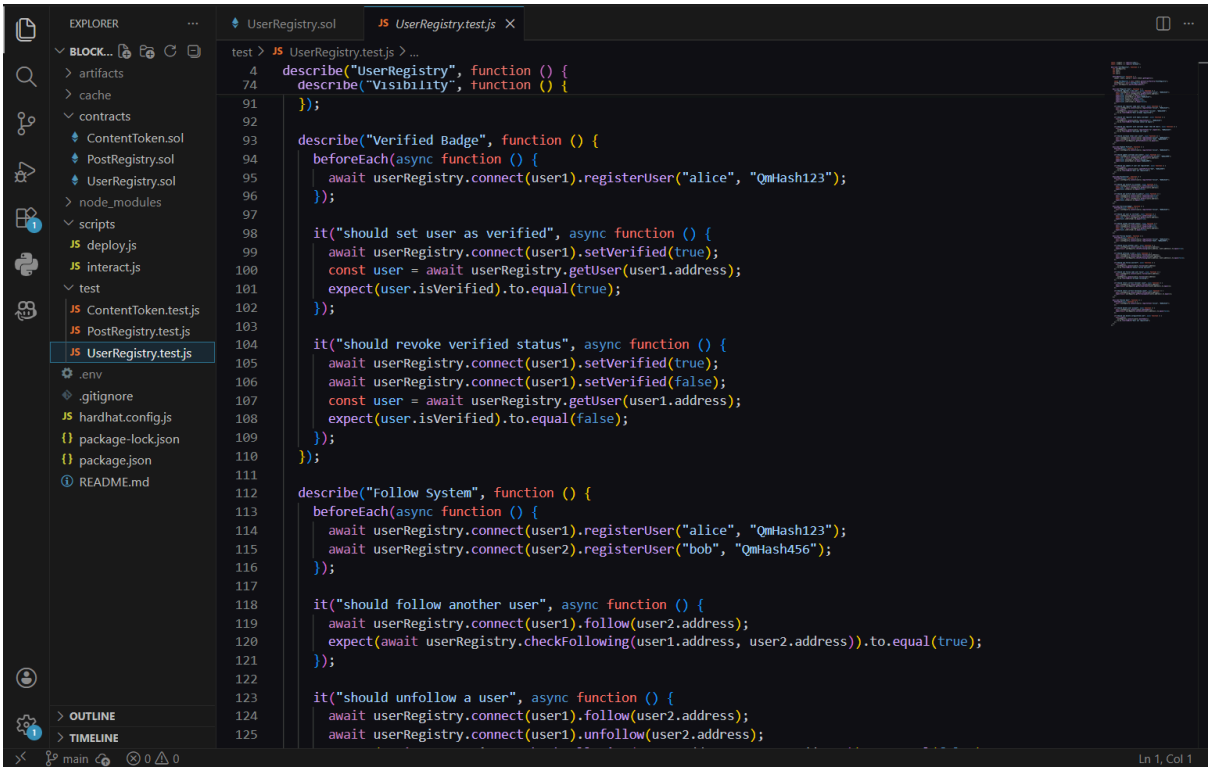


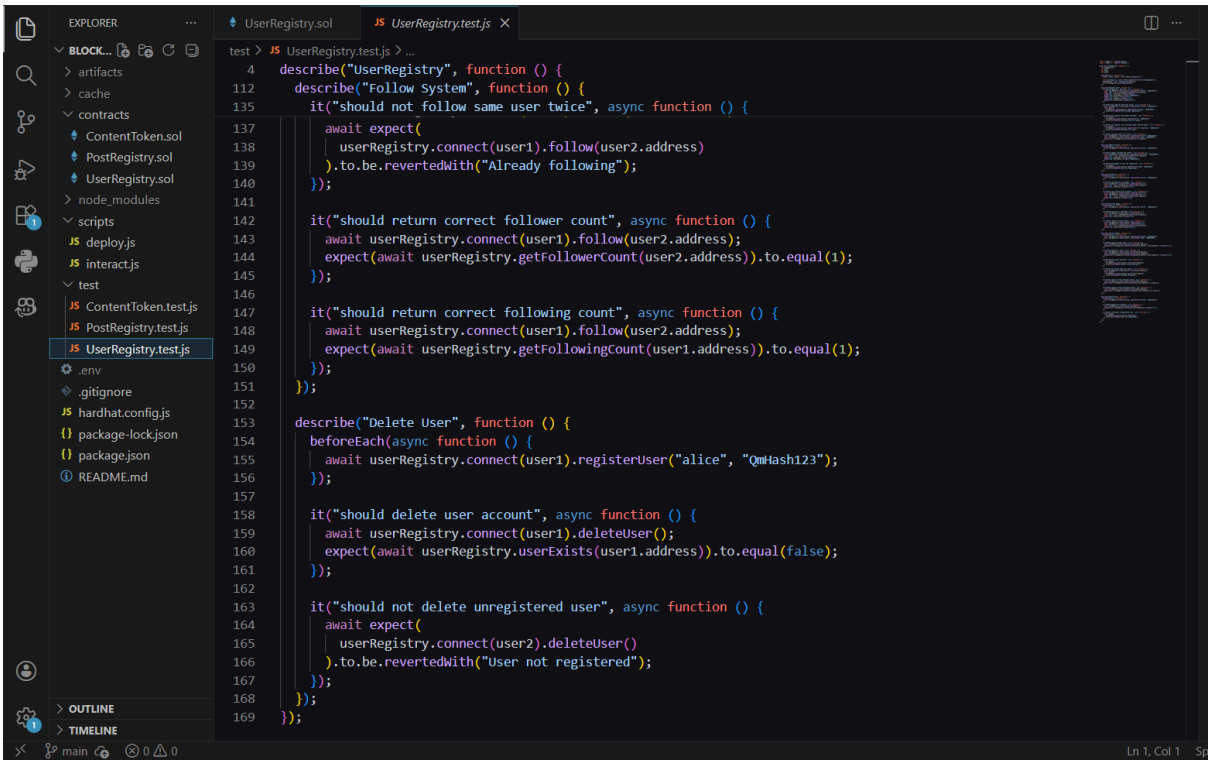


UserRegistry.test.js:

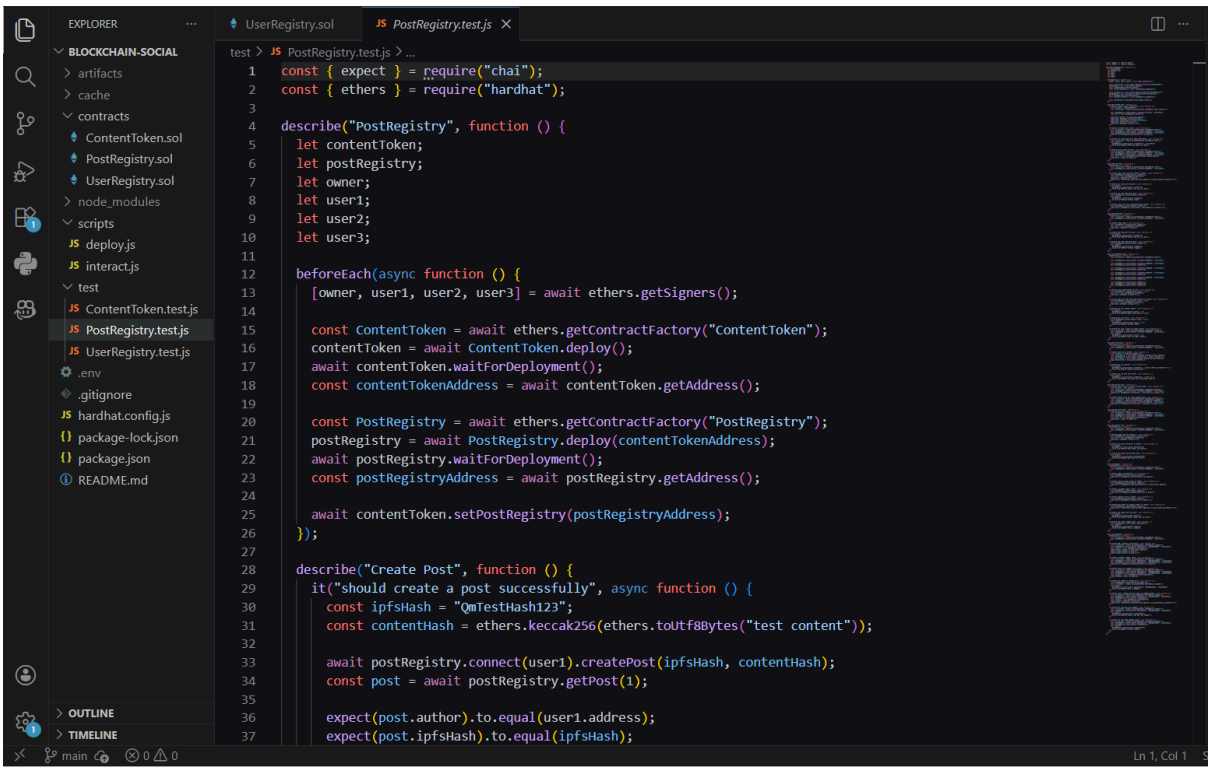

EXPLORER
BLOCKCHAIN-SOCIAL
artifacts
cache
contracts
ContentToken.sol
PostRegistry.sol
UserRegistry.sol
node_modules
scripts
deploy.js
interact.js
test
ContentToken.test.js
PostRegistry.test.js
UserRegistry.test.js
.env
.gitignore
hardhat.config.js
package-lock.json
package.json
README.md
OUTLINE
TIMELINE
UserRegistry.sol
UserRegistry.test.js
test > UserRegistry.test.js > ...
const { expect } = require("chai");
const { ethers } = require("hardhat");
describe("UserRegistry", function () {
let userRegistry;
let owner;
let user1;
let user2;
beforeEach(async function () {
[owner, user1, user2] = await ethers.getSigners();
const UserRegistry = await ethers.getContractFactory("UserRegistry");
userRegistry = await UserRegistry.deploy();
await userRegistry.waitForDeployment();
});
describe("Register User", function () {
it("should register a new user", async function () {
await userRegistry.connect(user1).registerUser("alice", "QmHash123");
const user = await userRegistry.getUser(user1.address);
expect(user.username).to.equal("alice");
expect(user.avatarHash).to.equal("QmHash123");
expect(user.exists).to.equal(true);
expect(user.isPublic).to.equal(true);
expect(user.isVerified).to.equal(false);
});
it("should not register same user twice", async function () {
await userRegistry.connect(user1).registerUser("alice", "QmHash123");
await expect(
userRegistry.connect(user1).registerUser("alice2", "QmHash456")
).to.be.revertedWith("User already registered");
});
it("should not register with empty username", async function () {
await expect(
main
Ln 1, Col 1

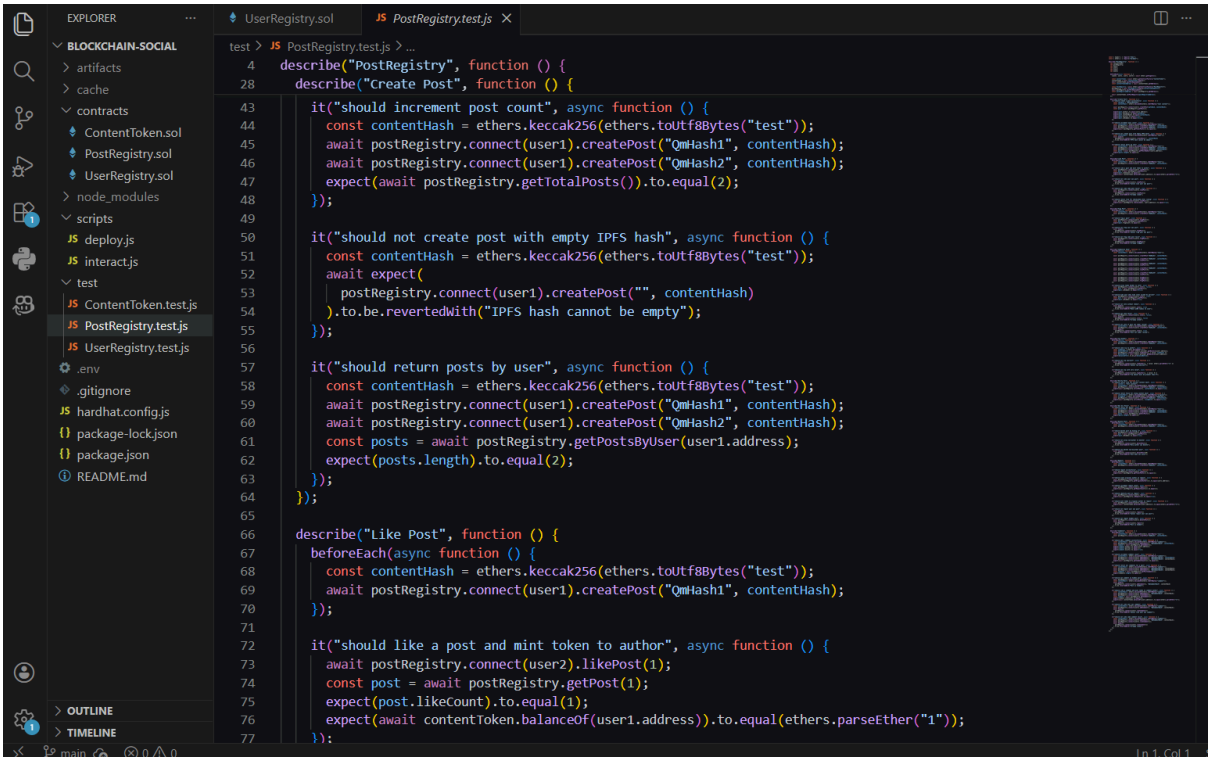

EXPLORER
BLOCKCHAIN-SOCIAL
artifacts
cache
contracts
ContentToken.sol
PostRegistry.sol
UserRegistry.sol
node_modules
scripts
deploy.js
interact.js
test
ContentToken.test.js
PostRegistry.test.js
UserRegistry.test.js
.env
.gitignore
hardhat.config.js
package-lock.json
package.json
README.md
OUTLINE
TIMELINE
UserRegistry.sol
UserRegistry.test.js
test > UserRegistry.test.js > ...
describe("UserRegistry", function () {
describe("Register User", function () {
it("should increase total user count", async function () {
await userRegistry.connect(user1).registerUser("alice", "QmHash123");
await userRegistry.connect(user2).registerUser("bob", "QmHash456");
expect(await userRegistry.getTotalUsers()).to.equal(2);
});
});
describe("Update Profile", function () {
beforeEach(async function () {
await userRegistry.connect(user1).registerUser("alice", "QmHash123");
});
it("should update username and avatar", async function () {
await userRegistry.connect(user1).updateProfile("alice2", "QmHash999");
const user = await userRegistry.getUser(user1.address);
expect(user.username).to.equal("alice2");
expect(user.avatarHash).to.equal("QmHash999");
});
it("should not update if user not registered", async function () {
await expect(
userRegistry.connect(user2).updateProfile("bob", "QmHash456")
).to.be.revertedWith("User not registered");
});
});
describe("Visibility", function () {
beforeEach(async function () {
await userRegistry.connect(user1).registerUser("alice", "QmHash123");
});
it("should set profile to private", async function () {
await userRegistry.connect(user1).setVisibility(false);
main
Ln 1, Col 1

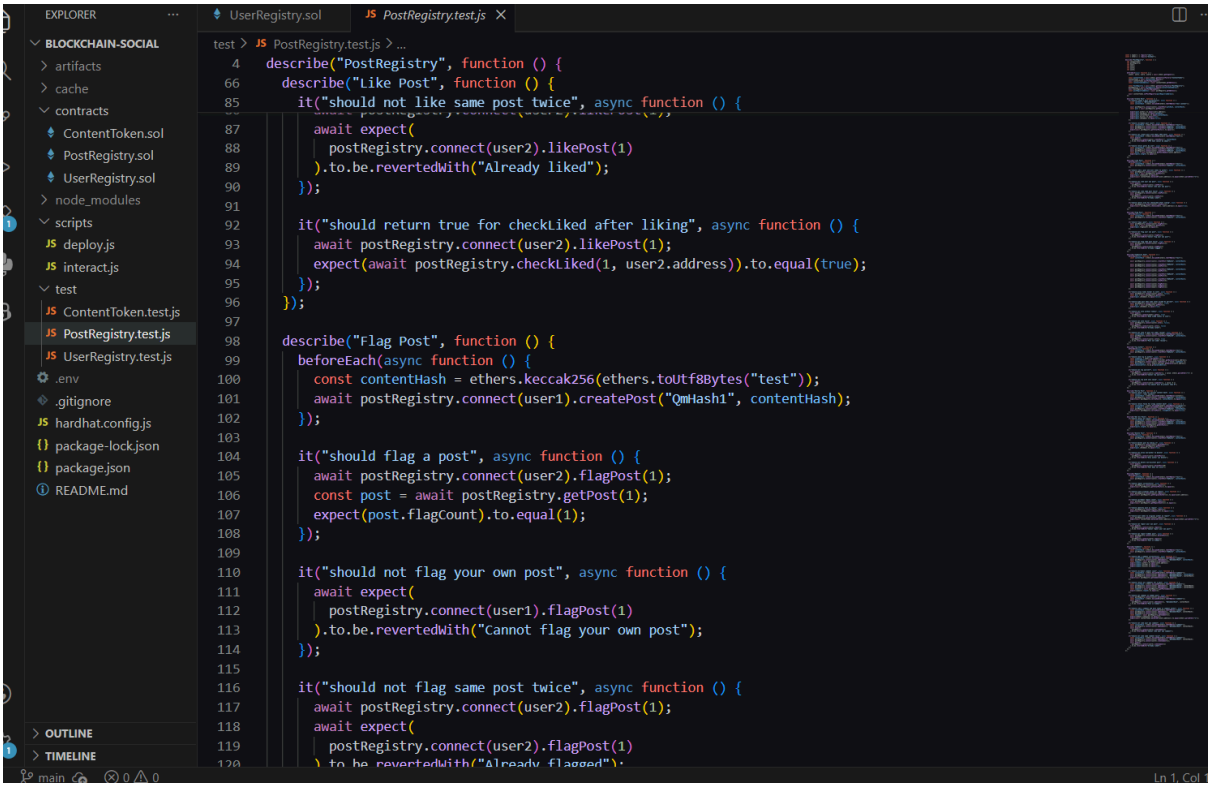


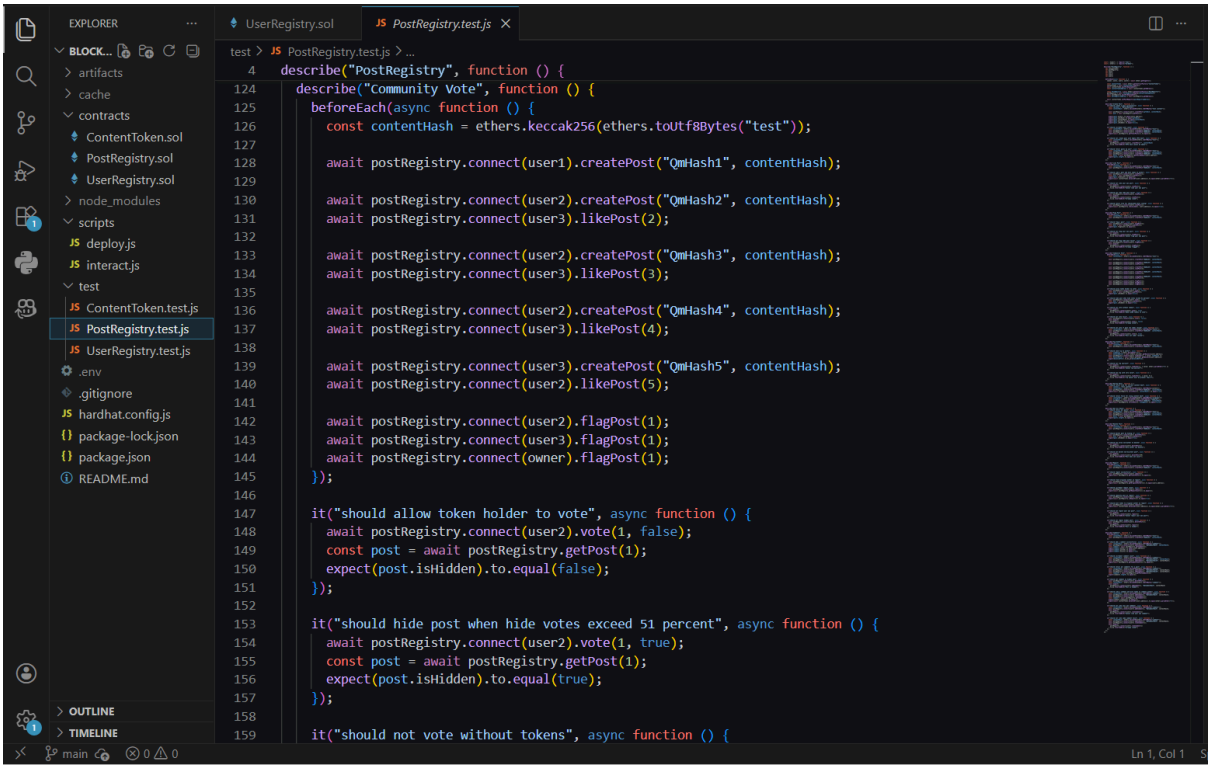


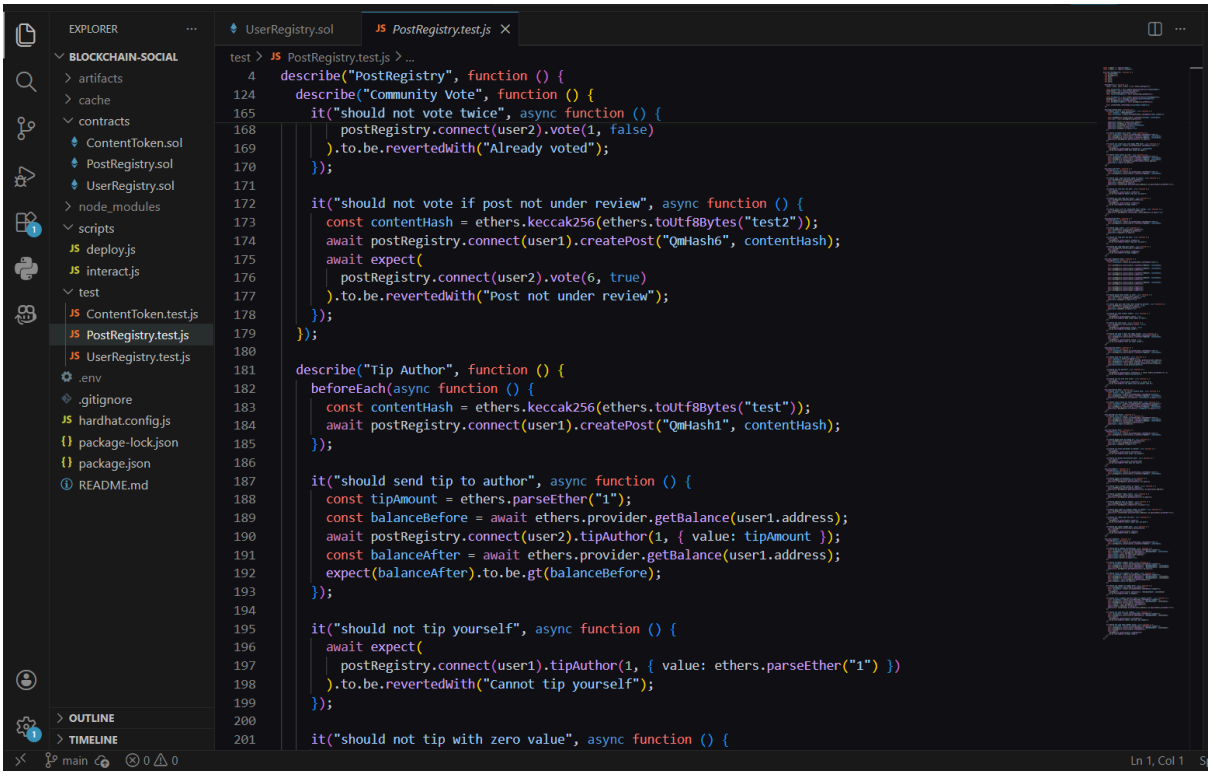
PostRegistry.test.js:

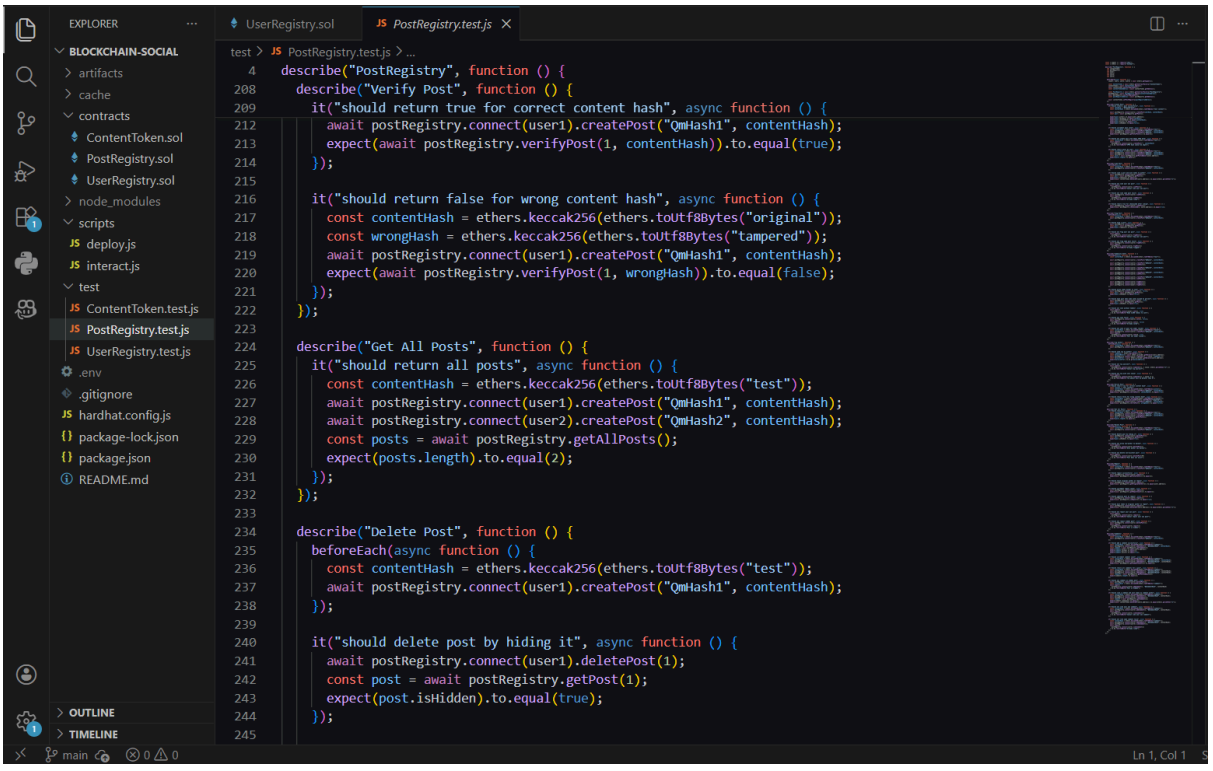

EXPLORER
BLOCKCHAIN-SOCIAL
artifacts
cache
contracts
ContentToken.sol
PostRegistry.sol
UserRegistry.sol
node_modules
scripts
deploy.js
interact.js
test
ContentToken.test.js
PostRegistry.test.js
UserRegistry.test.js
.env
.gitignore
hardhat.config.js
package-lock.json
package.json
README.md
OUTLINE
TIMELINE
UserRegistry.sol
PostRegistry.test.js
test > PostRegistry.test.js > ...
const { expect } = require("chai");
const { ethers } = require("hardhat");
describe("PostRegistry", function () {
let contentToken;
let postRegistry;
let owner;
let user1;
let user2;
let user3;
beforeEach(async function () {
[owner, user1, user2, user3] = await ethers.getSigners();
const ContentToken = await ethers.getContractFactory("ContentToken");
contentToken = await ContentToken.deploy();
await contentToken.waitForDeployment();
const contentTokenAddress = await contentToken.getAddress();
const PostRegistry = await ethers.getContractFactory("PostRegistry");
postRegistry = await PostRegistry.deploy(contentTokenAddress);
await postRegistry.waitForDeployment();
const postRegistryAddress = await postRegistry.getAddress();
await contentToken.setPostRegistry(postRegistryAddress);
});
describe("Create Post", function () {
it("should create a post successfully", async function () {
const ipfsHash = "QmTestHash123";
const contentHash = ethers.keccak256(ethers.toUtf8Bytes("test content"));
await postRegistry.connect(user1).createPost(ipfsHash, contentHash);
const post = await postRegistry.getPost(1);
expect(post.author).to.equal(user1.address);
expect(post.ipfsHash).to.equal(ipfsHash);
main
Ln 1, Col 1

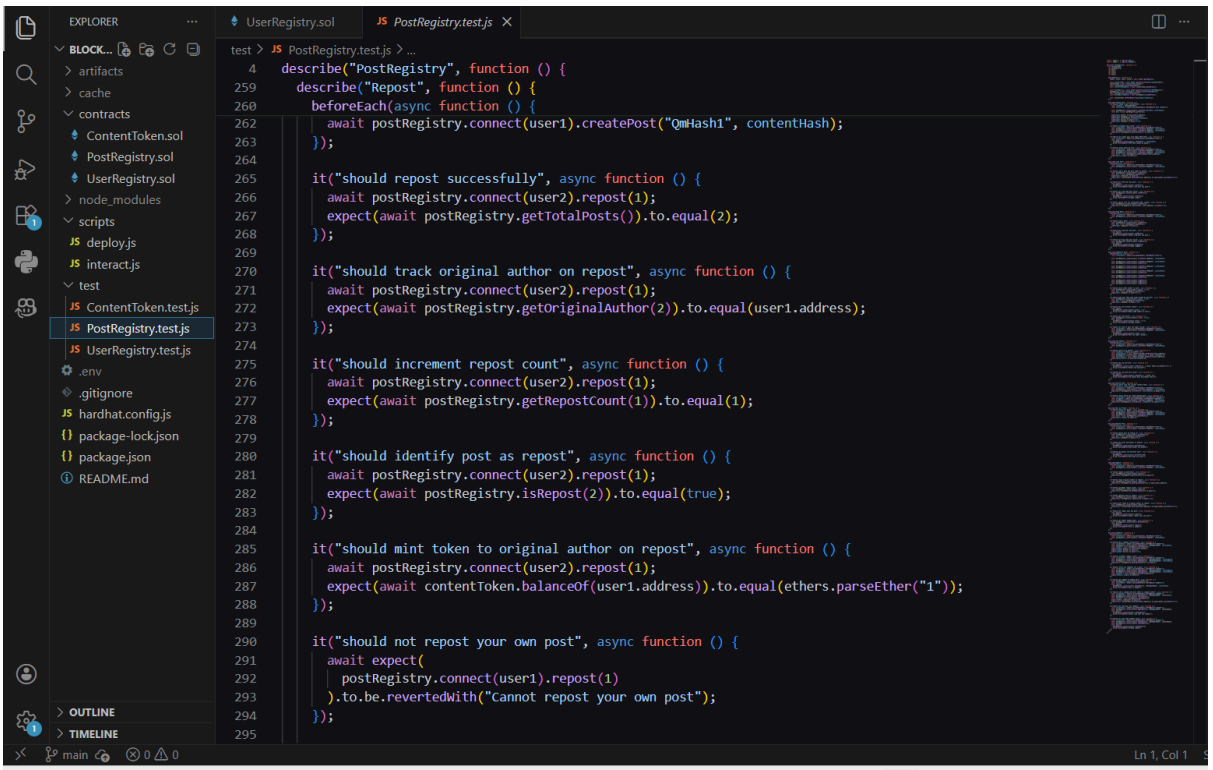

EXPLORER
BLOCKCHAIN-SOCIAL
artifacts
cache
contracts
ContentToken.sol
PostRegistry.sol
UserRegistry.sol
node_modules
scripts
deploy.js
interact.js
test
ContentToken.test.js
PostRegistry.test.js
UserRegistry.test.js
.env
.gitignore
hardhat.config.js
package-lock.json
package.json
README.md
OUTLINE
TIMELINE
UserRegistry.sol
PostRegistry.test.js
test > PostRegistry.test.js > ...
describe("PostRegistry", function () {
describe("Create Post", function () {
it("should increment post count", async function () {
const contentHash = ethers.keccak256(ethers.toUtf8Bytes("test"));
await postRegistry.connect(user1).createPost("QmHash1", contentHash);
await postRegistry.connect(user1).createPost("QmHash2", contentHash);
expect(await postRegistry.getTotalPosts()).to.equal(2);
});
it("should not create post with empty IPFS hash", async function () {
const contentHash = ethers.keccak256(ethers.toUtf8Bytes("test"));
await expect(
postRegistry.connect(user1).createPost("", contentHash)
).to.be.revertedWith("IPFS hash cannot be empty");
});
it("should return posts by user", async function () {
const contentHash = ethers.keccak256(ethers.toUtf8Bytes("test"));
await postRegistry.connect(user1).createPost("QmHash1", contentHash);
await postRegistry.connect(user1).createPost("QmHash2", contentHash);
const posts = await postRegistry.getPostsByUser(user1.address);
expect(posts.length).to.equal(2);
});
});
describe("Like Post", function () {
beforeEach(async function () {
const contentHash = ethers.keccak256(ethers.toUtf8Bytes("test"));
await postRegistry.connect(user1).createPost("QmHash1", contentHash);
});
it("should like a post and mint token to author", async function () {
await postRegistry.connect(user2).likePost(1);
const post = await postRegistry.getPost(1);
expect(post.likeCount).to.equal(1);
expect(await contentToken.balanceOf(user1.address)).to.equal(ethers.parseEther("1"));
});


EXPLORER
BLOCKCHAIN-SOCIAL
artifacts
cache
contracts
ContentToken.sol
PostRegistry.sol
UserRegistry.sol
node_modules
scripts
deploy.js
interact.js
test
ContentToken.test.js
PostRegistry.test.js
UserRegistry.test.js
.env
.gitignore
hardhat.config.js
package-lock.json
package.json
README.md
OUTLINE
TIMELINE
UserRegistry.sol
PostRegistry.test.js
test > PostRegistry.test.js > ...
4 describe("PostRegistry", function () {
66 describe("Like Post", function () {
85 it("should not like same post twice", async function () {
87 await expect(
88 postRegistry.connect(user2).likePost(1)
89).to.be.revertedWith("Already liked");
90 });
91
92 it("should return true for checkLiked after liking", async function () {
93 await postRegistry.connect(user2).likePost(1);
94 expect(await postRegistry.checkLiked(1, user2.address)).to.equal(true);
95 });
96 });
97
98 describe("Flag Post", function () {
99 beforeEach(async function () {
100 const contentHash = ethers.keccak256(ethers.toUtf8Bytes("test"));
101 await postRegistry.connect(user1).createPost("QmHash1", contentHash);
102 });
103
104 it("should flag a post", async function () {
105 await postRegistry.connect(user2).flagPost(1);
106 const post = await postRegistry.getPost(1);
107 expect(post.flagCount).to.equal(1);
108 });
109
110 it("should not flag your own post", async function () {
111 await expect(
112 postRegistry.connect(user1).flagPost(1)
113).to.be.revertedWith("Cannot flag your own post");
114 });
115
116 it("should not flag same post twice", async function () {
117 await postRegistry.connect(user2).flagPost(1);
118 await expect(
119 postRegistry.connect(user2).flagPost(1)
main
Ln 1, Col


EXPLORER
BLOCK...
artifacts
cache
contracts
ContentToken.sol
PostRegistry.sol
UserRegistry.sol
node_modules
scripts
deploy.js
interact.js
test
ContentToken.test.js
PostRegistry.test.js
UserRegistry.test.js
.env
.gitignore
hardhat.config.js
package-lock.json
package.json
README.md
OUTLINE
TIMELINE
UserRegistry.sol
PostRegistry.test.js
test > PostRegistry.test.js > ...
4 describe("PostRegistry", function () {
124 describe("Community Vote", function () {
125 beforeEach(async function () {
126 const contentHash = ethers.keccak256(ethers.toUtf8Bytes("test"));
127
128 await postRegistry.connect(user1).createPost("QmHash1", contentHash);
129
130 await postRegistry.connect(user2).createPost("QmHash2", contentHash);
131 await postRegistry.connect(user3).likePost(2);
132
133 await postRegistry.connect(user2).createPost("QmHash3", contentHash);
134 await postRegistry.connect(user3).likePost(3);
135
136 await postRegistry.connect(user2).createPost("QmHash4", contentHash);
137 await postRegistry.connect(user3).likePost(4);
138
139 await postRegistry.connect(user3).createPost("QmHash5", contentHash);
140 await postRegistry.connect(user2).likePost(5);
141
142 await postRegistry.connect(user2).flagPost(1);
143 await postRegistry.connect(user3).flagPost(1);
144 await postRegistry.connect(owner).flagPost(1);
145 });
146
147 it("should allow token holder to vote", async function () {
148 await postRegistry.connect(user2).vote(1, false);
149 const post = await postRegistry.getPost(1);
150 expect(post.isHidden).to.equal(false);
151 });
152
153 it("should hide post when hide votes exceed 51 percent", async function () {
154 await postRegistry.connect(user2).vote(1, true);
155 const post = await postRegistry.getPost(1);
156 expect(post.isHidden).to.equal(true);
157 });
158
159 it("should not vote without tokens", async function () {
main
Ln 1, Col 1


EXPLORER
BLOCKCHAIN-SOCIAL
artifacts
cache
contracts
ContentToken.sol
PostRegistry.sol
UserRegistry.sol
node_modules
scripts
deploy.js
interact.js
test
ContentToken.test.js
PostRegistry.test.js
UserRegistry.test.js
.env
.gitignore
hardhat.config.js
package-lock.json
package.json
README.md
OUTLINE
TIMELINE
UserRegistry.sol
PostRegistry.test.js
test > PostRegistry.test.js > ...
4 describe("PostRegistry", function () {
124 describe("Community Vote", function () {
165 it("should not vote twice", async function () {
168 postRegistry.connect(user2).vote(1, false)
169).to.be.revertedWith("Already voted");
170 });
171
172 it("should not vote if post not under review", async function () {
173 const contentHash = ethers.keccak256(ethers.toUtf8Bytes("test2"));
174 await postRegistry.connect(user1).createPost("QmHash6", contentHash);
175 await expect(
176 postRegistry.connect(user2).vote(6, true)
177).to.be.revertedWith("Post not under review");
178 });
179 });
180
181 describe("Tip Author", function () {
182 beforeEach(async function () {
183 const contentHash = ethers.keccak256(ethers.toUtf8Bytes("test"));
184 await postRegistry.connect(user1).createPost("QmHash1", contentHash);
185 });
186
187 it("should send tip to author", async function () {
188 const tipAmount = ethers.parseEther("1");
189 const balanceBefore = await ethers.provider.getBalance(user1.address);
190 await postRegistry.connect(user2).tipAuthor(1, { value: tipAmount });
191 const balanceAfter = await ethers.provider.getBalance(user1.address);
192 expect(balanceAfter).to.be.gt(balanceBefore);
193 });
194
195 it("should not tip yourself", async function () {
196 await expect(
197 postRegistry.connect(user1).tipAuthor(1, { value: ethers.parseEther("1") })
198).to.be.revertedWith("Cannot tip yourself");
199 });
200
201 it("should not tip with zero value", async function () {
main
0 0
Ln 1, Col 1


EXPLORER
BLOCKCHAIN-SOCIAL
artifacts
cache
contracts
ContentToken.sol
PostRegistry.sol
UserRegistry.sol
node_modules
scripts
deploy.js
interact.js
test
ContentToken.test.js
PostRegistry.test.js
UserRegistry.test.js
.env
.gitignore
hardhat.config.js
package-lock.json
package.json
README.md
OUTLINE
TIMELINE
UserRegistry.sol
PostRegistry.test.js
test > PostRegistry.test.js > ...
4 describe("PostRegistry", function () {
208 describe("Verify Post", function () {
209 it("should return true for correct content hash", async function () {
212 await postRegistry.connect(user1).createPost("QmHash1", contentHash);
213 expect(await postRegistry.verifyPost(1, contentHash)).to.equal(true);
214 });
215
216 it("should return false for wrong content hash", async function () {
217 const contentHash = ethers.keccak256(ethers.toUtf8Bytes("original"));
218 const wrongHash = ethers.keccak256(ethers.toUtf8Bytes("tampered"));
219 await postRegistry.connect(user1).createPost("QmHash1", contentHash);
220 expect(await postRegistry.verifyPost(1, wrongHash)).to.equal(false);
221 });
222 });
223
224 describe("Get All Posts", function () {
225 it("should return all posts", async function () {
226 const contentHash = ethers.keccak256(ethers.toUtf8Bytes("test"));
227 await postRegistry.connect(user1).createPost("QmHash1", contentHash);
228 await postRegistry.connect(user2).createPost("QmHash2", contentHash);
229 const posts = await postRegistry.getAllPosts();
230 expect(posts.length).to.equal(2);
231 });
232 });
233
234 describe("Delete Post", function () {
235 beforeEach(async function () {
236 const contentHash = ethers.keccak256(ethers.toUtf8Bytes("test"));
237 await postRegistry.connect(user1).createPost("QmHash1", contentHash);
238 });
239
240 it("should delete post by hiding it", async function () {
241 await postRegistry.connect(user1).deletePost(1);
242 const post = await postRegistry.getPost(1);
243 expect(post.isHidden).to.equal(true);
244 });
245
main
0 0
Ln 1, Col 1



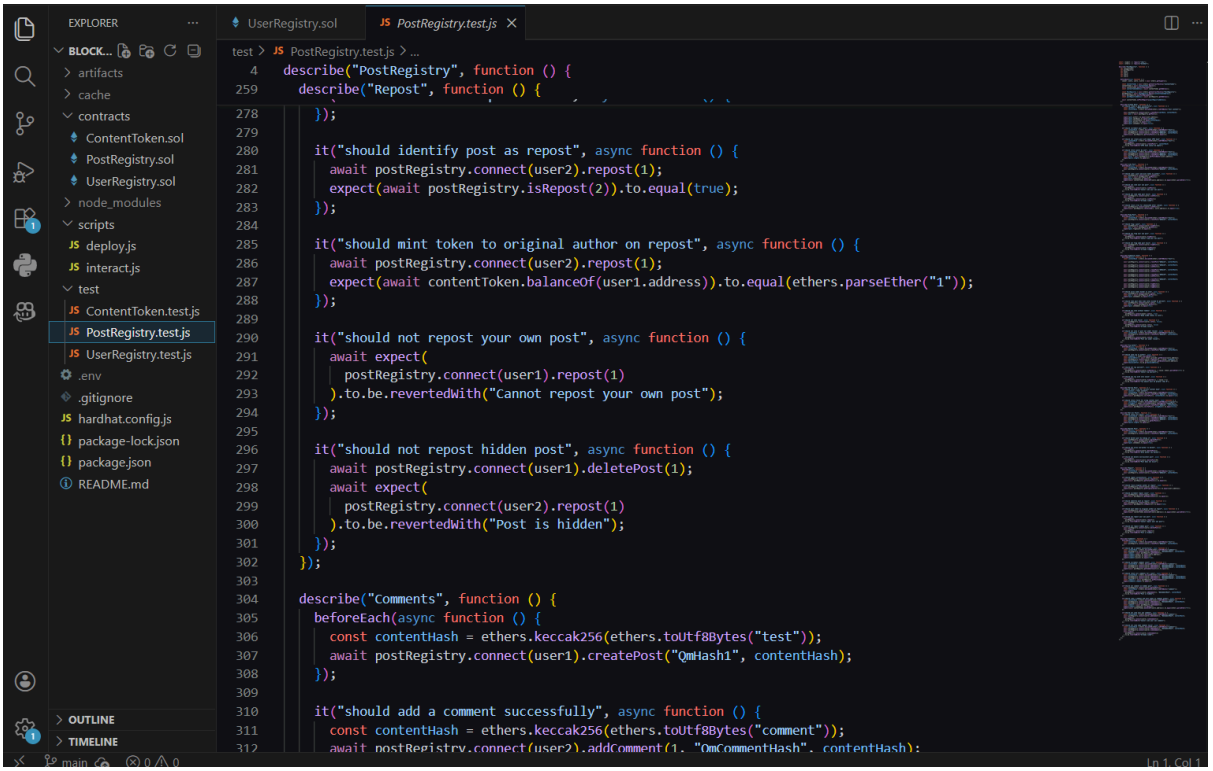
```
EXPLORER
BLOCK...
  artifacts
  cache
  contracts
    ContentToken.sol
    PostRegistry.sol
    UserRegistry.sol
  node_modules
  scripts
    deploy.js
    interact.js
  test
    ContentToken.test.js
    PostRegistry.test.js
    UserRegistry.test.js
  .env
  .gitignore
  hardhat.config.js
  package-lock.json
  package.json
  README.md
OUTLINE
TIMELINE
```

UserRegistry.sol | PostRegistry.test.js

test > PostRegistry.test.js > ...

```
4   describe("PostRegistry", function () {
259   describe("Repost", function () {
260     beforeEach(async function () {
262       await postRegistry.connect(user1).createPost("QmHash1", contentHash);
263     });
264
265     it("should repost successfully", async function () {
266       await postRegistry.connect(user2).repost(1);
267       expect(await postRegistry.getTotalPosts()).to.equal(2);
268     });
269
270     it("should track original author on repost", async function () {
271       await postRegistry.connect(user2).repost(1);
272       expect(await postRegistry.getOriginalAuthor(2)).to.equal(user1.address);
273     });
274
275     it("should increment repost count", async function () {
276       await postRegistry.connect(user2).repost(1);
277       expect(await postRegistry.getRepostCount(1)).to.equal(1);
278     });
279
280     it("should identify post as repost", async function () {
281       await postRegistry.connect(user2).repost(1);
282       expect(await postRegistry.isRepost(2)).to.equal(true);
283     });
284
285     it("should mint token to original author on repost", async function () {
286       await postRegistry.connect(user2).repost(1);
287       expect(await contentToken.balanceOf(user1.address)).to.equal(ethers.parseEther("1"));
288     });
289
290     it("should not repost your own post", async function () {
291       await expect(
292         postRegistry.connect(user1).repost(1)
293       ).to.be.revertedWith("Cannot repost your own post");
294     });
295
```

main ⊗ 0 ⚠ 0 — Ln 1, Col 1



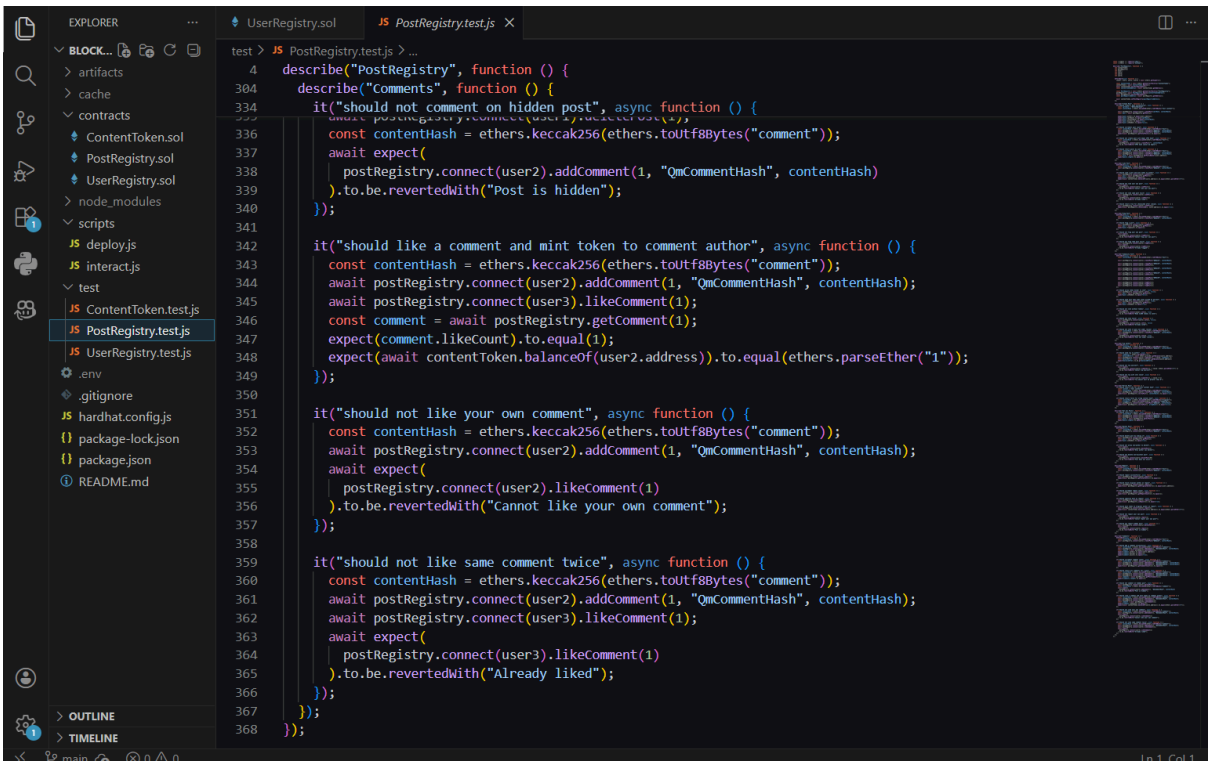
```
EXPLORER
BLOCK...
  artifacts
  cache
  contracts
    ContentToken.sol
    PostRegistry.sol
    UserRegistry.sol
  node_modules
  scripts
    deploy.js
    interact.js
  test
    ContentToken.test.js
    PostRegistry.test.js
    UserRegistry.test.js
  .env
  .gitignore
  hardhat.config.js
  package-lock.json
  package.json
  README.md
OUTLINE
TIMELINE
```

UserRegistry.sol | PostRegistry.test.js

test > PostRegistry.test.js > ...

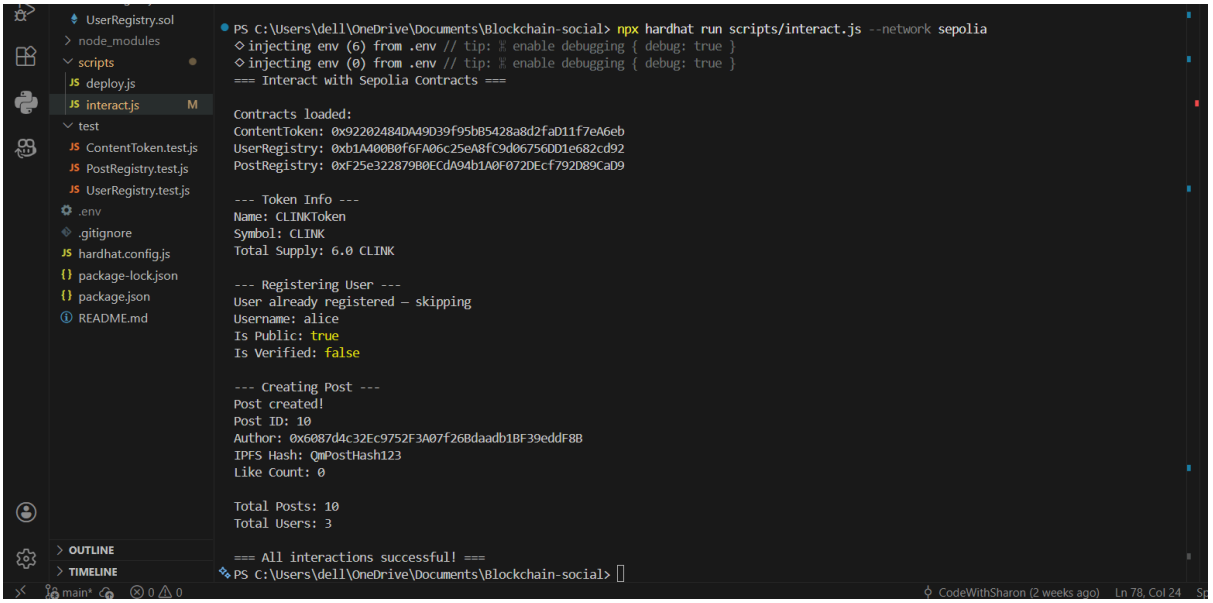
```
4   describe("PostRegistry", function () {
259   describe("Repost", function () {
278     });
279
280     it("should identify post as repost", async function () {
281       await postRegistry.connect(user2).repost(1);
282       expect(await postRegistry.isRepost(2)).to.equal(true);
283     });
284
285     it("should mint token to original author on repost", async function () {
286       await postRegistry.connect(user2).repost(1);
287       expect(await contentToken.balanceOf(user1.address)).to.equal(ethers.parseEther("1"));
288     });
289
290     it("should not repost your own post", async function () {
291       await expect(
292         postRegistry.connect(user1).repost(1)
293       ).to.be.revertedWith("Cannot repost your own post");
294     });
295
296     it("should not repost hidden post", async function () {
297       await postRegistry.connect(user1).deletePost(1);
298       await expect(
299         postRegistry.connect(user2).repost(1)
300       ).to.be.revertedWith("Post is hidden");
301     });
302   });
303
304   describe("Comments", function () {
305     beforeEach(async function () {
306       const contentHash = ethers.keccak256(ethers.toUtf8Bytes("test"));
307       await postRegistry.connect(user1).createPost("QmHash1", contentHash);
308     });
309
310     it("should add a comment successfully", async function () {
311       const contentHash = ethers.keccak256(ethers.toUtf8Bytes("comment"));
312       await postRegistry.connect(user2).addComment(1, "QmCommentHash", contentHash);
```

main ⊗ 0 ⚠ 0 — Ln 1, Col 1

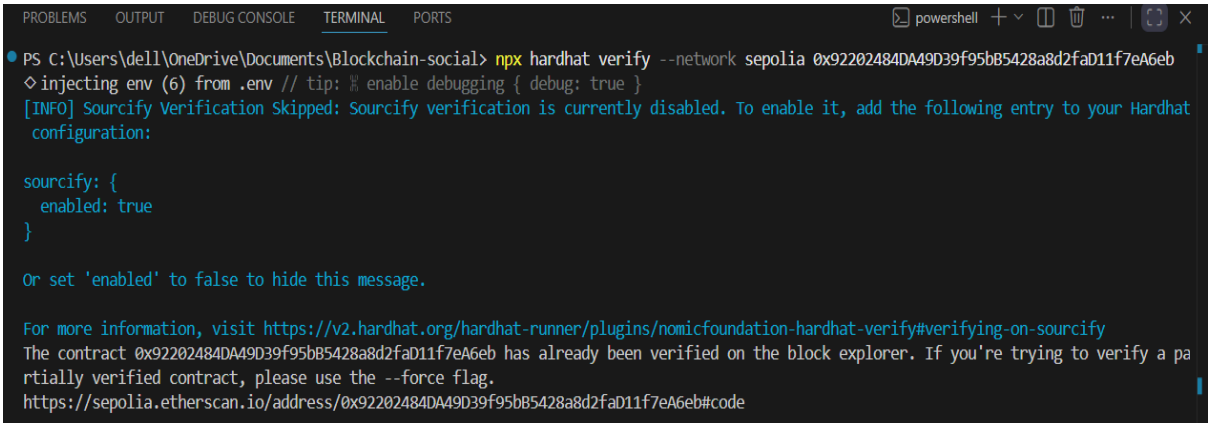


Output:

Deployment & Live Demo:



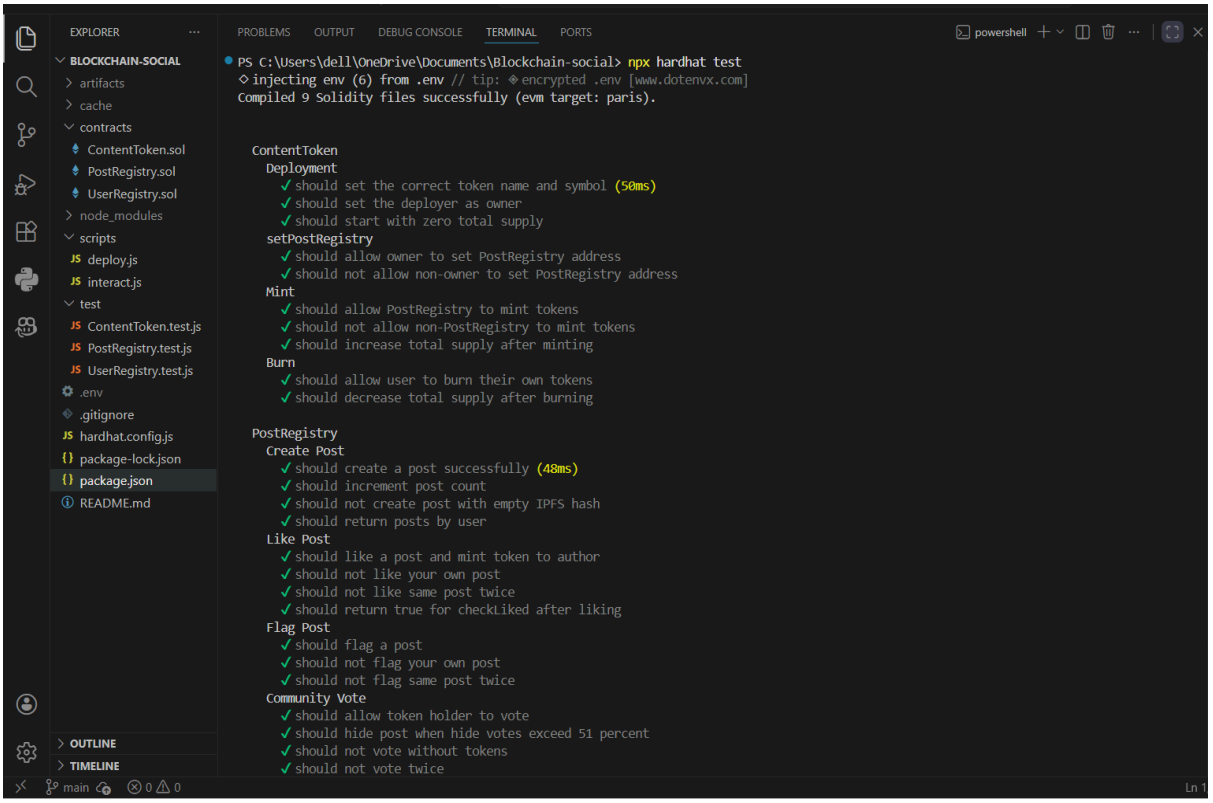
Smart Contract Verification:

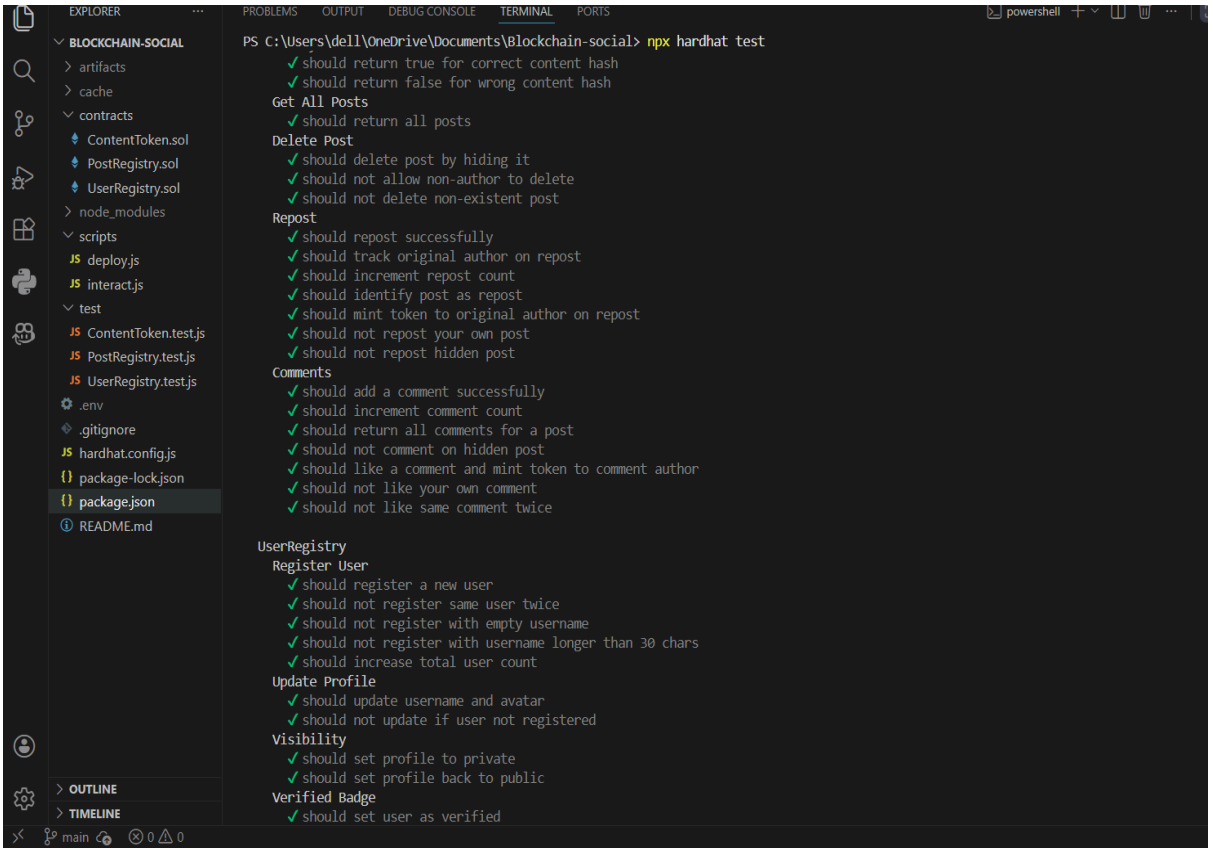


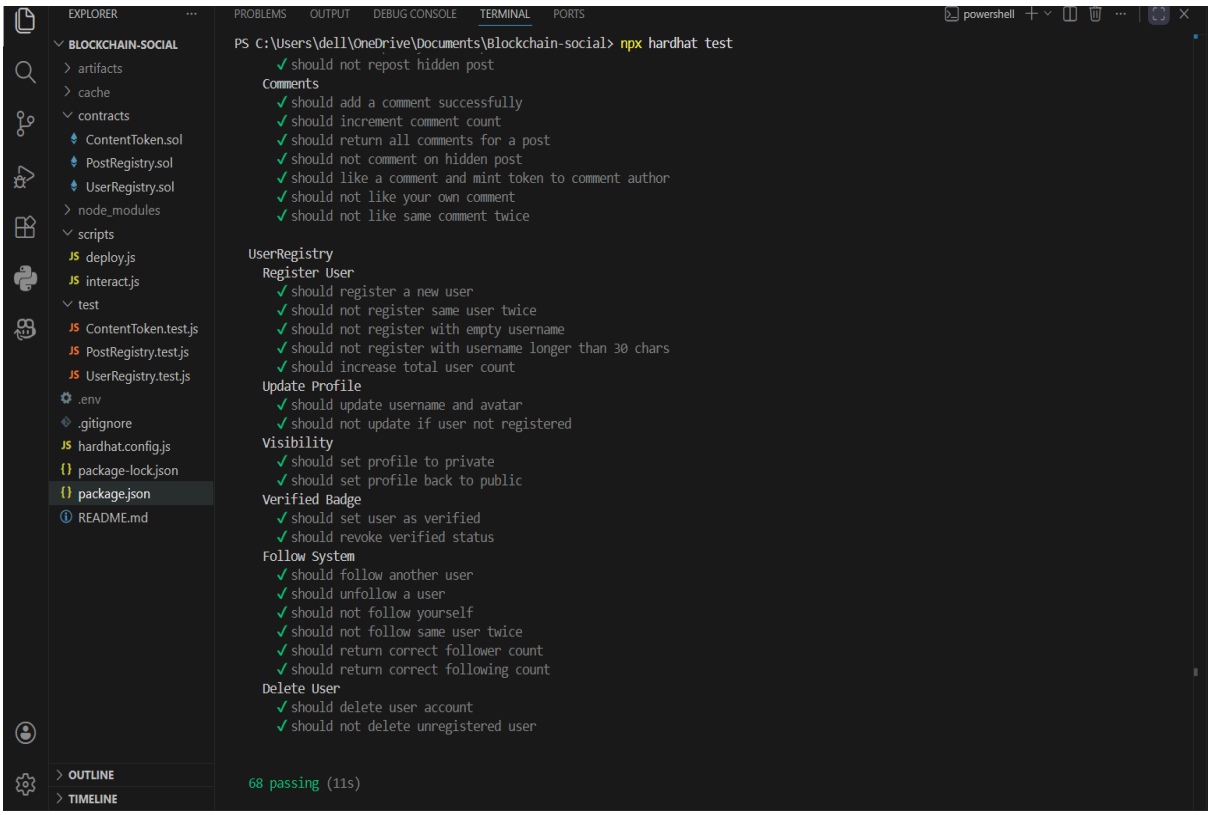




Testcases: 68 testcases passed







Frontend and backend

contracts.js :

blockchain-social-p2 > frontend > src > contracts.js > ...

```
export const CONTRACT_ADDRESSES = {
  UserRegistry: "0x..",
  PostRegistry: "0x..",
  ContentToken: "0x..",
};

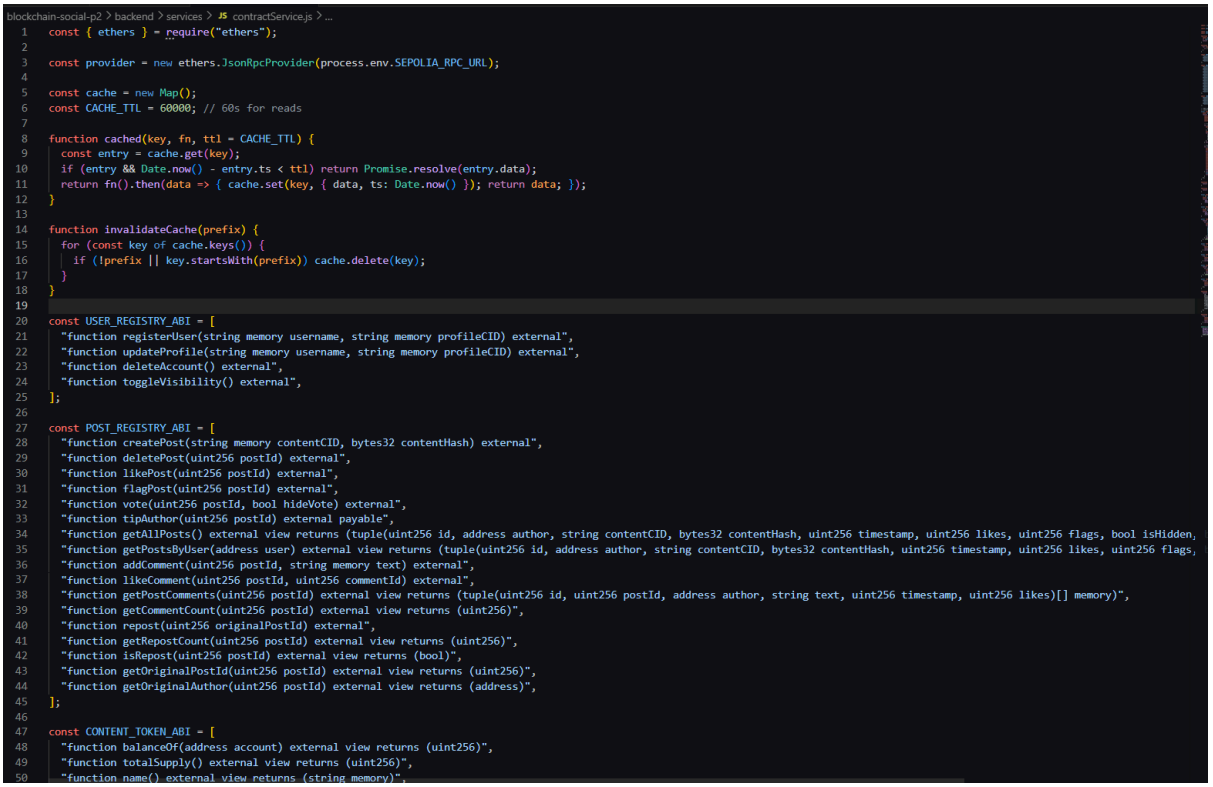
export const USER_REGISTRY_ABI = [
  "function registerUser(string memory username, string memory profileCID) external",
  "function updateProfile(string memory username, string memory profileCID) external",
  "function deleteAccount() external",
  "function toggleVisibility() external",
  "function getUser(address wallet) external view returns (address wallet, string username, string profileCID, bool isVisible, bool isDeleted, bool extra, uint256 timestamp)",
];

export const POST_REGISTRY_ABI = [
  "function createPost(string memory contentCID, bytes32 contentHash) external",
  "function deletePost(uint256 postId) external",
  "function likePost(uint256 postId) external",
  "function flagPost(uint256 postId) external",
  "function vote(uint256 postId, bool hideVote) external",
  "function tipAuthor(uint256 postId) external payable",
  "function getAllPosts() external view returns (tuple(uint256 id, address author, string contentCID, bytes32 contentHash, uint256 timestamp, uint256 likes, uint256 flags, bool isHidden)
  "function getPostsByUser(address user) external view returns (tuple(uint256 id, address author, string contentCID, bytes32 contentHash, uint256 timestamp, uint256 likes, uint256 flags,
  "function addComment(uint256 postId, string memory text) external",
  "function likeComment(uint256 postId, uint256 commentId) external",
  "function getPostComments(uint256 postId) external view returns (tuple(uint256 id, uint256 postId, address author, string text, uint256 timestamp, uint256 likes)[] memory)",
  "function getCommentCount(uint256 postId) external view returns (uint256)",
  "function repost(uint256 originalPostId) external",
  "function getRepostCount(uint256 postId) external view returns (uint256)",
  "function isRepost(uint256 postId) external view returns (bool)",
  "function getOriginalPostId(uint256 postId) external view returns (uint256)",
  "function getOriginalAuthor(uint256 postId) external view returns (address)",
];

export const CONTENT_TOKEN_ABI = [
  "function balanceOf(address account) external view returns (uint256)",
  "function totalSupply() external view returns (uint256)",
  "function name() external view returns (string memory)",
  "function symbol() external view returns (string memory)",
];

export const SEPOLIA_CHAIN_ID = "0xaa36a7";
```

Contract Services.js :



```
blockchain-social-p2 > backend > services > JS contractService.js > ...
const { ethers } = require("ethers");

const provider = new ethers.JsonRpcProvider(process.env.SEPOLIA_RPC_URL);

const cache = new Map();
const CACHE_TTL = 60000; // 60s for reads

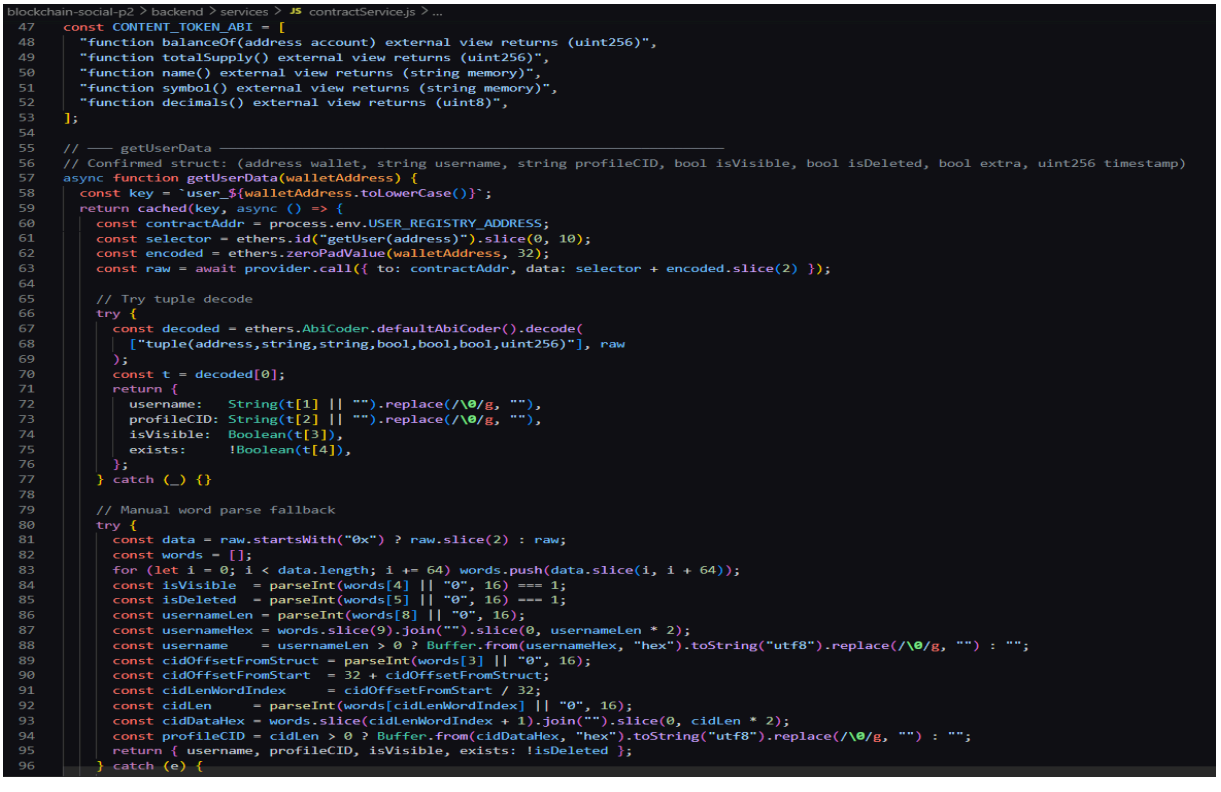
function cached(key, fn, ttl = CACHE_TTL) {
  const entry = cache.get(key);
  if (entry && Date.now() - entry.ts < ttl) return Promise.resolve(entry.data);
  return fn().then(data => { cache.set(key, { data, ts: Date.now() }); return data; });
}

function invalidateCache(prefix) {
  for (const key of cache.keys()) {
    if (!prefix || key.startsWith(prefix)) cache.delete(key);
  }
}

const USER_REGISTRY_ABI = [
  "function registerUser(string memory username, string memory profileCID) external",
  "function updateProfile(string memory username, string memory profileCID) external",
  "function deleteAccount() external",
  "function toggleVisibility() external",
];

const POST_REGISTRY_ABI = [
  "function createPost(string memory contentCID, bytes32 contentHash) external",
  "function deletePost(uint256 postId) external",
  "function likePost(uint256 postId) external",
  "function flagPost(uint256 postId) external",
  "function vote(uint256 postId, bool hideVote) external",
  "function tipAuthor(uint256 postId) external payable",
  "function getAllPosts() external view returns (tuple(uint256 id, address author, string contentCID, bytes32 contentHash, uint256 timestamp, uint256 likes, uint256 flags, bool isHidden,
  "function getPostsByUser(address user) external view returns (tuple(uint256 id, address author, string contentCID, bytes32 contentHash, uint256 timestamp, uint256 likes, uint256 flags,
  "function addComment(uint256 postId, string memory text) external",
  "function likeComment(uint256 postId, uint256 commentId) external",
  "function getPostComments(uint256 postId) external view returns (tuple(uint256 id, uint256 postId, address author, string text, uint256 timestamp, uint256 likes)[] memory)",
  "function getCommentCount(uint256 postId) external view returns (uint256)",
  "function repost(uint256 originalPostId) external",
  "function getRepostCount(uint256 postId) external view returns (uint256)",
  "function isRepost(uint256 postId) external view returns (bool)",
  "function getOriginalPostId(uint256 postId) external view returns (uint256)",
  "function getOriginalAuthor(uint256 postId) external view returns (address)",
];

const CONTENT_TOKEN_ABI = [
  "function balanceOf(address account) external view returns (uint256)",
  "function totalSupply() external view returns (uint256)",
  "function name() external view returns (string memory)",
```

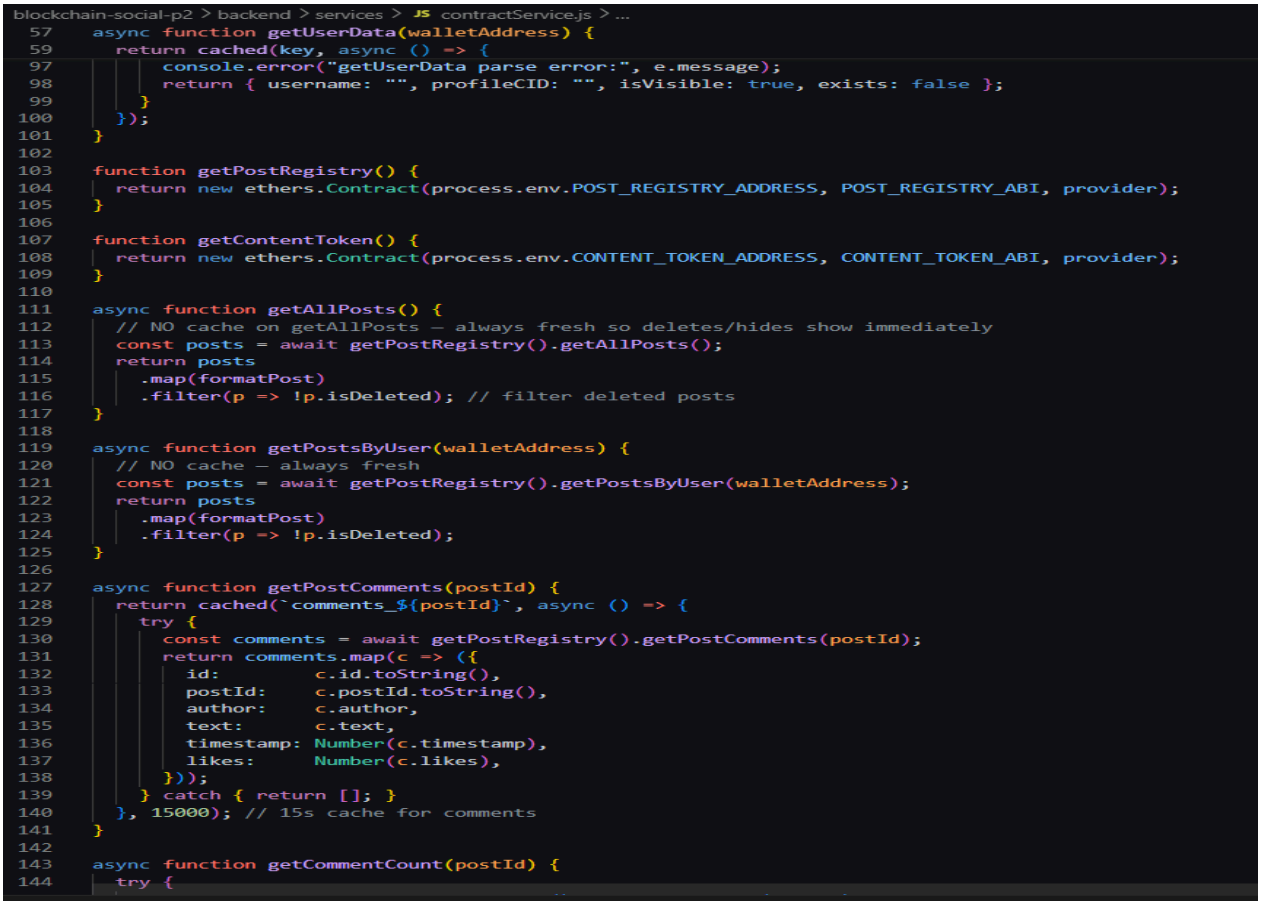


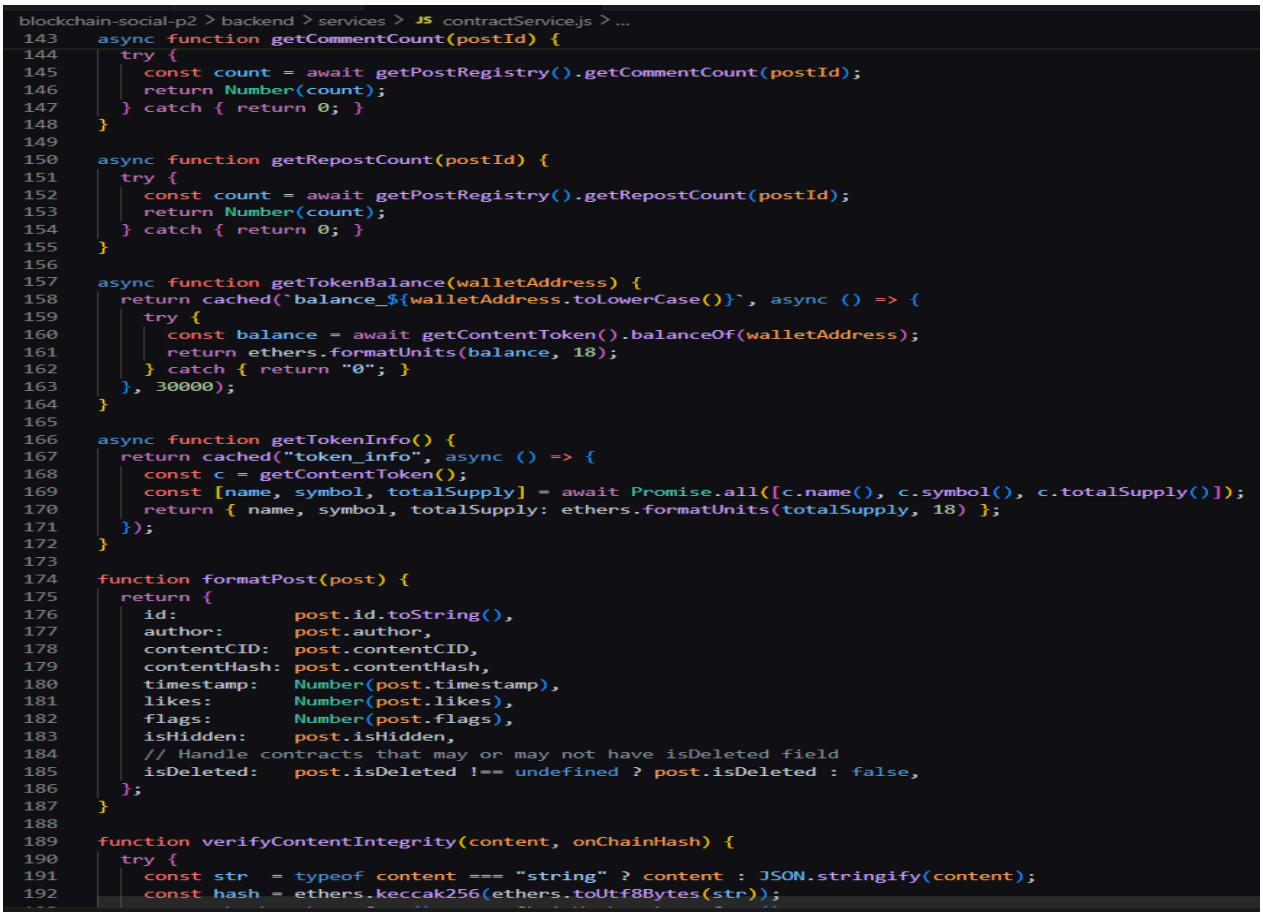
```
blockchain-social-p2 > backend > services > JS contractService.js > ...
  const CONTENT_TOKEN_ABI = [
    "function balanceOf(address account) external view returns (uint256)",
    "function totalSupply() external view returns (uint256)",
    "function name() external view returns (string memory)",
    "function symbol() external view returns (string memory)",
    "function decimals() external view returns (uint8)",
  ];

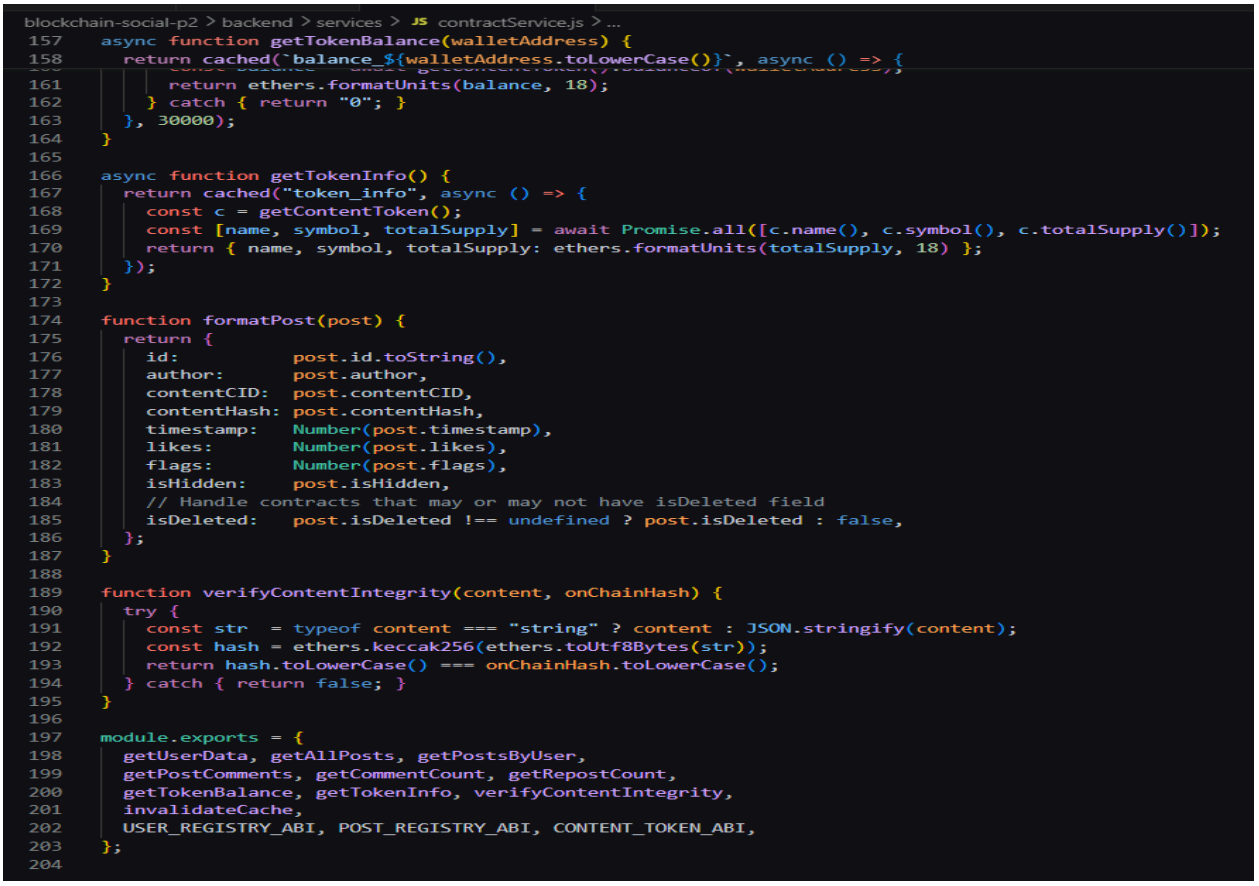
  // ─── getUserData ──────────────────────────────────────────────────────
  // Confirmed struct: (address wallet, string username, string profileCID, bool isVisible, bool isDeleted, bool extra, uint256 timestamp)
  async function getUserData(walletAddress) {
    const key = `user_${walletAddress.toLowerCase()}`;
    return cached(key, async () => {
      const contractAddr = process.env.USER_REGISTRY_ADDRESS;
      const selector = ethers.id("getUser(address)").slice(0, 10);
      const encoded = ethers.zeroPadValue(walletAddress, 32);
      const raw = await provider.call({ to: contractAddr, data: selector + encoded.slice(2) });

      // Try tuple decode
      try {
        const decoded = ethers.AbiCoder.defaultAbiCoder().decode(
          ["tuple(address,string,string,bool,bool,bool,uint256)"], raw
        );
        const t = decoded[0];
        return {
          username:   String(t[1] || "").replace(/\0/g, ""),
          profileCID: String(t[2] || "").replace(/\0/g, ""),
          isVisible:  Boolean(t[3]),
          exists:     !Boolean(t[4]),
        };
      } catch (_) {}

      // Manual word parse fallback
      try {
        const data = raw.startsWith("0x") ? raw.slice(2) : raw;
        const words = [];
        for (let i = 0; i < data.length; i += 64) words.push(data.slice(i, i + 64));
        const isVisible  = parseInt(words[4] || "0", 16) === 1;
        const isDeleted  = parseInt(words[5] || "0", 16) === 1;
        const usernameLen = parseInt(words[8] || "0", 16);
        const usernameHex = words.slice(9).join("").slice(0, usernameLen * 2);
        const username    = usernameLen > 0 ? Buffer.from(usernameHex, "hex").toString("utf8").replace(/\0/g, "") : "";
        const cidOffsetFromStruct = parseInt(words[3] || "0", 16);
        const cidOffsetFromStart  = 32 + cidOffsetFromStruct;
        const cidLenWordIndex     = cidOffsetFromStart / 32;
        const cidLen      = parseInt(words[cidLenWordIndex] || "0", 16);
        const cidDataHex = words.slice(cidLenWordIndex + 1).join("").slice(0, cidLen * 2);
        const profileCID = cidLen > 0 ? Buffer.from(cidDataHex, "hex").toString("utf8").replace(/\0/g, "") : "";
        return { username, profileCID, isVisible, exists: !isDeleted };
      } catch (e) {
```

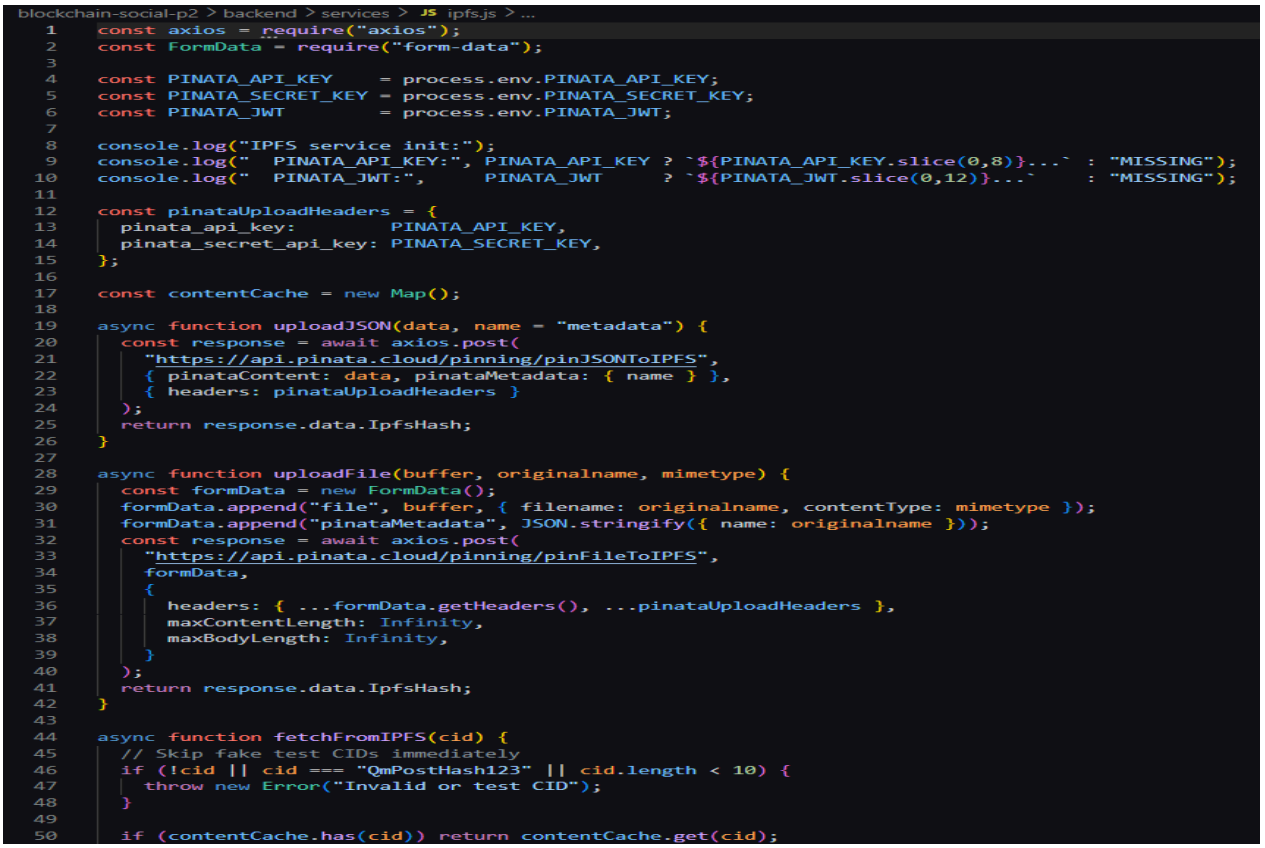

blockchain-social-p2 > backend > services > JS contractService.js > ...
57 async function getUserData(walletAddress) {
59 return cached(key, async () => {
97 console.error("getUserData parse error:", e.message);
98 return { username: "", profileCID: "", isVisible: true, exists: false };
99 }
100 });
101 }
102
103 function getPostRegistry() {
104 return new ethers.Contract(process.env.POST_REGISTRY_ADDRESS, POST_REGISTRY_ABI, provider);
105 }
106
107 function getContentToken() {
108 return new ethers.Contract(process.env.CONTENT_TOKEN_ADDRESS, CONTENT_TOKEN_ABI, provider);
109 }
110
111 async function getAllPosts() {
112 // NO cache on getAllPosts — always fresh so deletes/hides show immediately
113 const posts = await getPostRegistry().getAllPosts();
114 return posts
115 .map(formatPost)
116 .filter(p => !p.isDeleted); // filter deleted posts
117 }
118
119 async function getPostsByUser(walletAddress) {
120 // NO cache — always fresh
121 const posts = await getPostRegistry().getPostsByUser(walletAddress);
122 return posts
123 .map(formatPost)
124 .filter(p => !p.isDeleted);
125 }
126
127 async function getPostComments(postId) {
128 return cached(`comments_${postId}`, async () => {
129 try {
130 const comments = await getPostRegistry().getPostComments(postId);
131 return comments.map(c => ({
132 id: c.id.toString(),
133 postId: c.postId.toString(),
134 author: c.author,
135 text: c.text,
136 timestamp: Number(c.timestamp),
137 likes: Number(c.likes),
138 }));
139 } catch { return []; }
140 }, 15000); // 15s cache for comments
141 }
142
143 async function getCommentCount(postId) {
144 try {


blockchain-social-p2 > backend > services > JS contractService.js > ...
143 async function getCommentCount(postId) {
144 try {
145 const count = await getPostRegistry().getCommentCount(postId);
146 return Number(count);
147 } catch { return 0; }
148 }
149
150 async function getRepostCount(postId) {
151 try {
152 const count = await getPostRegistry().getRepostCount(postId);
153 return Number(count);
154 } catch { return 0; }
155 }
156
157 async function getTokenBalance(walletAddress) {
158 return cached(`balance_${walletAddress.toLowerCase()}`, async () => {
159 try {
160 const balance = await getContentToken().balanceOf(walletAddress);
161 return ethers.formatUnits(balance, 18);
162 } catch { return "0"; }
163 }, 30000);
164 }
165
166 async function getTokenInfo() {
167 return cached("token_info", async () => {
168 const c = getContentToken();
169 const [name, symbol, totalSupply] = await Promise.all([c.name(), c.symbol(), c.totalSupply()]);
170 return { name, symbol, totalSupply: ethers.formatUnits(totalSupply, 18) };
171 });
172 }
173
174 function formatPost(post) {
175 return {
176 id: post.id.toString(),
177 author: post.author,
178 contentCID: post.contentCID,
179 contentHash: post.contentHash,
180 timestamp: Number(post.timestamp),
181 likes: Number(post.likes),
182 flags: Number(post.flags),
183 isHidden: post.isHidden,
184 // Handle contracts that may or may not have isDeleted field
185 isDeleted: post.isDeleted !== undefined ? post.isDeleted : false,
186 };
187 }
188
189 function verifyContentIntegrity(content, onChainHash) {
190 try {
191 const str = typeof content === "string" ? content : JSON.stringify(content);
192 const hash = ethers.keccak256(ethers.toUtf8Bytes(str));



```
blockchain-social-p2 > backend > services > JS contractService.js > ...
157   async function getTokenBalance(walletAddress) {
158     return cached(`balance_${walletAddress.toLowerCase()}`, async () => {
161       return ethers.formatUnits(balance, 18);
162     } catch { return "0"; }
163   }, 30000);
164   }
165
166   async function getTokenInfo() {
167     return cached("token_info", async () => {
168       const c = getContentToken();
169       const [name, symbol, totalSupply] = await Promise.all([c.name(), c.symbol(), c.totalSupply()]);
170       return { name, symbol, totalSupply: ethers.formatUnits(totalSupply, 18) };
171     });
172   }
173
174   function formatPost(post) {
175     return {
176       id:          post.id.toString(),
177       author:      post.author,
178       contentCID:  post.contentCID,
179       contentHash: post.contentHash,
180       timestamp:   Number(post.timestamp),
181       likes:       Number(post.likes),
182       flags:       Number(post.flags),
183       isHidden:    post.isHidden,
184       // Handle contracts that may or may not have isDeleted field
185       isDeleted:   post.isDeleted !== undefined ? post.isDeleted : false,
186     };
187   }
188
189   function verifyContentIntegrity(content, onChainHash) {
190     try {
191       const str  = typeof content === "string" ? content : JSON.stringify(content);
192       const hash = ethers.keccak256(ethers.toUtf8Bytes(str));
193       return hash.toLowerCase() === onChainHash.toLowerCase();
194     } catch { return false; }
195   }
196
197   module.exports = {
198     getUserData, getAllPosts, getPostsByUser,
199     getPostComments, getCommentCount, getRepostCount,
200     getTokenBalance, getTokenInfo, verifyContentIntegrity,
201     invalidateCache,
202     USER_REGISTRY_ABI, POST_REGISTRY_ABI, CONTENT_TOKEN_ABI,
203   };
204
```

ipfs.js :



```
blockchain-social-p2 > backend > services > JS ipfs.js > ...
1   const axios = require("axios");
2   const FormData = require("form-data");
3
4   const PINATA_API_KEY    = process.env.PINATA_API_KEY;
5   const PINATA_SECRET_KEY = process.env.PINATA_SECRET_KEY;
6   const PINATA_JWT        = process.env.PINATA_JWT;
7
8   console.log("IPFS service init:");
9   console.log("  PINATA_API_KEY:", PINATA_API_KEY ? `${PINATA_API_KEY.slice(0,8)}...` : "MISSING");
10  console.log("  PINATA_JWT:",     PINATA_JWT     ? `${PINATA_JWT.slice(0,12)}...`    : "MISSING");
11
12  const pinataUploadHeaders = {
13    pinata_api_key:        PINATA_API_KEY,
14    pinata_secret_api_key: PINATA_SECRET_KEY,
15  };
16
17  const contentCache = new Map();
18
19  async function uploadJSON(data, name = "metadata") {
20    const response = await axios.post(
21      "https://api.pinata.cloud/pinning/pinJSONToIPFS",
22      { pinataContent: data, pinataMetadata: { name } },
23      { headers: pinataUploadHeaders }
24    );
25    return response.data.IpfsHash;
26  }
27
28  async function uploadFile(buffer, originalname, mimetype) {
29    const formData = new FormData();
30    formData.append("file", buffer, { filename: originalname, contentType: mimetype });
31    formData.append("pinataMetadata", JSON.stringify({ name: originalname }));
32    const response = await axios.post(
33      "https://api.pinata.cloud/pinning/pinFileToIPFS",
34      formData,
35      {
36        headers: { ...formData.getHeaders(), ...pinataUploadHeaders },
37        maxContentLength: Infinity,
38        maxBodyLength: Infinity,
39      }
40    );
41    return response.data.IpfsHash;
42  }
43
44  async function fetchFromIPFS(cid) {
45    // Skip fake test CIDs immediately
46    if (!cid || cid === "QmPostHash123" || cid.length < 10) {
47      throw new Error("Invalid or test CID");
48    }
49
50    if (contentCache.has(cid)) return contentCache.get(cid);
```

```
blockchain-social-p2 > backend > services > JS ipfs.js > ...
  async function fetchFromIPFS(cid) {
    if (PINATA_JWT) {
      try {
        const res = await axios.get(`https://gateway.pinata.cloud/ipfs/${cid}`, {
          headers: { Authorization: `Bearer ${PINATA_JWT}` },
          timeout: 20000,
        });
        if (res.data) {
          console.log("✅ IPFS fetch success (Pinata auth)");
          contentCache.set(cid, res.data);
          return res.data;
        }
      } catch (e) {
        console.warn(`❌ Pinata auth failed: ${e.response?.status} ${e.message}`);
      }
    }

    // Method 2: ipfs.io (no auth needed, good uptime)
    try {
      const res = await axios.get(`https://ipfs.io/ipfs/${cid}`, { timeout: 20000 });
      if (res.data) {
        console.log("✅ IPFS fetch success (ipfs.io)");
        contentCache.set(cid, res.data);
        return res.data;
      }
    } catch (e) {
      console.warn(`❌ ipfs.io failed: ${e.response?.status} ${e.message}`);
    }

    // Method 3: nftstorage link
    try {
      const res = await axios.get(`https://${cid}.ipfs.nftstorage.link/`, { timeout: 20000 });
      if (res.data) {
        console.log("✅ IPFS fetch success (nftstorage)");
        contentCache.set(cid, res.data);
        return res.data;
      }
    } catch (e) {
      console.warn(`❌ nftstorage failed: ${e.response?.status} ${e.message}`);
    }

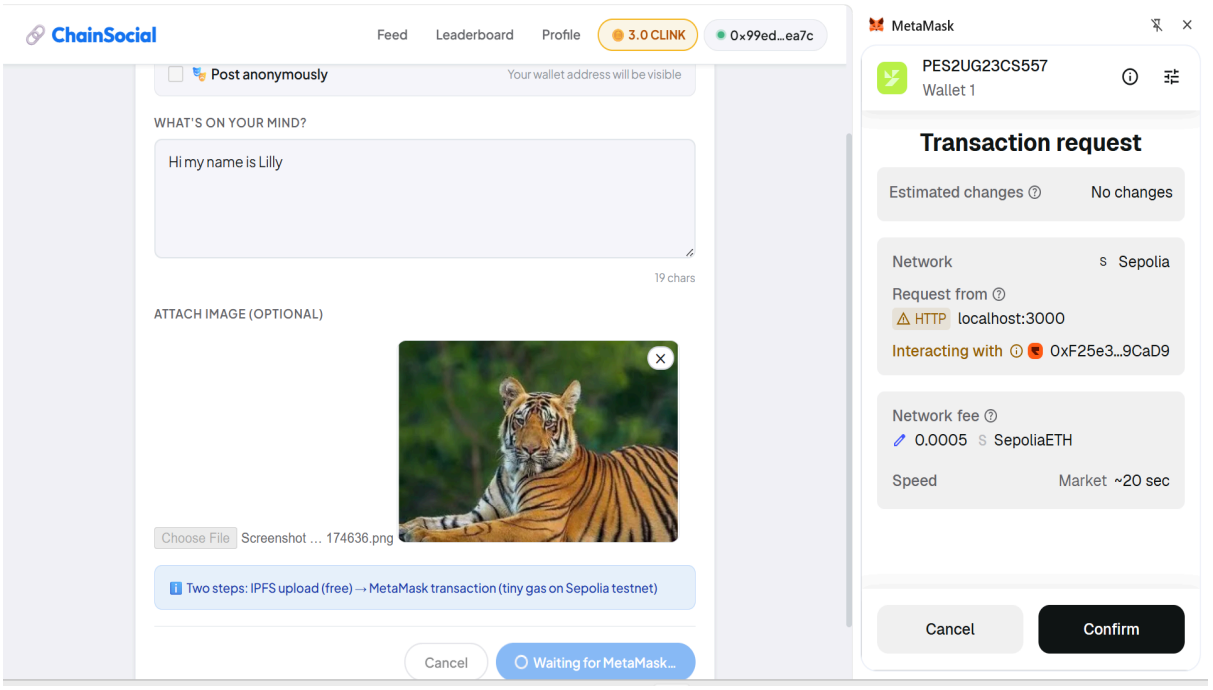
    throw new Error("All IPFS gateways failed");
  }

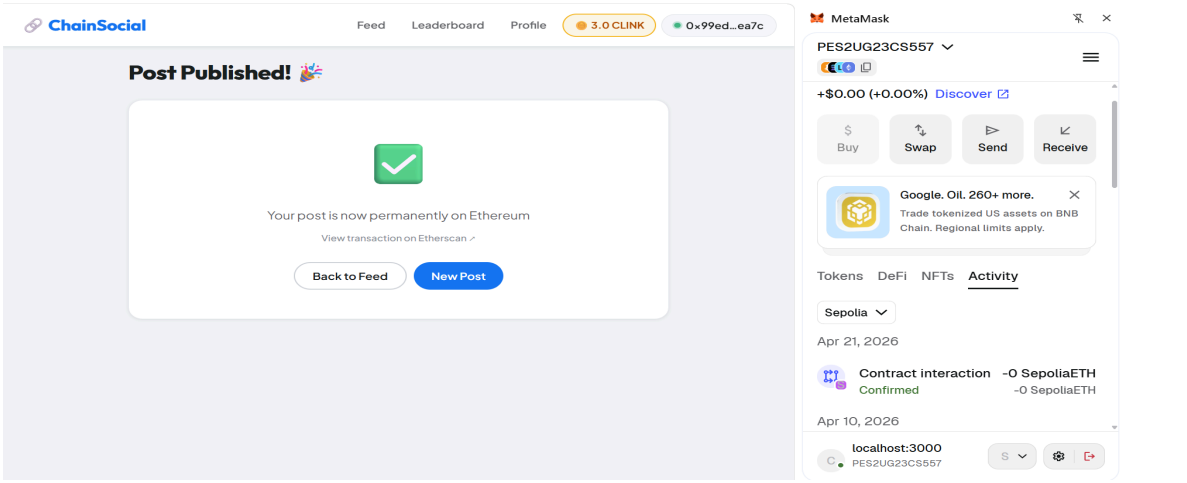
  function getIPFSUrl(cid) {
    if (!cid || cid === "QmPostHash123") return null;
    return `/api/ipfs/image/${cid}`;
  }

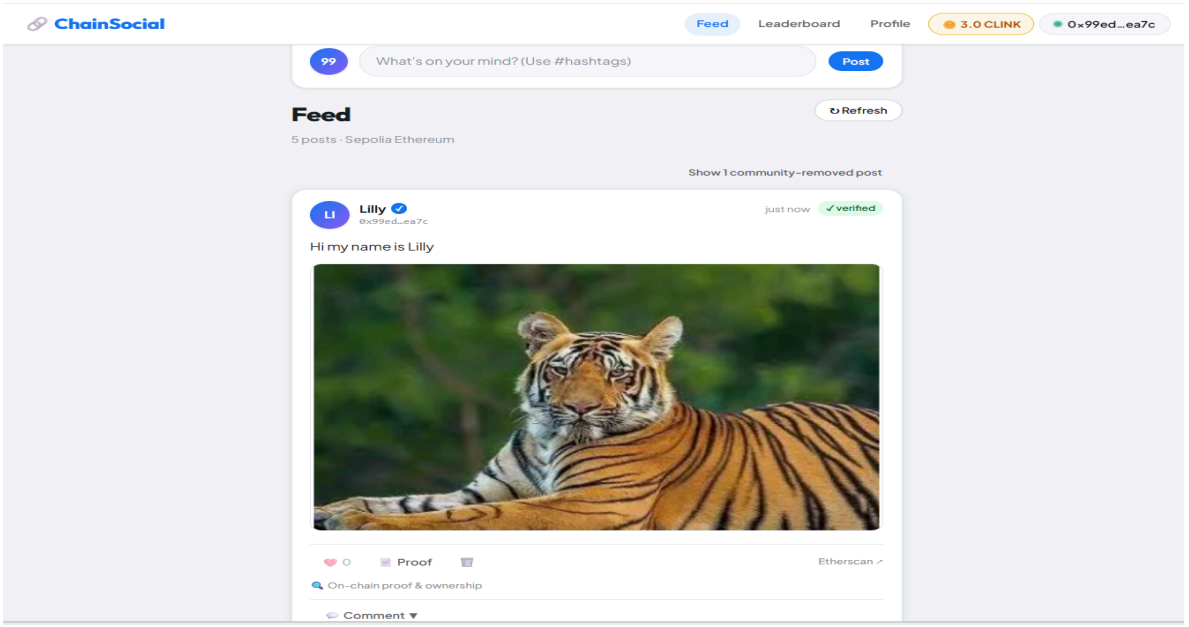
  module.exports = { uploadJSON, uploadFile, fetchFromIPFS, getIPFSUrl };
```

OUTPUT

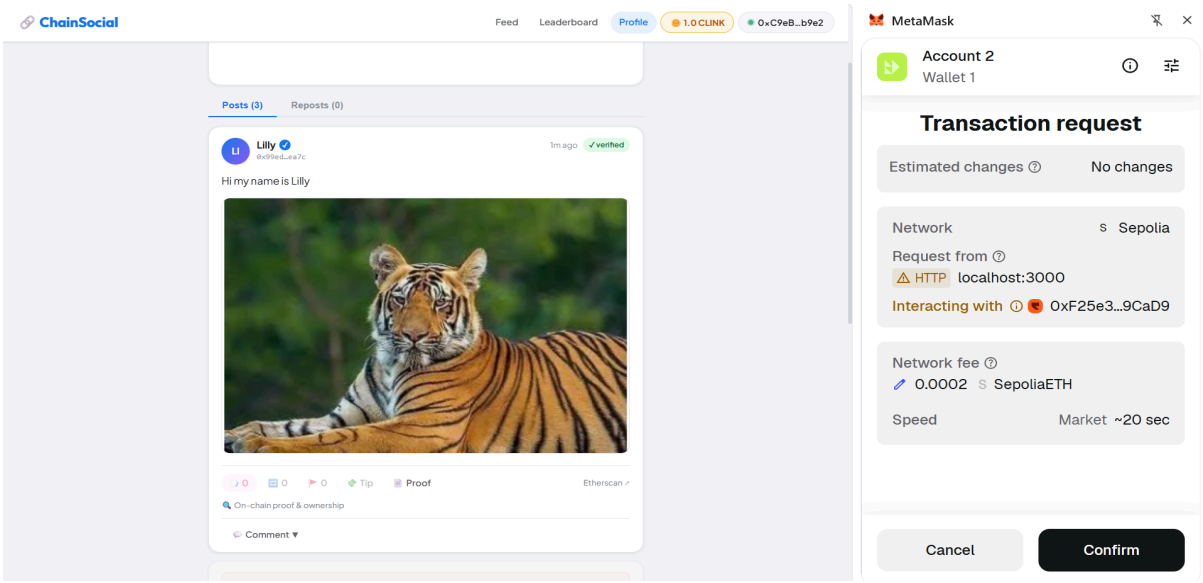
Create post :

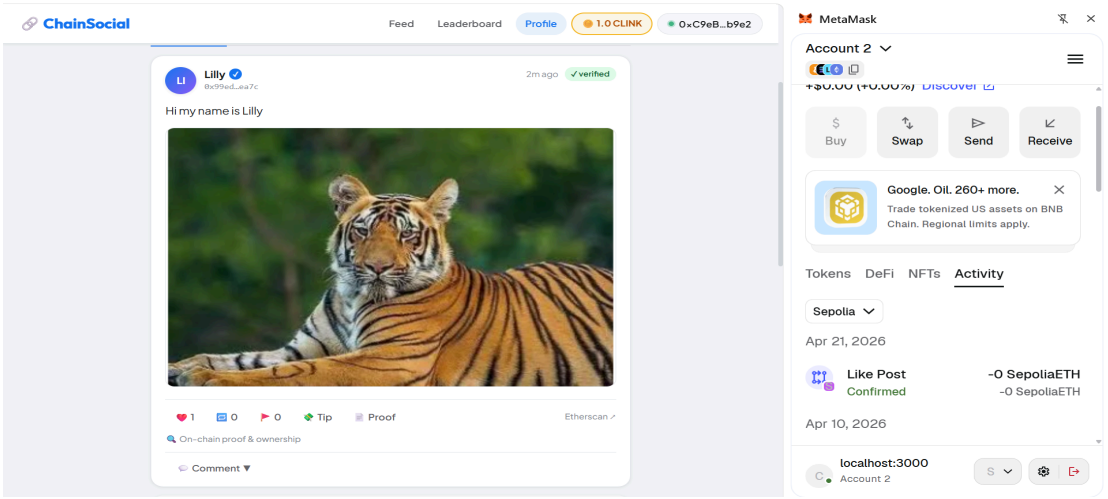




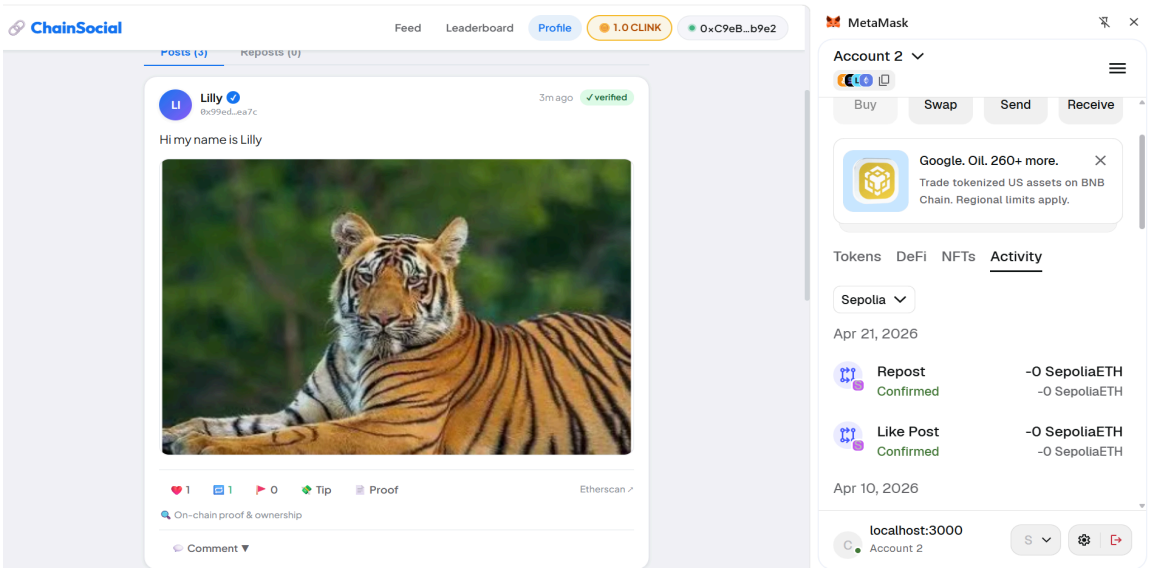


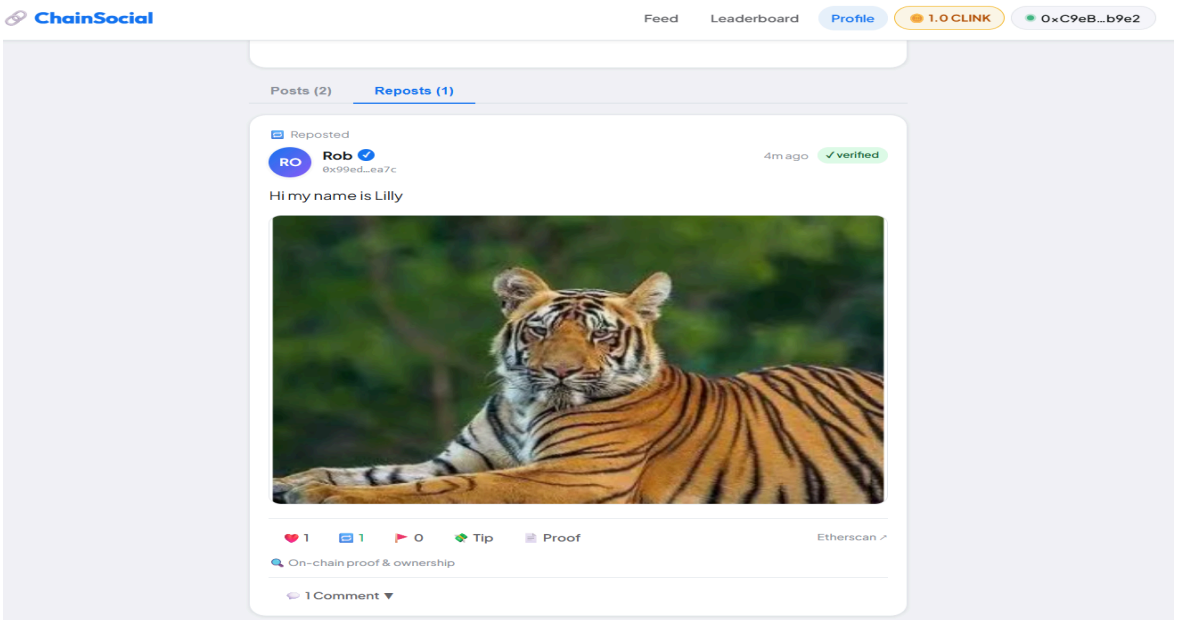
Like post from a different account:



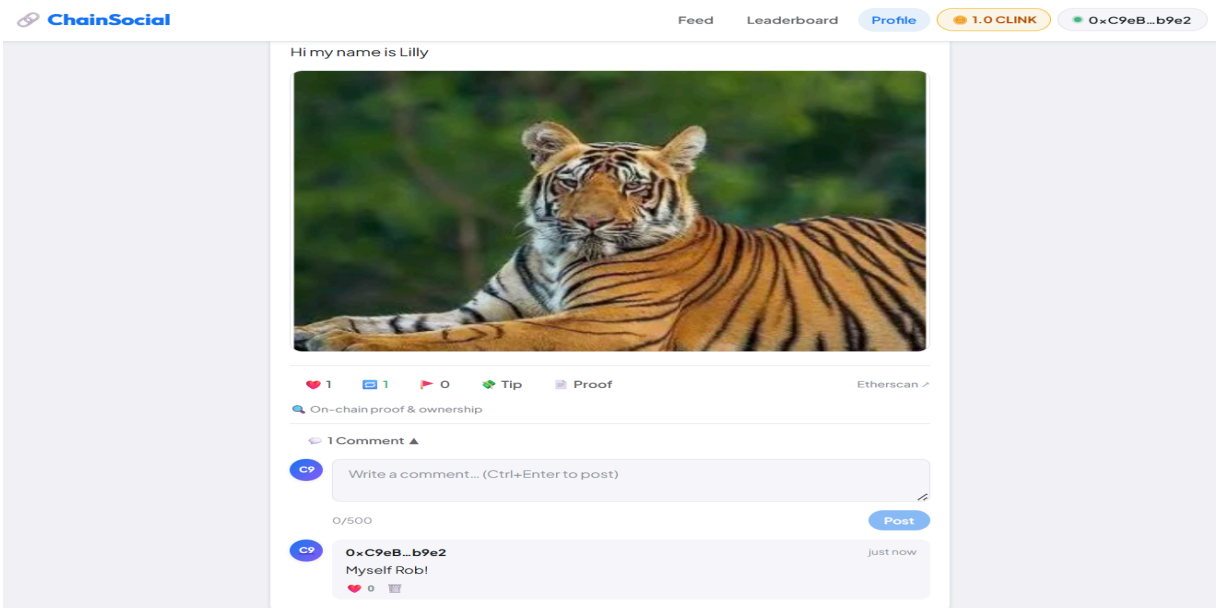

ChainSocial
Feed
Leaderboard
Profile
1.0 CLINK
0xC9eB...b9e2
Lilly
2m ago
verified
Hi my name is Lilly
1
0
0
Tip
Proof
Etherscan
On-chain proof & ownership
Comment
MetaMask
Account 2
Buy
Swap
Send
Receive
Google. Oil. 260+ more.
Trade tokenized US assets on BNB Chain. Regional limits apply.
Tokens
DeFi
NFTs
Activity
Sepolia
Apr 21, 2026
Like Post
Confirmed
-0 SepoliaETH
-0 SepoliaETH
Apr 10, 2026
localhost:3000
Account 2

Repost:

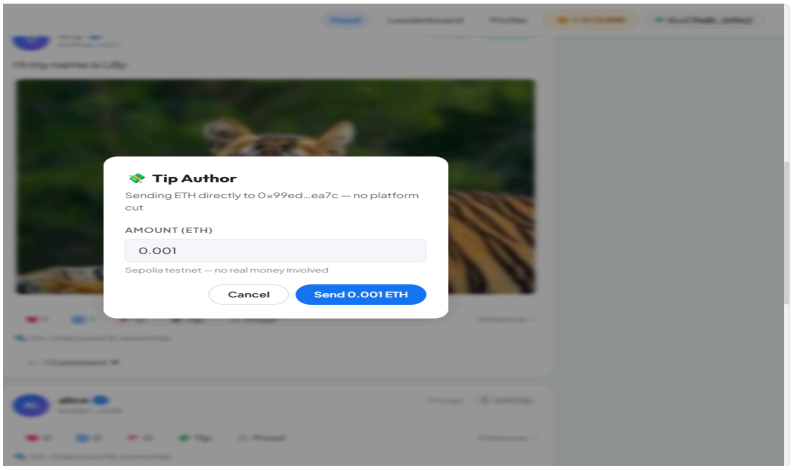

ChainSocial
Feed
Leaderboard
Profile
1.0 CLINK
0xC9eB...b9e2
Posts (3)
Reposts (0)
Lilly
3m ago
verified
Hi my name is Lilly
1
1
0
Tip
Proof
Etherscan
On-chain proof & ownership
Comment
MetaMask
Account 2
Buy
Swap
Send
Receive
Google. Oil. 260+ more.
Trade tokenized US assets on BNB Chain. Regional limits apply.
Tokens
DeFi
NFTs
Activity
Sepolia
Apr 21, 2026
Repost
Confirmed
-0 SepoliaETH
-0 SepoliaETH
Like Post
Confirmed
-0 SepoliaETH
-0 SepoliaETH
Apr 10, 2026
localhost:3000
Account 2

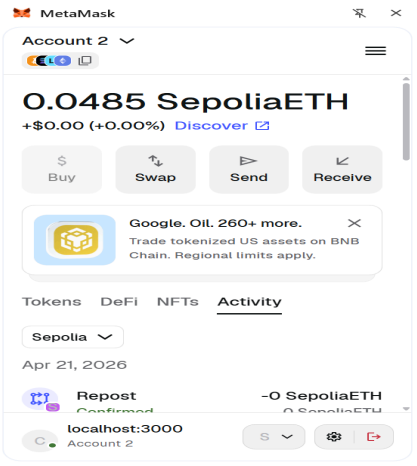


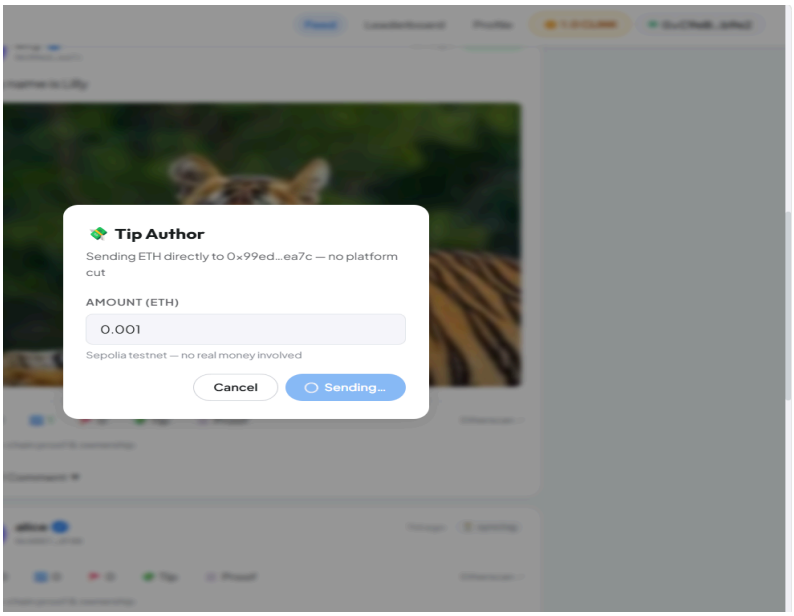
Comment:

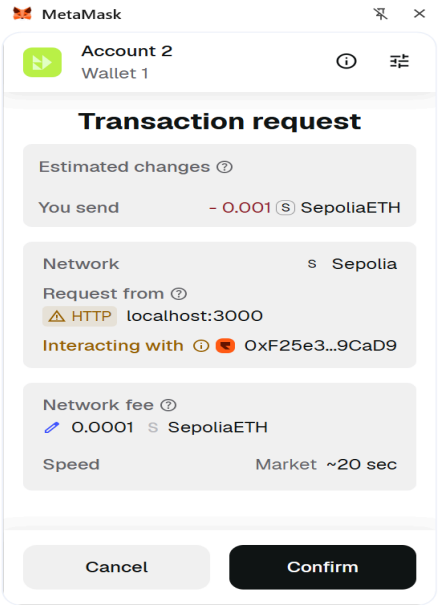


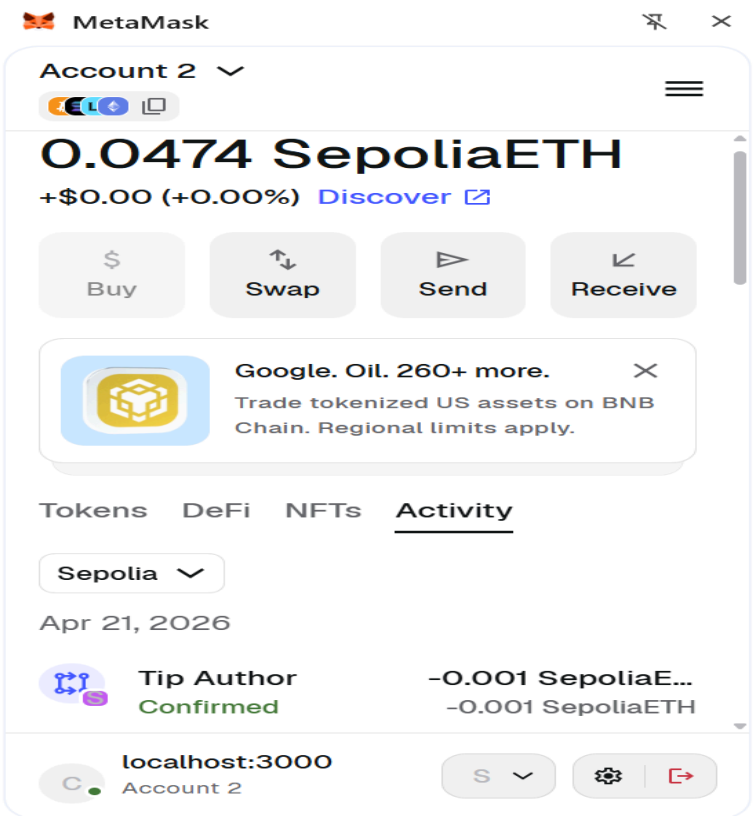
Tip Author :


Tip Author
Sending ETH directly to 0x99ed...ea7c — no platform cut
AMOUNT (ETH)
0.001
Sepolia testnet — no real money involved
Cancel
Send 0.001 ETH

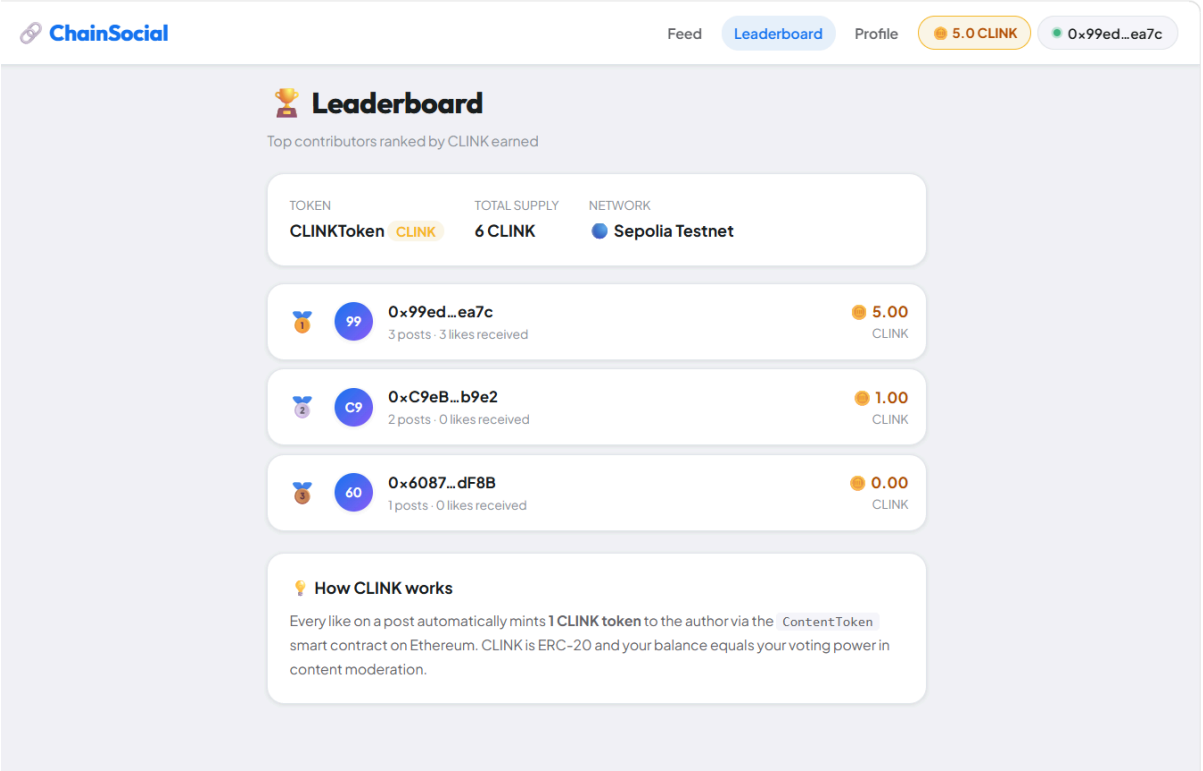

MetaMask
Account 2
0.0485 SepoliaETH
+$0.00 (+0.00%) Discover
Buy
Swap
Send
Receive
Google. Oil. 260+ more.
Trade tokenized US assets on BNB Chain. Regional limits apply.
Tokens
DeFi
NFTs
Activity
Sepolia
Apr 21, 2026
Repost
-0 SepoliaETH
localhost:3000
Account 2


Tip Author
Sending ETH directly to 0x99ed...ea7c — no platform cut
AMOUNT (ETH)
0.001
Sepolia testnet — no real money involved
Cancel
Sending...


MetaMask
Account 2
Wallet 1
Transaction request
Estimated changes
You send
- 0.001 SepoliaETH
Network
Sepolia
Request from
HTTP localhost:3000
Interacting with
0xF25e3...9CaD9
Network fee
0.0001 SepoliaETH
Speed
Market ~20 sec
Cancel
Confirm



Leaderboard: Each like rewards the user with CLINK tokens



Proof of the post on block chain:

Hi my name is Lilly



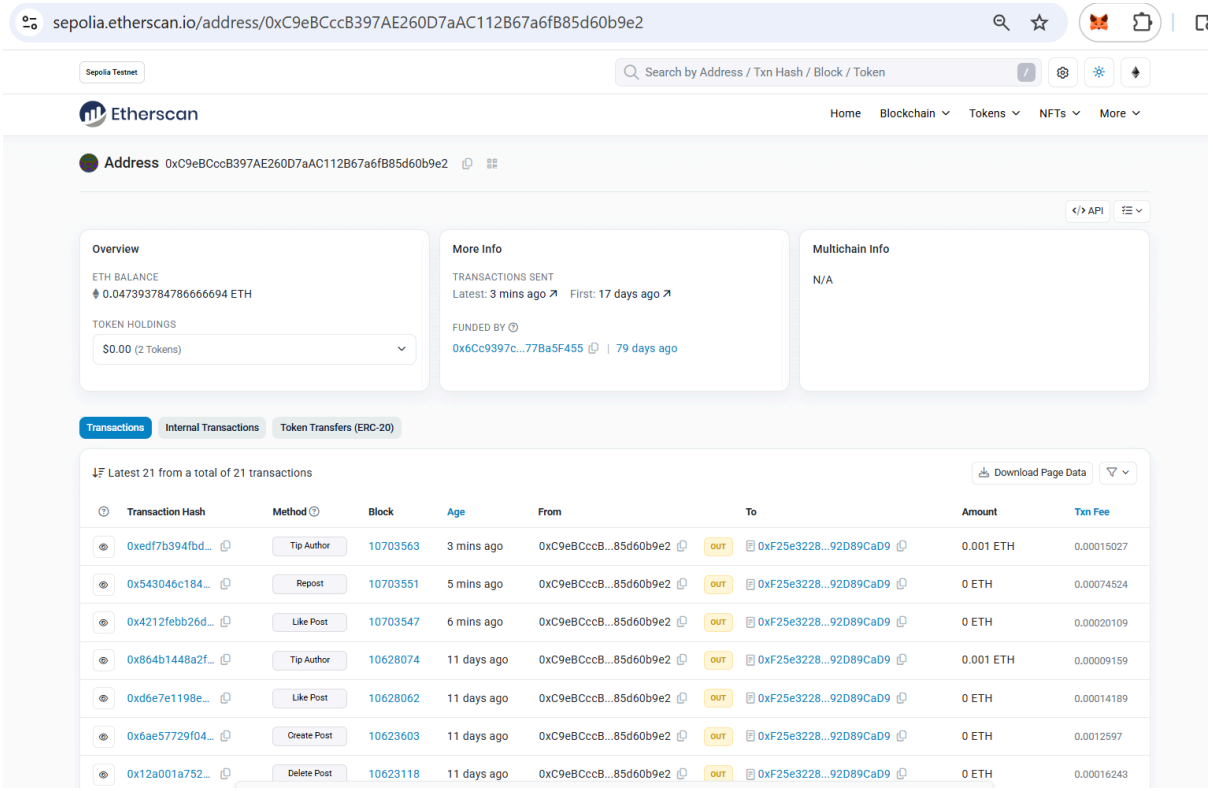
0 0 0 Tip Proof Etherscan ↗

On-chain proof & ownership

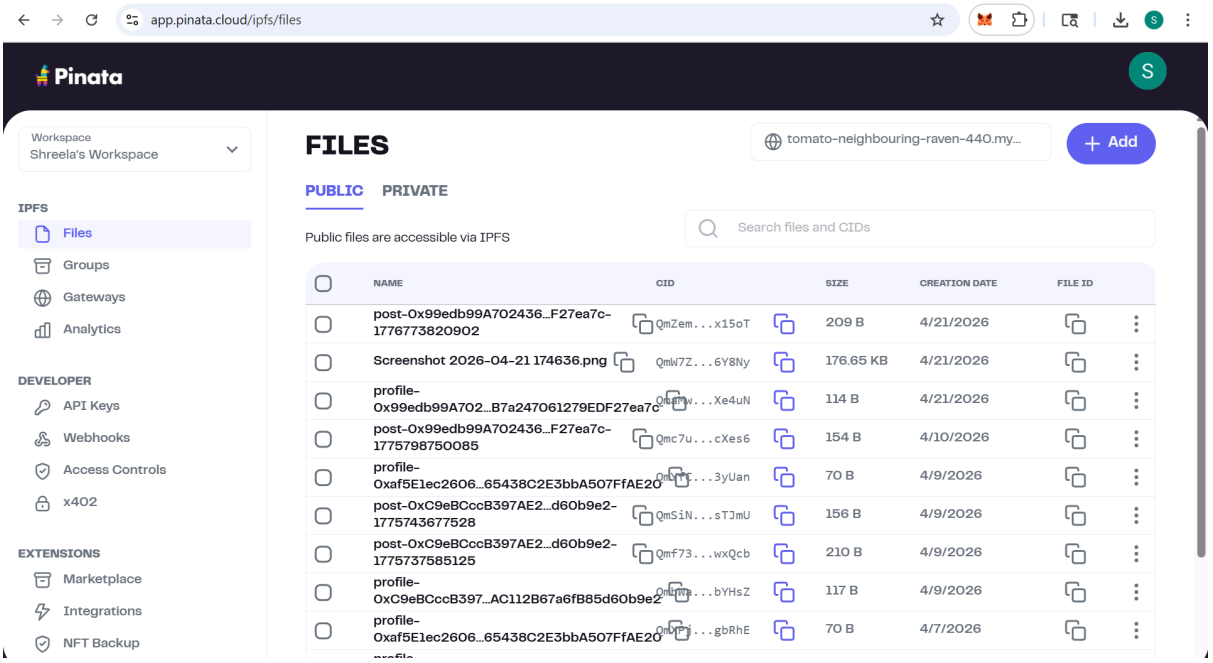
Post ID: #9
Author wallet: 0xC9eBCccB397AE260D7aAC112B67a6fB85d60b9e2
Anonymous: No
IPFS CID: QmZemZJYrEMQDxUDUkAr4YJztyoLamSZGonD4F1Q1x15oT
Content hash: 0x73befaa06d3b47671c7569812cc5b319c0b89320d00e717e5e9251dd51159479
Published: 4/21/2026, 5:50:24 PM
Tags: none
Integrity: ✓ Content matches on-chain hash — original

View on IPFS ↗ View on Etherscan ↗

Etherscan output:



Pinata IPFS output :



Conclusion

Chain.Social demonstrates how decentralized technologies can be effectively used to build a secure and transparent social media platform. By leveraging the Ethereum Sepolia, IPFS, and MetaMask, the system eliminates reliance on centralized servers and gives users full ownership of their data and identity.

The integration of smart contracts ensures that all interactions such as posting, liking, commenting, and tipping are tamper-proof and verifiable. Additionally, the CLINK token introduces a reward-based ecosystem that encourages active participation and community-driven moderation.

Overall, the project successfully showcases a scalable and efficient Web3-based alternative to traditional social media platforms, highlighting the potential of blockchain in building trustless and user-centric applications.